

PROJECT DESIGN DOCUMENT

CausalRoadmap

An LLM-Assisted Causal Inference Exploratory & Estimation Pipeline

Built on the Petersen–van der Laan Causal Roadmap
with hardware-aware local LLM orchestration and comprehensive method
coverage

Author: Brennan Kelley · B.S. Computational Data Science, Utah Valley University

Provisional codename: CausalRoadmap (rename considered — see §32)

Document status: v0.3 — flagging system as LLM foundation, plus v0.2 PhD
methods, dataset-locking, audit, federation

*Predecessor: CausalRAG (TCGA-BRCA precision oncology proof-of-concept, ACIC 2026 poster).
This document specifies the next-generation, domain-agnostic Python package.*

Contents

1. Executive Summary

This document specifies the design of CausalRoadmap, a general-purpose, LLM-assisted causal inference exploratory and estimation package for Python. It is the next-generation successor to the CausalRAG proof-of-concept presented at ACIC 2026 — a precision oncology pipeline that used statistical power as a pre-filter, retrieval-augmented LLM hypothesis generation, a three-layer hallucination prevention gate, and DoWhy + EconML for double machine learning estimation.

The next-generation package generalizes that work along five major axes.

- Domain-agnostic data ingestion from CSV files, Parquet, SQL databases, NoSQL stores, and REST APIs — replacing the original PubMed-specific retrieval stage.
- A two-step Discovery & Domain Expertise phase in which the local LLM acts first as an investigator (extracting per-column meaning) and then as a domain expert (providing context, identifying treatments, confounders, mediators, colliders, instruments, and unmeasured-confounder candidates).
- A flexible feasibility filter (statistical power and sample size) that can be configured manually, calibrated by the LLM, or expanded with data-type-aware checks.
- Two hypothesis pathways — a manual mode for the causal-inference specialist and an automated mode in which the LLM proposes a ranked set of high-impact testable hypotheses (with a configurable share devoted to counterfactual queries).
- Strict adherence to the Petersen-van der Laan Causal Roadmap for every hypothesis: separating the scientific question from the statistical estimation problem through eight rigorously-defined steps.

The package targets three tiers of users with sensible defaults for each: applied data scientists who want power without writing custom code; academic causal-inference researchers who require flexibility and methodological depth; and domain experts with limited statistical background who need guardrails. Comprehensive method coverage spans the modern Python ecosystem (DoWhy, EconML, CausalML, DoubleML, causal-learn, lingam, Tigramite) plus an rpy2 bridge to leading R packages (grf with Causal Survival Forests, ltm, tverse / tmle3 / sl3, sensemakr, MatchIt, marginaleffects, bnlearn, mediation, survRM2).

Local LLM orchestration uses Ollama exclusively. Hardware detection at install time chooses the largest reasoning-capable open-weights model the user's machine can run, with 16 billion parameters as the floor — meaning a DeepSeek-R1 14-16B-class distillation, Qwen3-14B, or equivalent. The package never requires a cloud API key.

The deliverable for v0.1 is a working pipeline that takes a raw dataset and a one-sentence research question, produces a domain-expert profile, generates ranked hypotheses, runs the eight-step Causal Roadmap for each, and emits a reproducible HTML report. The deliverable for v1.0 is a full ecosystem of estimators, sensitivity

tools, and a plugin API rivaling DoWhy + EconML in coverage while uniquely offering end-to-end LLM-assisted exploration.

2. Vision & Design Principles

CausalRoadmap exists because the gap between observational data and credible causal claims is widening, not narrowing. Modern tabular datasets carry hundreds of columns whose meaning is ambiguous; the analyst who once spent a week understanding the data now skips that step or substitutes intuition. The package treats data discovery as a first-class problem and treats causal claims as constrained statistical statements that must pass identifiability, positivity, and sensitivity checks before they appear in a report.

2.1 Vision statement

A single Python package that takes any tabular dataset — clinical, financial, behavioral, industrial — and walks a user from raw rows to a published-quality causal analysis without abdicating analyst judgment. The LLM is a structured prior over hypotheses, variable roles, and DAGs; the statistical engine is the verifier and final arbiter.

2.2 Ten design principles

1. **Causal Roadmap as backbone.** Every hypothesis goes through the eight Petersen–van der Laan steps. The scientific question (Step 1) and the statistical estimand (Step 6) are never collapsed. The architecture mirrors the methodology.
2. **LLM proposes, statistics disposes.** Every LLM-generated artifact — variable role, DAG edge, hypothesis triple — is structurally validated, statistically checked, and overridable. The LLM is never the final arbiter of a causal claim.
3. **Local by default.** Ollama is the only required LLM backend. The package runs offline. A cloud LLM is an explicit, opt-in escape hatch reserved for v1.0.
4. **Hardware-aware.** On install the package profiles the user's CPU, GPU, VRAM, and unified RAM, then selects the largest reasoning-capable open-weights model that fits. The hard floor is 16B-class (Qwen3-14B, DeepSeek-R1-14B distillation, or equivalent).
5. **Data-flag-driven dispatch.** Phase 1 discovery emits a typed flag set (`continuous_treatment`, `right_censored_outcome`, `longitudinal`, `mixed_exposure`, etc.) that propagates downstream. Every method in the catalog declares which flag combinations it supports; dispatch is automatic.
6. **Comprehensive method coverage.** Python where possible, R-bridged where necessary. No defensible method should be inaccessible to a Python user.
7. **Triangulation by default.** Multiverse analysis, cross-DAG robustness, and cross-estimator ensembles are not opt-in features — they are the default reporting frame.

8. **Identifiability is a hard gate.** Step 5 of the Causal Roadmap blocks estimation by default if the requested estimand is not identifiable from the chosen DAG. Override requires an explicit flag and is logged prominently in the report.
9. **Tiered defaults, not tiered capabilities.** Data-scientist, academic, and domain-expert tiers differ only in defaults, prompts, and which knobs are surfaced — never in what is reachable.
10. **Honest provenance.** Every LLM call records model digest, prompt version, seed, and timestamp. Every estimator records backend version. Every user override is preserved as a diff against the LLM's original suggestion.

3. Naming & Positioning

The original project is called CausalRAG. That name is already taken by Wang, Han, Singh, Ma, and Chaudhary's ACL Findings 2025 paper, which integrates causal graphs into retrieval. Continuing under the same name invites confusion in conference conversations, citation searches, and PyPI.

The provisional new name for this package is CausalRoadmap, reflecting its faithful implementation of the Petersen–van der Laan eight-step roadmap as the architectural backbone. Alternative candidates considered:

Candidate	Pros	Cons
CausalRoadmap	Direct mapping to the methodology; signals rigor; unique on PyPI	Slightly long; tightly couples brand to one framework
causality-lab	Welcoming, exploratory tone	Generic; many similar names exist
pywhy-explorer	Signals PyWhy ecosystem alignment	Requires PyWhy organizational buy-in; pre-empts namespace
drcausal	Short; doubly-robust + discovery	Cryptic; sounds like a brand of medical software
causalkit	Memorable; suggests breadth	Generic

Final naming will be decided before v0.1 PyPI release; this document uses CausalRoadmap throughout. Section 32 documents the decision criteria.

4. The Causal Roadmap as Architectural Spine

The Causal Roadmap, developed by Maya Petersen and Mark van der Laan at UC Berkeley, is the methodological backbone of CausalRoadmap. It is a cornerstone of the Targeted Learning ecosystem and is widely discussed at modern statistical gatherings such as the American Causal Inference Conference. The Roadmap forces a strict separation between the abstract scientific question and the statistical estimation problem — a separation that becomes architecturally essential when constructing LLM-assisted causal pipelines, because it pinpoints exactly where natural-language reasoning ends and rigorous statistics begins.

4.1 The eight steps

In its canonical form the Roadmap consists of eight steps. Each step appears as an explicit phase in CausalRoadmap's per-hypothesis pipeline. The package never silently skips a step; every hypothesis carries a complete record of its journey through all eight.

Step	Name	Question it answers	CausalRoadmap module
1	Specify the Causal Question	What scientific question are we asking, in plain language?	<code>roadmap.q1_question</code>
2	Define the Causal Model	What do we assume about the data-generating process? (DAG / NPSEM)	<code>roadmap.q2_model</code>
3	Define the Causal Estimand	What counterfactual quantity formalizes the question?	<code>roadmap.q3_estimand</code>
4	Specify the Observed Data	What is actually measured, how, when, with what missingness?	<code>roadmap.q4_data</code>
5	Assess Identifiability	Can the causal estimand be calculated from observed data under our assumptions?	<code>roadmap.q5_identify</code>
6	Define the Statistical Estimand	What statistical functional of the observed-data distribution maps to the causal estimand?	<code>roadmap.q6_statistical_estimand</code>
7	Estimate the Statistical Estimand	What estimator do we apply, and what are its statistical properties?	<code>roadmap.q7_estimate</code>
8	Interpret the Results	What do the numbers mean back in the language of the question, and how robust are they?	<code>roadmap.q8_interpret</code>

4.2 Why this matters architecturally

LLM-assisted causal pipelines are particularly vulnerable to a specific failure mode: conflating the scientific question with whatever statistical procedure is convenient. An LLM, asked "does X cause Y," will happily produce a regression coefficient, a Pearson correlation, or a propensity-matched difference and call any of them a causal effect. The Roadmap is the most precise antidote in the literature.

By organizing the per-hypothesis pipeline as a literal Roadmap walk, CausalRoadmap enforces the discipline at the architectural level. The Step 1 module accepts natural language and produces a structured causal question; it cannot proceed to estimation. The Step 5 module checks identifiability and either greenlights Step 6 or routes back to Step 1 with a request to revise the question. The Step 6 module produces a statistical functional and is the first module allowed to talk to estimators. The Step 8 module is the only module that re-engages with natural-language interpretation.

The cost of this discipline is that the pipeline is more verbose than a one-shot "estimate the effect" call. The benefit is that every output is a properly conditioned statistical claim, attached to an explicit causal question, with all assumptions surfaced.

4.3 Roadmap-to-module mapping

Each Roadmap step is implemented as a Python submodule under `causalrag.roadmap`. Steps 1, 2, 3, 4, and 8 may invoke the LLM (always with structured-output guards); Steps 5, 6, and 7 are deterministic statistical operations on the artifacts produced by their predecessors.

Step	LLM involvement	Statistical operations	Output object
1	Yes — translate user prompt	None	CausalQuestion
2	Yes — propose DAG, role candidates	Statistical consistency checks (CI tests, fast PC)	CausalModel
3	Yes — pick estimand class	None	CausalEstimand
4	Optional — temporal-order narrative	Deterministic profiling; missingness; censoring detection	ObservedDataSpec
5	No	DoWhy do-calculus; ID algorithm; positivity diagnostics	IdentificationResult
6	No	Functional translation; canonical form	StatisticalEstimand
7	No	Estimator dispatch via data-flag routing	EstimationResult
8	Yes — narrative interpretation	Sensitivity (E-value, sensemakr, Rosenbaum, multiverse)	Interpretation

5. Pipeline Overview

CausalRoadmap reorders the original CausalRAG pipeline so that data discovery and domain expertise are first-class, the feasibility filter is data-type-aware, and the per-hypothesis stage is the full Causal Roadmap. The high-level flow has seven phases.

5.1 The seven phases at a glance

Phase	Name	Inputs	Outputs
0	Initialization	User project name; tier preference	Project skeleton; doctor report
1	Discovery & Domain Expertise	Dataset (any source); optional research question	DiscoveryReport with flags, roles, candidate DAGs, domain context
2	Feasibility Filter	DiscoveryReport	FeasibilityReport — admissible (treatment, outcome) pairs by power and N
3	Hypothesis Pathway	FeasibilityReport; mode (manual / automated); counterfactual ratio	HypothesisQueue — ranked, scoped, falsifiable hypotheses
4	Per-Hypothesis Causal Roadmap	Each hypothesis	Steps 1-8 artifacts; multiverse if enabled
5	Sensitivity & Robustness	Estimation results	E-values; sensemakr; Rosenbaum; multiverse curves; triangulation
6	Reporting & Aggregation	All artifacts	HTML / PDF / Quarto report; BH-adjusted summary table; provenance log

5.2 Phase-to-Roadmap mapping

Phases 0–3 of CausalRoadmap establish the inputs that the Causal Roadmap (Phases 4–6) consumes. Specifically:

- Phase 1 discovery produces the raw material for Step 2 (causal model) and Step 4 (observed data).
- Phase 2 feasibility ensures Step 5 (identifiability) and Step 7 (estimation) can succeed before resources are spent.
- Phase 3 hypothesis generation produces Step 1 (causal question) and seeds Step 3 (causal estimand).
- Phase 4 is the literal Causal Roadmap walk for every hypothesis: eight Roadmap steps per hypothesis, executed in order, with checkpoints between steps.
- Phase 5 corresponds to the sensitivity component of Step 8 (interpretation).
- Phase 6 produces the analyst-facing artifacts that translate Step 8 into deliverables.

6. Phase 0 — Project Initialization & Configuration

The first interaction with the package establishes the project scaffold, profiles the hardware, and selects the LLM stack. This phase exists before any data is loaded and answers a single question: "is this environment ready to do credible causal work?"

6.1 Project skeleton

Running `causalrag init` creates the following structure inside the current directory:

```
my_study/
├── study.causalrag.yaml      # the central StudyProtocol
├── data/                    # input datasets (linked or copied)
├── .causalrag/
│   ├── cassettes/          # cached LLM calls for reproducibility
│   ├── history.jsonl        # every hypothesis ever tested
│   ├── llm.lock             # LLM model digest + config
│   ├── hardware.json        # detected hardware profile
│   └── env.lock             # python + R package versions
├── reports/                 # generated artifacts (html, pdf)
└── README.md
```

6.2 Doctor — environment sanity check

`causalrag doctor` runs an environment audit and prints a structured report. It is the first command to run after install and should be re-run when the environment changes.

The doctor checks:

- Python version (≥ 3.11 required).
- CPU cores (physical and logical) and total RAM via `psutil`.
- GPU presence and VRAM via `pynvml` (NVIDIA), `torch.cuda` backend, or platform-specific probes for Apple Silicon and AMD ROCm.
- Ollama installation and reachability (`GET /api/version`, `/api/tags`).
- R installation, R version, and `rpy2` importability if the `rbridge` extra is requested.
- Required R packages (`grf`, `sensemakr`, `lmtp`, `MatchIt`, `marginalEffects`, `mediation`, `survRM2`, `bnlearn`, `tlverse`) with installation status.
- Disk space available for cassettes and model storage.

With `--install-r-deps` the doctor will programmatically invoke `utils::install.packages` inside R to fetch missing packages from CRAN. With `--install-ollama-models` it will pull the recommended models for the detected hardware tier.

6.3 Hardware tier selection

The doctor's output includes the assigned hardware tier, which drives subsequent model selection. The tiers are described in detail in Section 16; the floor for the package is Tier 2 (12–16 GB VRAM or 32 GB unified RAM), since a 16B-class reasoning model is required for the hypothesis generation stage. Tier 0 and Tier 1 systems can still run discovery and estimation against an external Ollama host, but the doctor will warn that local hypothesis generation will be slow or use a smaller fallback.

6.4 Tier preference

`causalrag init --tier {data-scientist|academic|domain-expert|auto}` sets the default verbosity, prompt style, and which knobs are exposed by default. Tier is editable in `study.causalrag.yaml` and can be overridden per command. The tier affects defaults only; every capability is reachable from every tier.

7. Phase 1 — Data Discovery & Domain Expertise Acquisition

Phase 1 is the most architecturally important contribution of this package. It replaces the manual data-prep step that derails most observational causal analyses with a structured, LLM-assisted, statistically-grounded discovery workflow. The phase has five sub-stages.

7.1 Stage 1a — Multi-source data ingestion

The data loader is implemented in `causalrag.data.connectors` with a uniform Connector protocol. Every supported source must implement `to_arrow()` (returning a `pyarrow Table`), `describe()` (returning lightweight metadata), and `supports_lazy()`. The connectors shipped with v0.1:

Source	Reader	Notes
CSV / TSV	<code>pyarrow.csv</code> with type inference; <code>pandas</code> fallback	Type sniffing; UTF-8 only by default
Parquet	<code>pyarrow.parquet</code>	Predicate pushdown supported; partitioned datasets supported
Feather / Arrow IPC	<code>pyarrow.feather</code>	Native zero-copy
JSON / JSONL	<code>pyarrow.json</code>	Auto-flattens nested records; warns on heterogeneous schemas
Excel (.xlsx)	<code>openpyxl</code> or <code>calamine</code>	Sheet selection; merged-cell warnings
SQL (Postgres, MySQL, MSSQL, SQLite, DuckDB, Snowflake)	<code>SQLAlchemy</code> + <code>connectorx</code> when fast	Read-only by default; explicit allow-list of queryable tables; LIMIT enforced on preview
MongoDB	<code>pymongo</code> + <code>flatten</code>	Document flattening with schema inference over a sample
REST APIs	<code>httpx</code> + JSONata-style mapper	Pagination handling; rate-limit-aware
Hugging Face Datasets	<code>datasets</code>	Direct hub access for public benchmarks
FHIR (clinical)	<code>fhirpy</code>	v0.5+; flattens resources to tidy frames
Stata / SAS / SPSS	<code>pyreadstat</code>	Preserves variable labels and value labels — used by LLM discovery

Loaders are lazy-imported; missing optional dependencies surface as actionable errors. All loaders normalize to Arrow first; downstream stages consume `pyarrow.Table` or `pandas.DataFrame` interchangeably.

7.2 Stage 1b — Deterministic statistical profiling

Before the LLM sees anything, the package profiles the dataset deterministically. The profiler is fast (designed to complete in under 10 seconds for datasets up to 10

million rows on commodity hardware) and produces a structured ColumnProfile for every column. Profiles drive both the LLM prompts and the feasibility filter.

For every column the profiler records:

- Inferred logical dtype (binary, ordinal, categorical, continuous, count, time-to-event, date, datetime, identifier, text, geometry).
- Cardinality, unique-value sample (top 20 with frequencies), missingness rate and pattern.
- For continuous: mean, SD, quantiles (p5, p25, p50, p75, p95), skewness, kurtosis, count of outliers by Tukey-fence rule.
- For categorical: mode, entropy, evidence of imbalance, plausible ordinal ordering.
- Heuristic detection of: identifier columns (high uniqueness, low variability of length); date columns (string parses with > 80% success); duration columns (numeric with units in column name or values clustered around plausible day/week/month counts); censoring indicators (paired columns of the form time/event, where the latter is 0/1).
- Cross-column structure: pairwise correlations above a threshold; perfect collinearity warnings; duplicate-key detection across joined frames.

The output of Stage 1b is a compact JSON document, ~5–20 KB for typical datasets, which becomes the primary context for Stage 1c.

7.3 Stage 1c — Column semantic understanding (LLM, investigator role)

With the deterministic profile in hand, the LLM is invoked in its first role — the investigator. It receives:

- The Stage 1b ColumnProfile JSON.
- The first 10 rows of the dataset (sampled to balance categorical levels).
- The optional user research question.
- Any user-provided data dictionary or codebook (if shipped alongside the dataset).

It is instructed to emit, per column, the following structured fields:

-
- **domain_meaning** — a 1–3 sentence plain-language explanation of what the column measures, what units, how it would normally be collected.
- **domain_tag** — inferred domain (clinical, financial, marketing, education, manufacturing, social-science, web-analytics, environmental, other).
- **value_interpretation** — for categorical columns, an explanation of each modal value ("M = male, F = female"); for ordinal, the implied ordering.
- **temporal_position** — "baseline / pre-treatment / treatment-era / post-treatment / outcome / unknown" based on column name and domain context.

- **watch_for** — a list of warning tags: high_missing, low_variance, suspected_identifier, suspected_target_leakage, derived_from_outcome, post_treatment_proxy, contains_pii.

Every response is validated against a Pydantic schema. Invalid responses are retried up to twice with the validation error as feedback; persistent failures fall back to flagging the column as unknown and surfacing it for analyst review.

Privacy and safety: by default the package does not transmit raw row content to any external service. Because the LLM runs locally via Ollama, the row sample is processed in-process. If the [online] extra is installed and explicitly enabled, the user is shown which rows would be transmitted and is required to confirm.

7.4 Stage 1d — Data-flag emission (routing brain)

Stage 1d is the routing brain of the entire package. The flag set produced here determines which causal methods are even considered downstream. Flags are emitted from both the deterministic profiler (Stage 1b) and the LLM-interpreted profile (Stage 1c), with the profiler having veto authority.

The complete flag taxonomy is described in Section 15; an abbreviated preview:

Flag	Trigger	Downstream consequence
binary_treatment	Treatment column with exactly two non-missing values	Enables: IPW, AIPW, PSM, all DML variants, T/S/X learners
continuous_treatment	Treatment column with > 2 levels and continuous dtype	Enables: lmtpt, txshift, generalized propensity; disables: vanilla PSM
mixture_exposure	Multiple treatment columns implicated in the hypothesis	Enables: lmtpt; disables: most single-treatment DML
right_censored_outcome	Paired time/event columns detected	Enables: Causal Survival Forest, RMST, IPCW-DML; disables: vanilla DML on raw days
longitudinal	Time-varying covariates or repeated measurements detected	Enables: lmtpt longitudinal, g-formula, MSM; disables: cross-sectional methods
instrumental_candidate_present	LLM flagged a column as plausible IV	Enables: 2SLS, DeepIV, OrthoIV, MR-PRESSO if appropriate
competing_risks	Multiple terminal event types detected	Enables: Fine-Gray, cause-specific hazards
high_dimensional	$p > \sqrt{n}$ after one-hot encoding	Forces: regularized nuisance estimators; enables: sparse DML
small_sample	$n < 200$	Enables: exact tests, permutation; disables: aggressive ML nuisance
heavy_missingness	Any required column > 20% missing	Forces: multiple imputation pre-step; or routes to MNAR-aware methods
panel_structure	Subject-id × time tidy structure detected	Enables: DiD, synthetic control, panel DML

Flag	Trigger	Downstream consequence
single_treated_unit	Exactly one treated unit detected	Enables: synthetic control, CausalImpact; disables: standard ATE

7.5 Stage 1e — Domain expert context generation (LLM, expert role)

Stage 1e is the LLM's second role — domain expert. With the column-level investigation complete, the LLM is reinvoked at a higher level of abstraction. It receives the entire Stage 1c+1d output and is instructed to write a Domain Expert Brief: a structured document that, in subsequent phases, acts as a system prompt anchor.

The Domain Expert Brief contains:

11. A one-paragraph domain summary identifying the apparent study setting.
12. A list of plausible treatment columns, ranked by their suitability for causal-effect estimation and annotated with the typical research questions they would support.
13. A list of plausible outcome columns, with notes on their measurement quality, censoring properties, and clinical/economic interpretability.
14. A list of plausible confounders for each (treatment, outcome) candidate pair, drawn from both the dataset and the LLM's pretrained domain knowledge.
15. A list of plausible mediators and effect modifiers.
16. A list of suspected unmeasured confounders. For each, the LLM names what is missing (e.g., "performance status", "household income"), why it likely matters, and which observed proxies might partially capture it.
17. A list of K candidate DAGs (default K = 3), each in GML, each with a one-sentence rationale and a list of edges that distinguish it from the others.
18. Domain-specific identification warnings (e.g., "in healthcare claims data, treatment is heavily confounded by physician selection").

Every claim in the Brief is statistically validated where possible. Confounder candidates are run through a conditional-independence test on the data; if the test contradicts the LLM's claim with high evidence, the contradiction is logged and the analyst is shown both views. The Brief is the final artifact of Phase 1 and is saved as part of the StudyProtocol.

Why two LLM roles?

The investigator role is narrow and column-focused, producing structured per-column metadata; it is allowed to hallucinate only at the cost of being trivially caught by schema validation. The domain-expert role is broader and synthesizes across columns; it would be much harder to validate if we mixed it with column-level extraction. Splitting the roles also lets us route the investigator to a smaller general-purpose model and the expert to the largest reasoning model the hardware supports.

8. Phase 2 — Feasibility Filter

Phase 2 is the direct descendant of the bounding-box concept from the original CausalRAG: a deterministic, pre-LLM filter that prevents the package from pursuing hypotheses for which the data is fundamentally inadequate. Generalized to support every flag combination, it acts as a discipline on Phase 3.

8.1 What the filter checks

For each candidate (treatment, outcome) pair surfaced in the Domain Expert Brief, the filter computes:

- Sample size in each treatment arm (or sufficient density across the continuous treatment range).
- Marginal positivity: are there enough units exposed and unexposed?
- Minimum detectable effect (MDE) under the appropriate estimand, given the flagged data type.
- Statistical power against a plausible effect band — either a user-supplied band or an LLM-suggested band drawn from the domain expert's pretrained knowledge.
- Coverage of plausible effect modifiers — if subgroup CATEs are being considered, do the subgroups exist with sufficient n?
- Censoring rate (for survival outcomes) — if > 90%, downstream methods will be marked "signal-only, magnitude untrustworthy".

8.2 MDE calculations by data flag

Flag combination	MDE calculator	Notes
binary_treatment + continuous_outcome	Two-sample Cohen's d → statsmodels TTestIndPower	Default; what CausalRAG used in oncology
binary_treatment + binary_outcome	Two-proportion power test	Phi effect size; arcsine-transformed power
binary_treatment + right_censored_outcome	RMST variance (Royston-Parmar closed form)	Calls survRM2 via rpy2
continuous_treatment + continuous_outcome	Pearson r-equivalent on residualized form, or GPS-based	Dosing-curve detection if monotone
binary_treatment + count_outcome	Negative-binomial GLM power simulation	Pre-tabulated for common parameter ranges
CATE in subgroup	Effective subgroup n × subgroup effect heterogeneity	Conservative — uses 0.5 × MDE of the full ATE as default
mixture_exposure	Sample-size-only check; full MDE not analytic	Falls back to simulation when n < 1000
IV setting	F-statistic of first stage × Wald estimator variance	Requires the IV candidate column

Flag combination	MDE calculator	Notes
mediation	Joint-significance Monte Carlo (Imai et al.)	Cached lookup table for common NDE/NIE scenarios
panel / DiD	Within-unit variance + treatment timing variation	Estimated via the Goodman-Bacon decomposition
single_treated_unit (synth control)	Pre-period length × outcome volatility	Heuristic; reported with wide uncertainty

8.3 LLM-adjusted thresholds

The original CausalRAG used fixed thresholds ($N \text{ treated} \geq 120$, $\text{power} \geq 0.70$ at $\alpha = 0.05$). CausalRoadmap supports three modes of threshold setting:

19. **Default.** Conservative defaults: $N \text{ per arm} \geq 100$, $\text{power} \geq 0.70$ at $\alpha = 0.05$. Sensible for most settings; reproduces the CausalRAG spirit at slightly looser thresholds.
20. **Manual.** User sets thresholds in the StudyProtocol or via CLI flags (`--n-floor 150 --power-floor 0.80 --alpha 0.025`).
21. **LLM-calibrated.** The LLM receives the dataset profile and Domain Expert Brief and proposes thresholds tailored to the domain. Clinical-trial-style data gets stricter thresholds ($\text{power} \geq 0.80$, α adjusted for multiple testing); exploratory web-analytics data gets looser thresholds. The LLM's proposal is shown to the user and requires confirmation by default in the academic tier; the data-scientist tier accepts it automatically.

8.4 Plausible effect bands

Power depends on the assumed true effect size. Three sources of effect bands:

- User-supplied band: explicit minimum and maximum clinically-meaningful effect, possibly with prior-RCT references.
- LLM-suggested band: the domain expert proposes a band based on pretrained knowledge of the domain literature (e.g., "weekly paclitaxel in BRCA: 0.20–0.40 Cohen's d-equivalent on OS-days").
- Empirical band: bootstrapped estimate from the data itself, used only as a last resort because it is biased toward zero in low-signal regimes.

The chosen band and its source are recorded in the StudyProtocol; the report cites both.

8.5 Output: FeasibilityReport

Phase 2 emits a FeasibilityReport that lists, for every (treatment, outcome) pair from the Domain Expert Brief:

- Whether the pair passes all gates (PASS / SOFT_PASS with warnings / FAIL with reasons).
- The MDE achieved and the effect band used.

- Suggested estimand types that are appropriate (ATE, ATT, CATE, RMST, NDE/NIE, LATE).
- Required data flags for downstream method dispatch.
- Specific deficiencies that would unlock additional methods if remedied (e.g., "add 50 more controls to unlock the strict threshold").

The FeasibilityReport feeds Phase 3.

9. Phase 3 — Hypothesis Pathways

Phase 3 produces the queue of testable hypotheses that Phase 4 will process. There are two pathways: manual and automated. They share the same downstream pipeline, so a project can mix manually-supplied hypotheses with automatically-generated ones.

9.1 Manual mode — analyst-driven

In manual mode, the causal-inference specialist supplies one or more candidate hypotheses through one of three interfaces: the CLI (`causalrag hypothesize add --treatment X --outcome Y --question "..."`), the YAML protocol file, or the interactive REPL (`%hyp add`).

For each manually-supplied hypothesis the package performs three checks:

22. **Feasibility lookup.** Does this (treatment, outcome) pair exist in the FeasibilityReport? If yes, attach the feasibility metadata. If no, run the feasibility filter for this specific pair and surface the result.
23. **Identifiability preview.** Run DoWhy's `identify_effect` against the current rank-1 DAG. If the requested estimand is non-identifiable, return a structured "what's missing" report: which variables (named by domain meaning) would have to be measured or which assumptions would have to be modified for the hypothesis to be testable. This is the diagnostic the original prompt mentioned.
24. **LLM critique.** The LLM, acting as the domain expert, returns a one-paragraph critique of the hypothesis: is it specific enough, is the estimand class right, is there a known confounding hazard, would a CATE be more informative than an ATE?

Manual hypotheses are queued whether or not they pass all checks; failing hypotheses are flagged but still run through Phase 4 so the analyst sees the explicit Step 5 (identifiability) report. This is intentional: refusing to run them would hide the very diagnostic the analyst needs.

9.2 Automated mode — LLM-driven, impact-ranked

In automated mode, the LLM enters a generation loop. The loop is configured by:

- `N` (default 10) — the total number of hypotheses to produce.
- `counterfactual_ratio` (default 0.3) — the share of hypotheses that should be counterfactual queries ("What would Y have been under $X = 1$?",) versus straightforward effect-size questions ("What is the ATE of X on Y?").
- `estimand_distribution` — preferred mix across ATE, ATT, CATE, RMST, NDE/NIE.
- `novelty_bias` — penalty applied to hypotheses similar to those already in the history.

- `temperature_schedule` — start at 0.4 for the highest-impact hypotheses, increase to 0.7 for the tail to encourage diversity.

9.2.1 Generation loop

25. **Seed.** Build the prompt context: FeasibilityReport, Domain Expert Brief, hypothesis history (already-tested hypotheses), data flags, and the user's optional research question.
26. **Propose.** Ask the LLM to propose a batch of $M = N \times 2$ candidate hypotheses. Each candidate must specify treatment, outcome, optional effect modifiers, estimand class, expected direction, expected magnitude, and a one-sentence impact rationale ("why does answering this matter?").
27. **Validate.** Drop candidates that reference phantom variables, fail feasibility, propose unidentifiable estimands, or are structural duplicates of history entries.
28. **Score.** Each surviving candidate gets a scalar impact score combining: (a) LLM-rated importance, (b) statistical power achieved under the assumed effect, (c) novelty vs. history, (d) coverage of the treatment $\times$ outcome $\times$ modifier space, (e) preference for specific over general (CATE > ATE when defensible).
29. **Rank and select.** Maximal-marginal-relevance reranker enforces coverage; pick the top N. Enforce the `counterfactual_ratio` by re-balancing if the natural top-N is skewed.
30. **Emit.** The selected hypotheses are emitted as a HypothesisQueue, each annotated with its score components and provenance.

9.2.2 The counterfactual share

Counterfactual questions (e.g., "What would the mortality rate have been if every patient had received Drug A instead of Drug B?") are scientifically distinct from effect-size questions and call for different statistical treatments. CausalRoadmap surfaces the distinction explicitly because it is the single most common point of confusion in applied causal work.

The `counterfactual_ratio` parameter (default 0.30) controls what share of the hypothesis queue is devoted to counterfactual queries versus effect-size queries. Counterfactual hypotheses route preferentially to estimators that produce full counterfactual distributions (Imtp, g-computation, BCF) rather than estimators that produce a single average (LinearDML, simple AIPW). The default 0.30 share reflects the empirical balance in modern applied work and can be raised to 1.0 for explicitly counterfactual studies (e.g., policy simulation) or lowered to 0.0 for pure-discovery exploration.

9.3 Impact scoring details

The impact score is computed as a weighted geometric mean of normalized component scores in $[0, 1]$:

```
Impact = w_imp · LLM_importance
        + w_pow · Power_achieved
        + w_nov · Novelty
        + w_cov · Coverage_contribution
        + w_spec · Specificity_bonus
        + w_iden · Identifiability_confidence
```

```
Defaults: w_imp = 0.30, w_pow = 0.20, w_nov = 0.15,
          w_cov = 0.15, w_spec = 0.10, w_iden = 0.10
```

All weights are exposed in the StudyProtocol for tuning. The academic tier defaults to weights that favor identifiability and power; the data-scientist tier weights LLM importance and coverage more heavily.

9.4 Hybrid mode

In practice most projects mix the two pathways. A specialist supplies three or four hypotheses they care about; the LLM fills the remaining slots with high-impact suggestions. The HypothesisQueue is a single ordered list with provenance flags (manual / automated / overridden) on each entry.

10. Phase 4 — Causal Roadmap Per Hypothesis

Phase 4 is the methodological core of CausalRoadmap. For every hypothesis in the queue, the package executes the eight steps of the Petersen-van der Laan Causal Roadmap in order. Steps cannot be skipped; the artifacts of each step are persisted; failures are surfaced rather than swallowed.

This section documents each step in terms of inputs, processing, outputs, and which modules of the package are responsible.

10.1 Step 1 — Specify the Causal Question

Inputs

- A hypothesis entry from the HypothesisQueue (manual or automated).
- The Domain Expert Brief from Phase 1.
- Optional analyst clarifications captured in the REPL or YAML.

Processing

The LLM produces a structured CausalQuestion object with five fields:

- **exposure** — the hypothetical intervention in domain language (e.g., "prescribing weekly paclitaxel at first-line for stage II breast cancer").
- **comparator** — the counterfactual condition (e.g., "no paclitaxel" or "standard-of-care chemotherapy without taxanes").
- **outcome** — the response of interest in domain terms (e.g., "overall survival measured in days from diagnosis").
- **target_population** — the population the answer is intended to generalize to.
- **time_horizon** — for time-to-event outcomes, the relevant horizon.

The CausalQuestion is intentionally non-mathematical. It is the language a clinician, economist, or product manager would use in conversation.

Output

10.2 Step 2 — Define the Causal Model

Inputs

- CausalQuestion from Step 1.
- Domain Expert Brief: candidate DAGs, suspected unmeasured confounders, role candidates.

Processing

The Causal Model is encoded as a Non-Parametric Structural Equation Model (NPSEM) or, in the more common simplified form, a DAG. The package supports three encodings:

31. **GML graph (default)** — directly consumable by DoWhy; nodes typed with their causal role; latent (unmeasured) nodes labeled with the U prefix.
32. **PyWhy NetworkX ADMG** — for graphs with bidirected (latent confounder) edges; consumed by Ananke and pywhy-graphs.
33. **NPSEM specification** — for the academic tier and tlverse compatibility; a list of structural equations with their argument variables.

If multiple candidate DAGs survive from Phase 1, the analyst may select one or run all of them as a cross-DAG multiverse. The default is to run the rank-1 DAG and emit a warning if rank-2 disagrees materially.

Step 2 also runs deterministic statistical checks on the chosen DAG:

- Implied conditional independencies are tested on the data; violations are reported but do not block (DAGs are model-of-the-world, not data-fits).
- Acyclicity is enforced.
- Variable existence: every node in the DAG either resolves to a dataset column or is labeled as latent.

Output

CausalModel — graph object plus a manifest of which nodes are observed vs. latent.

10.3 Step 3 — Define the Causal Estimand

Inputs

- CausalQuestion (Step 1) and CausalModel (Step 2).
- Counterfactual flag from the hypothesis.

Processing

Step 3 translates the natural-language question into a formal counterfactual quantity using potential outcomes. The estimand catalog includes:

Estimand	Formal definition	When used
ATE	$E[Y(1) - Y(0)]$	Default for binary treatment, no specified subgroup
ATT	$E[Y(1) - Y(0) \mid T = 1]$	When the policy question concerns the treated
ATC / ATU	$E[Y(1) - Y(0) \mid T = 0]$	Rare; "what would untreated have gained?"
CATE	$E[Y(1) - Y(0) \mid X = x]$	Effect heterogeneity by modifier X
LATE	Local ATE among compliers	IV setting
RMST contrast	$E[\min(T_{\text{surv}}, \tau) \mid A=1] - E[\min(T_{\text{surv}}, \tau) \mid A=0]$	Survival outcomes, censored
NDE / NIE	Natural direct / indirect effect through mediator M	Mediation analysis

Estimand	Formal definition	When used
Counterfactual distribution	$F_{\{Y(a)\}}$ for all $a \in A$	Stochastic interventions; Imtp
Modified treatment policy	$E[Y(d(A,W))]$ for shift function d	Continuous / mixture exposures; Imtp
Counterfactual quantile	$Q_{\tau}(Y(1)) - Q_{\tau}(Y(0))$	Distributional effects

The estimand class is recorded in canonical form and reused downstream. If the original hypothesis was counterfactual, the chosen estimand prefers full-counterfactual-distribution targets (e.g., Imtp) where data flags allow.

Output

CausalEstimand — typed object with class, treatment/outcome/modifier names, and the formal expression.

10.4 Step 4 — Specify the Observed Data

Inputs

- The dataset from Phase 1, restricted to the columns relevant to this hypothesis.
- Phase 1 column profiles.

Processing

Step 4 emits an *ObservedDataSpec* describing exactly what is measured and how. It is the LLM-generated narrative companion to the column profile, restricted to the variables touched by this hypothesis:

- Variable list with logical types, units, value ranges.
- Missingness rates and inferred mechanism (MCAR / MAR / MNAR) based on patterns and LLM judgment.
- Temporal ordering: which variables are pre-treatment baseline, which are time-varying, which are post-treatment, which are outcomes.
- Sample size, including any joins or filters that reduce the analytic sample below the raw dataset size.
- Censoring properties if applicable: censoring rate, evidence for non-informative vs. informative censoring.

Step 4 is the moment of truth for any temporality violations. If a variable the LLM proposed as a confounder is in fact post-treatment, this step catches it and routes back to Step 2 for a model revision.

Output

ObservedDataSpec — fully-typed, machine-readable description of the analytic dataset for this hypothesis.

10.5 Step 5 — Assess Identifiability

Inputs

- CausalModel (Step 2), CausalEstimand (Step 3), ObservedDataSpec (Step 4).

Processing

Step 5 is the most rigorous gate in the pipeline. It is the moment where the package decides whether the causal estimand can be computed from the observed data under the assumed causal model.

Identification is checked algorithmically:

34. **DoWhy do-calculus.** Apply back-door, front-door, and instrumental-variable criteria to the DAG. Return the identified expression if successful.
35. **ID algorithm.** For ADMGs with latent confounders, invoke the Shpitser-Pearl ID algorithm (via Ananke or a Python port). This covers identification settings beyond simple back-door adjustment.
36. **Untestable assumption surfacing.** Enumerate the assumptions on which identification rests — conditional exchangeability, positivity, consistency, SUTVA, exclusion restriction (IV), and any DAG-specific assumptions. The list is preserved verbatim into the report.
37. **Positivity diagnostic.** Fit a quick propensity model on the proposed adjustment set; check that propensity scores are bounded away from 0 and 1 with sufficient overlap in each stratum. Marginal and conditional positivity are checked separately.
38. **Failure handling.** If identification fails, Step 5 emits a structured "what's missing" report: which assumptions were violated, which additional measurements would restore identifiability, which alternative estimands (e.g., partial-identification bounds) might still be defensible. The hypothesis can either be abandoned, modified (return to Step 1), or proceed under explicit override with the failure logged.

Output

IdentificationResult — identification expression (or failure reason), list of untestable assumptions, positivity diagnostic, and decision flag.

10.6 Step 6 — Define the Statistical Estimand

Inputs

- IdentificationResult (Step 5).
- ObservedDataSpec (Step 4).

Processing

Step 6 translates the identified causal expression into a purely statistical functional of the observed-data distribution. For example, under conditional exchangeability the ATE becomes:

$$\psi(P) = E_W[E[Y | A = 1, W] - E[Y | A = 0, W]]$$

where P is the observed-data distribution and W is the adjustment set identified by the back-door criterion in Step 5.

Step 6 is the first moment where the package's universe is purely statistical: there is no more reference to potential outcomes, only to expectations of observed quantities under the empirical distribution. This separation matters because it makes the subsequent estimation step amenable to ordinary statistical theory — convergence rates, variance estimation, sensitivity bounds — without entangling them with causal assumptions.

Specific translations include:

Causal estimand	Identified statistical estimand under back-door W
ATE	$E[E[Y A=1,W] - E[Y A=0,W]]$
ATT	$E[E[Y A=1,W] - E[Y A=0,W] A=1]$
CATE(x)	$E[E[Y A=1,W=w] - E[Y A=0,W=w] X=x]$
RMST(τ)	$E[\min(T_{\text{surv}}(1), \tau) - \min(T_{\text{surv}}(0), \tau)]$ expressed via censoring-aware functional
NDE	Imai-Keele NDE expression with cross-world independence
MTP (Imtp)	$E[Y(d(A,W))]$ under shift function d

If Step 5 yielded only partial identification, Step 6 produces a pair of statistical functionals corresponding to the lower and upper bounds. Estimation in Step 7 then targets both.

Output

StatisticalEstimand — canonical-form functional with all referenced variables resolved to dataset columns.

10.7 Step 7 — Estimate the Statistical Estimand

Inputs

- *StatisticalEstimand* (Step 6).
- *ObservedDataSpec* (Step 4) and the dataset.
- Data flags from Phase 1.

Processing

Step 7 is where the package's comprehensive method catalog is dispatched. The estimator registry indexes every available method by:

- Supported estimand classes (ATE, CATE, RMST, NDE, ...).
- Required data flags (binary_treatment + observed_confounders, right_censored_outcome, longitudinal, ...).
- Excluded data flags (vanilla DML excludes right_censored_outcome unless wrapped with IPCW).

- Sample-size suitability bounds (some estimators degrade below $n = 200$).
- Backend (Python native or rpy2-bridged).

Dispatch logic finds the candidate estimators that satisfy the estimand $\times$ flag intersection, ranks them by appropriateness (defaults preferring DR estimators with valid inference), and runs them. The data-scientist tier picks the top candidate; the academic tier runs an ensemble of K (default 3) and reports the multiverse.

Every estimator returns a standardized `EstimationResult` containing:

- Point estimate and 95% confidence interval.
- Standard error and p-value (when applicable).
- Sample size used.
- Diagnostics: overlap, balance metrics (max absolute standardized mean difference), nuisance-model fit quality.
- Refutation tests: placebo treatment, random common cause, subset sensitivity (DoWhy's refutation suite).
- Backend version and seed for reproducibility.

If the estimand is partially identified, Step 7 returns a bounded interval rather than a point estimate; estimators that cannot produce bounds are excluded.

Output

EstimationResult (single estimator) or *MultiverseResult* (ensemble).

10.8 Step 8 — Interpret the Results

Inputs

- All artifacts from Steps 1–7.
- Phase 5 sensitivity outputs (computed in parallel).

Processing

Step 8 produces three complementary artifacts:

39. **Natural-language interpretation.** The LLM (in domain-expert role) translates the numerical estimate back into the language of the original CausalQuestion. The translation is constrained to the actual estimate and CI; it cannot claim significance the data did not produce.
40. **Assumption ledger.** An explicit table of every untestable assumption invoked along the way — conditional exchangeability, positivity, SUTVA, IV exclusion, and so on — with sensitivity bounds where computed. The table is the antidote to over-interpretation.
41. **Robustness verdict.** Aggregation of Phase 5 sensitivity outputs: E-value, sensemakr partial R^2 bounds, Rosenbaum Γ , multiverse range. A traffic-light verdict (green / yellow / red) is computed by a defensible rule (e.g., red if E-value < 1.5 and at least one neighboring multiverse spec produces a CI crossing zero).

Output

Interpretation — the artifact a non-statistician reader of the report actually sees first.

10.9 Step-by-step gating

The Roadmap walk enforces hard gates between certain steps:

- Step 1 → Step 2: the CausalQuestion must name an exposure and outcome that resolve to columns or are explicitly hypothetical (allowed only with a warning).
- Step 2 → Step 3: the CausalModel must be acyclic.
- Step 4 → Step 5: every confounder in the DAG must correspond to a measured column (or be explicitly latent).
- Step 5 → Step 6: identification must succeed or the analyst must explicitly accept a partial-identification path.
- Step 6 → Step 7: at least one estimator must exist that supports the data-flag set.
- Step 7 → Step 8: at least one EstimationResult must have been produced; refutations must have run.

Gate failures are surfaced in the report as red banners; they cannot be silently bypassed.

10.10 Per-hypothesis artifacts

By the end of Phase 4, each hypothesis has the following artifact tree:

```
hypothesis_H03/  
├─ q1_question.yaml  
├─ q2_model.gml  
├─ q3_estimand.yaml  
├─ q4_data.yaml  
├─ q5_identification.yaml  
├─ q6_statistical_estimand.yaml  
├─ q7_estimation/  
│   ├─ linear_dml.json  
│   ├─ causal_forest.json  
│   ├─ tmle3.json          # if rbridge  
│   └─ multiverse.json  
├─ q8_interpretation.md  
└─ sensitivity/  
    ├─ evaluate.json  
    ├─ sensemakr.json  
    └─ multiverse_specification_curve.png
```

These artifacts are the source of truth for the final report and for reproducibility.

11. Phase 5 — Sensitivity & Robustness

Phase 5 runs concurrently with Step 7 in Phase 4. It is conceptually part of Step 8 (interpretation) but is large enough to warrant its own phase in the architecture. Sensitivity is not an optional add-on in CausalRoadmap — it is the empirical antidote to the untestable assumptions surfaced in Step 5.

11.1 Sensitivity methods

Method	What it bounds	Reference
E-value (VanderWeele-Ding)	Minimum confounder strength to nullify effect	Ann Intern Med 2017
sensemakr (Cinelli-Hazlett)	Partial-R ² OVB on OLS final stage; robustness value	JRSS-B 2020
dml.sensemakr	Partial-R ² OVB extended to DML score	Chernozhukov, Cinelli, Newey, Sharma, Syrgkanis 2022 — R only
Rosenbaum bounds (Γ)	Bias due to differential treatment assignment in matched studies	Rosenbaum 2002
Manski bounds	Worst-case partial identification	Manski 1990
Bayesian bias-parameter priors	Posterior of effect under priors on bias	McCandless-Greenland
Negative-control outcome (NCO)	Implicit confounding via known-null association	Lipsitch 2010
Specification curve / multiverse	Distribution of effects over modeling choices	Simonsohn et al. 2020
Cross-DAG multiverse	Distribution of effects under competing causal models	Native to CausalRoadmap
Cross-estimator triangulation	Convergence across DR, matching, TMLE	Lawlor et al. 2016 IJE

11.2 Default sensitivity plan by estimand

Each estimand class has an associated default sensitivity plan that runs unless the user opts out:

- ATE / ATT (linear final stage): sensemakr + E-value + placebo refutation.
- ATE (matched designs): Rosenbaum Γ + E-value.
- CATE: sensemakr per subgroup + multiverse over alternative modifier specifications.
- RMST / survival: E-value adapted to hazard ratio + CSF-internal honest splits.
- NDE / NIE: Imai-Keele sensitivity to sequential ignorability violation.
- Stochastic intervention (Imtp): cross-estimator triangulation (TMLE vs. SDR vs. one-step).

11.3 Verdict aggregation

The robustness verdict is computed by the following rule (configurable in the StudyProtocol):

```
if E_value >= 2.0 AND |sensemakr_RV| > 0.10 AND multiverse_CI_consistent_sign:
    verdict = GREEN # finding survives plausible confounding
elif E_value >= 1.5 OR sensemakr_RV > 0.05:
    verdict = YELLOW # finding fragile; report as 'suggestive'
else:
    verdict = RED # finding does not survive plausible perturbation
```

Verdicts appear next to every point estimate in the final report. Yellow and red verdicts are accompanied by a paragraph explaining which assumption would have to fail for the finding to be nullified, in plain language.

12. Phase 6 — Reporting & Aggregation

Phase 6 aggregates the per-hypothesis Roadmap walks into a single coherent deliverable. The report exists in four formats with shared content:

- HTML (default) — interactive, with sortable tables and collapsible Roadmap-step sections.
- PDF — typeset via WeasyPrint, suitable for sharing or supplementary material.
- Quarto — for academic users who want to extend the report with custom analyses.
- Markdown — for archival and LLM consumption.

12.1 Report structure

42. Executive summary — one paragraph, written by the LLM under tight constraints (cannot claim significance beyond the data, must mention robustness verdicts).
43. Domain Expert Brief from Phase 1 — context, suspected unmeasured confounders, candidate DAGs.
44. Feasibility summary from Phase 2 — which hypotheses passed, which failed and why.
45. Hypothesis-by-hypothesis Roadmap walk — Steps 1 through 8 for every queue entry, with all artifacts linked.
46. Cross-hypothesis findings table — BH-adjusted p-values, all robustness verdicts, traffic-light coding.
47. Sensitivity appendix — full E-values, sensemakr contour plots, multiverse curves, Rosenbaum tables.
48. Provenance — LLM model digest, seeds, package versions, R packages (if rbridge used), execution time per phase.
49. Analyst-decision ledger — every default the analyst implicitly accepted, every LLM suggestion overridden, every assumption explicitly asserted.

12.2 Multiple-hypothesis correction

When $K > 1$ hypotheses produce p-values, the package applies Benjamini-Hochberg by default to control false discovery rate at α (default 0.05). The choice can be overridden in the StudyProtocol to use BY (under positive regression dependence) or Bonferroni FWER. The unadjusted and adjusted values are both shown in the findings table; the report's headline conclusions refer to adjusted values.

12.3 Analyst-decision ledger

The single most under-appreciated source of irreproducibility in observational causal work is the silent acceptance of defaults. CausalRoadmap solves this by

recording every analyst-facing decision in the StudyProtocol with a flag indicating whether it was explicitly set, implicitly accepted from a default, or accepted from an LLM suggestion. The decision ledger is rendered as a sortable table in the report.

Categories tracked:

- Choice of confounder set.
- Choice of estimand.
- Choice of estimator.
- Power / sample-size thresholds in Phase 2.
- DAG selection from candidates.
- Multiple-testing correction method.
- Sensitivity bounds and which assumptions were defended vs. accepted.

In the academic tier the ledger is shown at the top of the report; in the data-scientist tier it is at the bottom. In the domain-expert tier it is rendered as a checklist with each item briefly explained.

13. Package Architecture

The package follows a modular, lazy-import architecture organized by responsibility rather than by vendor. The top-level layout:

```
causalrag/  
├── core/                                # framework primitives – small, stable  
│   ├── protocol.py                    # StudyProtocol (Pydantic)  
│   ├── graph.py                      # CausalGraph (wraps NetworkX, GML I/O)  
│   ├── roles.py                      # VariableRole enum & dataclass  
│   ├── flags.py                      # DataFlag enum & combination logic  
│   ├── estimand.py                   # CausalEstimand, StatisticalEstimand  
│   ├── result.py                     # EstimationResult, MultiverseResult  
│   └── registry.py                   # method registry + dispatch  
├── data/                              # ingestion + profiling  
│   ├── connectors/                   # csv, parquet, sql, mongo, fhir, ...  
│   ├── profiler.py                  # deterministic Stage 1b  
│   ├── flags.py                     # flag emission from profile  
│   └── checks.py                    # positivity, balance, overlap  
├── discovery/                         # Phase 1 – LLM discovery agent  
│   ├── investigator.py              # Stage 1c – column understanding  
│   ├── expert.py                    # Stage 1e – domain expert brief  
│   ├── dag_proposer.py              # Stage 1e – K candidate DAGs  
│   └── prompts/                     # versioned Jinja templates  
├── feasibility/                       # Phase 2  
│   ├── power.py                     # MDE calculators per flag combo  
│   ├── thresholds.py                # default + manual + LLM-set modes  
│   └── report.py                     # FeasibilityReport builder  
├── hypothesize/                      # Phase 3  
│   ├── manual.py                    # analyst-driven path  
│   ├── automated.py                 # LLM loop with impact scoring  
│   ├── critique.py                  # LLM critique of manual hypotheses  
│   └── queue.py                     # HypothesisQueue object  
├── roadmap/                          # Phase 4 – the 8 Causal Roadmap steps  
│   ├── q1_question.py  
│   ├── q2_model.py  
│   ├── q3_estimand.py  
│   ├── q4_data.py  
│   ├── q5_identify.py  
│   ├── q6_statistical_estimand.py  
│   ├── q7_estimate.py  
│   └── q8_interpret.py  
└── estimators/                       # the method catalog
```

- └─ base.py # CausalEstimator Protocol
- └─ python/ # native python estimators
 - └─ dml.py # EconML wrappers
 - └─ doubleml.py
 - └─ causalmml.py
 - └─ matching.py
 - └─ synthcontrol.py
 - └─ meta.py # X/T/S/DR-learner
 - └─ discovery.py # causal-learn, lingam, tigramite
 - └─ bart.py # pymc-bart
 - └─ causalimpact.py
- └─ rbridge/ # rpy2 adapters
 - └─ _r.py # singleton R session
 - └─ grf.py # causal_forest, causal_survival_forest
 - └─ lmtp.py # modified treatment policies
 - └─ tmle3.py # tlverse
 - └─ sensemakr.py # OLS / IV / DML sensitivity
 - └─ matchit.py # MatchIt propensity matching
 - └─ marginaleffects.py
 - └─ mediation.py # mediation, CMAverse
 - └─ survRM2.py
 - └─ bnlearn.py
- └─ sensitivity/ # Phase 5
 - └─ evaluate.py
 - └─ sensemakr_py.py # python wrapper of PySensemakr
 - └─ rosenbaum.py
 - └─ manski.py
 - └─ bayes.py # PyMC bias-parameter priors
 - └─ multiverse.py # specification curve
 - └─ verdict.py # green/yellow/red aggregator
- └─ ensemble/ # cross-estimator + cross-DAG aggregation
 - └─ runner.py
 - └─ triangulation.py
 - └─ aggregator.py
- └─ llm/ # LLM orchestration
 - └─ ollama_client.py
 - └─ hardware.py # detection + tier assignment
 - └─ selector.py # auto model selection
 - └─ registry.py # the model catalog
 - └─ guards.py # hallucination guard layers
 - └─ cassette.py # caching for tests + reproducibility
 - └─ prompts/ # shared prompt utilities
- └─ reporting/ # Phase 6
 - └─ card.py # study summary card
 - └─ render_html.py

```

├── render_pdf.py
├── render_quarto.py
├── render_md.py
├── plots.py
├── cli/
│   ├── main.py          # Typer app
│   ├── interactive.py   # REPL
│   ├── tiers.py         # default profiles
│   └── doctor.py        # environment checks
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── fixtures/llm/    # cassettes for reproducible LLM tests
│   └── live_llm/        # opt-in live LLM suite

```

13.1 Dependency layering

Modules import only from layers below them, never sideways or up:

```

Layer 5: cli, reporting
Layer 4: roadmap, hypothesize, feasibility, discovery
Layer 3: estimators, sensitivity, ensemble
Layer 2: llm, data
Layer 1: core

```

This makes the system easier to reason about, easier to test, and makes optional features (like the R bridge) genuinely optional — nothing in Layer 1 imports from Layer 3.

14. Core Abstractions

14.1 StudyProtocol

The StudyProtocol is the central, serializable object. Every command reads from and updates it. Reproducibility derives entirely from it.

```
class StudyProtocol(BaseModel):
    name: str
    version: str = "0.1"
    created: datetime
    updated: datetime
    tier: Literal["data-scientist", "academic", "domain-expert"]

    research_question: Optional[str]
    dataset: DatasetSpec          # source, hash, schema

    discovery: Optional[DiscoveryReport]
    feasibility: Optional[FeasibilityReport]
    hypothesis_queue: list[Hypothesis]
    roadmap_walks: dict[str, RoadmapWalk] # keyed by hypothesis id

    flags: set[DataFlag]
    candidate_graphs: list[CausalGraph]
    selected_graph_index: int = 0

    multiple_testing: Literal["bh", "by", "bonferroni", "none"] = "bh"
    counterfactual_ratio: float = 0.30

    llm: LLMConfig                # model digests, seeds, hardware
    decision_ledger: list[Decision] # every analyst-facing choice
    overrides: list[Override]      # diffs of LLM suggestions
```

14.2 CausalEstimator protocol

Every estimator — Python-native or R-bridged — implements the same protocol:

```
@runtime_checkable
class CausalEstimator(Protocol):
    id: str                # e.g. 'rbridge.grf.causal_survival_forest'
    backend: Literal["python", "r"]
    supported_estimands: tuple[str, ...]
    required_flags: frozenset[DataFlag]
    excluded_flags: frozenset[DataFlag]
    min_sample_size: int
    produces_cate: bool
    produces_full_counterfactual: bool
    propensity_required: bool

    def fit(self, data: pd.DataFrame,
```

```

        protocol: StudyProtocol) -> 'CausalEstimator': ...
def estimate(self) -> EstimationResult: ...
def diagnose(self) -> dict: ...      # overlap, balance, residuals
def refute(self) -> dict: ...      # placebo, subset, unobserved

```

14.3 DataFlag enum

DataFlags are an enum representing all routing-relevant properties of the data and the proposed analysis. Flag emission is the responsibility of the profiler and discovery agent; flag consumption is the responsibility of the estimator registry.

```

class DataFlag(StrEnum):
    # treatment properties
    BINARY_TREATMENT = "binary_treatment"
    CATEGORICAL_TREATMENT = "categorical_treatment"
    CONTINUOUS_TREATMENT = "continuous_treatment"
    MIXTURE_EXPOSURE = "mixture_exposure"
    TIME_VARYING_TREATMENT = "time_varying_treatment"

    # outcome properties
    BINARY_OUTCOME = "binary_outcome"
    CONTINUOUS_OUTCOME = "continuous_outcome"
    COUNT_OUTCOME = "count_outcome"
    RIGHT_CENSORED_OUTCOME = "right_censored_outcome"
    COMPETING_RISKS = "competing_risks"
    REPEATED_OUTCOME = "repeated_outcome"

    # data structure
    PANEL_STRUCTURE = "panel_structure"
    LONGITUDINAL = "longitudinal"
    CLUSTERED = "clustered"
    NETWORK_INTERFERENCE = "network_interference"
    SINGLE_TREATED_UNIT = "single_treated_unit"

    # design hints
    INSTRUMENTAL_CANDIDATE_PRESENT = "instrumental_candidate_present"
    MEDIATOR_PROPOSED = "mediator_proposed"
    NEGATIVE_CONTROL_AVAILABLE = "negative_control_available"

    # sample properties
    SMALL_SAMPLE = "small_sample"          # n < 200
    HIGH_DIMENSIONAL = "high_dimensional"  # p > sqrt(n)
    HEAVY_MISSINGNESS = "heavy_missingness" # > 20%
    HEAVY_CENSORING = "heavy_censoring"     # > 70%
    POSITIVITY_VIOLATION = "positivity_violation"
    SUSPECTED_INFORMATIVE_CENSORING = "suspected_informative_censoring"

```

15. The Data Flag System — Routing Brain

Section 15 is referenced repeatedly above; this section gives the full specification.

15.1 Flag emission sources

Source	Authority	Flags emitted
Stage 1b deterministic profiler	Veto	BINARY/CONTINUOUS treatment & outcome, CENSORING, SMALL_SAMPLE, HIGH_DIMENSIONAL, HEAVY_MISSINGNESS, POSITIVITY_VIOLATION
Stage 1c LLM investigator	Suggest	All semantic flags: MEDIATOR_PROPOSED, INSTRUMENTAL_CANDIDATE_PRESENT, NEGATIVE_CONTROL_AVAILABLE, SUSPECTED_INFORMATIVE_CENSORING
Stage 1e LLM expert	Suggest	Structure flags: PANEL_STRUCTURE, LONGITUDINAL, CLUSTERED, NETWORK_INTERFERENCE
Stage 2 feasibility filter	Veto	SMALL_SAMPLE (refined), POSITIVITY_VIOLATION (refined)
Step 4 (ObservedDataSpec)	Refine	TIME_VARYING_TREATMENT, REPEATED_OUTCOME
Step 5 (Identifiability)	Veto	Sets IDENTIFICATION_FAILED if applicable

Conflicts between sources resolve in favor of the higher-authority source. A deterministic emitter overrides an LLM emitter for the same flag; the analyst always overrides both.

15.2 Flag-to-method dispatch matrix

The estimator registry exposes a query interface that resolves to candidate methods:

```
registry.candidates_for(  
  estimand="ATE",  
  required={DataFlag.BINARY_TREATMENT, DataFlag.RIGHT_CENSORED_OUTCOME},  
  excluded={DataFlag.LONGITUDINAL},  
)  
# returns:  
# [GrfCausalSurvivalForest, IpcwDml, SurvRM2Marginal, ...]
```

Selected dispatch outcomes by flag combination:

Flag combination	Top candidate methods
binary_treatment + continuous_outcome (baseline)	LinearDML, CausalForestDML, AIPW, tmle3 (R), PSM (R)
binary_treatment + binary_outcome	DR estimator, AIPW, BCF (R), TMLE (R), MatchIt
binary_treatment + right_censored_outcome	Causal Survival Forest (R, grf), IPCW-DML, survtmle (R), RMST contrast (R)
continuous_treatment + continuous_outcome	ltp (R), txshift (R), generalized propensity DML, dose-response curve

Flag combination	Top candidate methods
mixture_exposure	Imtp (R) — primary; quantile g-computation; BKMR
binary_treatment + longitudinal + time_varying_confounding	Imtp longitudinal (R), MSM, g-formula, sequential TMLE (tlverse)
IV setting (instrumental_candidate_present)	2SLS, DeepIV, OrthoIV, MR-PRESSO (R) for genetics
mediation (mediator_proposed)	Imai-Keele mediation (R), CMAverse (R), DoWhy.gcm direct/indirect
single_treated_unit + panel_structure	Synthetic Control (Synth R or pysyncon), CausalImpact (R or py), GSC
panel_structure + binary_treatment (DiD)	Goodman-Bacon DiD, callaway-sant'anna staggered DiD, panel DML
high_dimensional + small_sample	SparseLinearDML, Bayesian shrinkage, knockoffs for selection

15.3 Flag inheritance through Roadmap steps

Flags emitted in Phase 1 are inherited by every hypothesis touched in Phase 4, with refinement permitted. For example, a dataset flagged LONGITUDINAL produces hypotheses that inherit LONGITUDINAL by default; if a particular hypothesis chooses a cross-sectional estimand, the LONGITUDINAL flag is replaced by a CROSS_SECTIONAL_SLICE flag local to that hypothesis. The full inheritance and refinement tree is recorded for provenance.

16. LLM Orchestration

The LLM is a first-class component but never the sole decision-maker. This section specifies hardware detection, model selection, prompting strategy, and the hallucination guard pipeline.

16.1 Hardware detection

Cross-platform hardware detection runs as part of doctor and again at any LLM-using command. Detection order:

- 50.psutil → CPU cores (physical + logical), RAM total.
- 51.pynvml → NVIDIA GPU count, VRAM per device, driver, compute capability.
- 52.torch.cuda → cross-check VRAM and CUDA availability.
- 53.Apple Silicon detection: platform.machine() == arm64 and Darwin; treat unified memory as VRAM-equivalent.
- 54.AMD ROCm: pyrsmi or parse rocm-smi output.
- 55.Ollama: GET /api/version + /api/tags to confirm reachability and installed models.

16.2 Hardware tier table (revised for 16B floor)

Tier	Effective VRAM / unified	RAM	Discovery model default	Hypothesize model default
T2 (FLOOR)	12-16 GB	32 GB	qwen3:14b-q4_K_M	deepseek-r1:14b-q4_K_M
T3 prosumer	24 GB	32-64 GB	qwen3:32b-q4_K_M or gemma3:27b-q4_K_M	deepseek-r1:32b-q5_K_M or qwq:32b
T4 workstation	48 GB	64-128 GB	llama3.3:70b-q4_K_M	deepseek-r1:70b-distill-q4_K_M
T5 server	80+ GB	128+ GB	llama4:maverick or qwen3:72b	deepseek-r1:671b-MoE (if multi-GPU) or distill-70b q8_0
T6 Apple Silicon	≥ 32 GB unified	shared	size by 0.7 × unified RAM budget	same

Systems below T2 produce a doctor warning. Two fallback options: (a) the user points the package at an external Ollama host on a more capable machine (network configuration); (b) the package runs in degraded mode with hypothesis generation using a smaller model and reduced expected quality.

16.3 Model selection algorithm

On startup the selector executes:

- 56.Probe hardware → produce HardwareProfile.
- 57.Map HardwareProfile to a tier (using the table above).
- 58.Look up tier defaults in the model registry.

59. Verify Ollama has the models installed; if not, prompt the user (or auto-pull if --auto-pull was set on init).
60. Pin model digests in llm.lock so subsequent runs use the exact same model versions.

16.4 Quantization policy

Recommendations are anchored in the Liu et al. 2025 study on quantization effects in reasoning models:

- Q4_K_M is the default at every tier — empirically near-lossless on reasoning benchmarks.
- Q5_K_M or Q8_0 is preferred for the hypothesize slot at T4+ when latency is acceptable.
- Q3_K_M or below is never recommended automatically; the analyst can opt in via causalrag config set llm.quant Q3_K_M and will be warned.

16.5 Reasoning vs. general-purpose split

The selector assigns three distinct model slots:

- llm.discovery_model — general-purpose instruction-tuned (Qwen3-Instruct, Llama 3.3) — fast structured-JSON output, used in Phase 1 investigator and expert.
- llm.hypothesize_model — reasoning model (DeepSeek-R1 distill, QwQ, Qwen3-thinking) — slower but produces deeper, more counterfactual-friendly proposals.
- llm.utility_model — smallest available — used for JSON repair, schema fix, brief summarization.

16.6 Hallucination guard layers (generalized)

The three-layer gate from CausalRAG generalizes to four layers for the general-purpose package:

61. **Layer 1 — Prevention.** Prompts are templated; column names, value labels, and allowed enum values are injected directly into the prompt so the LLM is constrained at generation time. Ollama 0.4+ structured output enforcement is used wherever supported.
62. **Layer 2 — Schema validation.** Every LLM response is parsed by a Pydantic model. Failures retry once with the validation error as feedback; persistent failures fall back to safe defaults and log the failure.
63. **Layer 3 — Semantic validation.** Every variable the LLM names must exist in the dataset. Every temporal claim is cross-checked against detected temporal ordering. Every IV candidate fails its relevance check on the data — silently — and is downgraded.

64. **Layer 4 — Statistical validation.** Conditional-independence claims implied by an LLM-proposed DAG are tested on the data. Disagreements are not silently overridden in either direction; they are surfaced.

16.7 Reproducibility

Reproducibility of LLM calls requires three things in place:

- Fixed seed passed to Ollama on every call.
- Pinned model digest (recorded in `llm.lock`; SHA from ollama show `<model> --json`).
- Cassette-based replay for CI and re-execution: every LLM call records prompt + response in `.causalrag/cassettes/` keyed by prompt hash and replays from cache by default. The `--refresh-llm` flag forces a new live call.

17. R Bridge Architecture (rpy2)

The R bridge is the package's largest engineering risk and its largest competitive advantage. R hosts the canonical implementations of several methods (Causal Survival Forest, Imtp, sensemakr extensions, MatchIt, marginaleffects, tlverse) that have no production Python equivalent. CausalRoadmap bridges them cleanly through rpy2 with the following design.

17.1 Lazy isolation

Nothing in core, data, discovery, feasibility, hypothesize, or roadmap imports rpy2. The R bridge lives entirely in `causalrag.estimators.rbridge` and is imported only when a user calls a method whose backend is `r`.

17.2 Singleton R session

rpy2 embeds R in the Python process. There is only one R session per process, managed via `@functools.lru_cache` to avoid reinitialization cost:

```
@functools.lru_cache(maxsize=1)
def get_r():
    try:
        from rpy2 import robjects
        from rpy2.robjects.packages import importr, isinstalled
    except Exception as e:
        raise RNotAvailable(
            "Requires R + rpy2. Install: pip install 'causalrag[rbridge]'"
        ) from e
    return robjects, importr, isinstalled

def require_r_package(name: str):
    robjects, importr, isinstalled = get_r()
    if not isinstalled(name):
        raise RPackageMissing(name, hint=
            f"Run 'causalrag doctor --install-r-deps' or in R: "
            f"install.packages('{name}')"
        )
    return importr(name)
```

17.3 Type marshalling

Python ↔ R data interchange:

- pandas DataFrame → R data.frame via `rpy2.robjects.pandas2ri` (activated only inside the bridge).
- numpy ndarray → R numeric / integer / logical via `rpy2.robjects.numpy2ri`.
- R results are materialized to native Python (dict / dataclass / numpy) before returning, so the result is picklable and can be serialized into `EstimationResult`.

- Column names that contain '.' or special characters are sanitized to underscores on the way in and restored on the way out; the mapping is preserved in the adapter.

17.4 Error handling

rpy2 errors surface as `RRuntimeError`. The bridge catches these and re-raises with a Python-side message that includes:

- The R function called.
- A reproduction snippet (the R code the bridge attempted).
- The `R.sessionInfo()` at time of error.
- Suggested fixes (e.g., "install R package X", "upgrade R to 4.4+").

17.5 Reproducibility

Every R-bridged `EstimationResult` records:

- R version (`R.version.string`).
- R package name + version (`packageVersion('grf')`).
- rpy2 version.
- Random seed set in R.

This metadata appears in the report's provenance section.

17.6 Installation paths

Users can get an R-equipped environment three ways:

65. **Native install** — they install R and the required CRAN packages themselves; the doctor verifies.
66. **Auto-install via doctor** — `causalrag doctor --install-r-deps` invokes R from Python and runs `install.packages()` for missing dependencies. Requires R to already be installed.
67. **Container** — ship a `Dockerfile.r` and `devcontainer.json` with R 4.4+, rpy2, and the full required-packages set preinstalled. The recommended path for users who don't want to deal with R installation themselves.

18. Method Registry & Dispatch

The estimator registry is a singleton dict keyed by estimator id, populated at import time via decorator registration. Each registration declares the supported estimands, required and excluded flags, and a sample-size suitability range.

```
@registry.register(
    id="rbridge.grf.causal_survival_forest",
    backend="r",
    estimands=("RMST", "survival_probability", "CATE_RMST"),
    required={DataFlag.BINARY_TREATMENT, DataFlag.RIGHT_CENSORED_OUTCOME},
    excluded={DataFlag.LONGITUDINAL},
    min_n=200,
    description="Cui et al. (2023) Causal Survival Forest via grf R package.",
)
class CausalSurvivalForest:
    def fit(self, data, protocol): ...
    def estimate(self) -> EstimationResult: ...
```

18.1 Dispatch resolution

Given a StatisticalEstimand and the active flag set, the registry returns candidates ordered by an internal priority score combining: estimand specificity (exact match > category match), flag specificity (more required flags = more specific), backend preference (Python preferred over R for cold-start latency, R preferred when it's the canonical implementation), and sample-size match.

In ensemble mode the top K (default 3) candidates run in parallel; their results enter the multiverse aggregator. In single-estimator mode the top-1 runs and a note records what else could have been tried.

18.2 Plugin discovery

Third parties can register estimators by shipping a package that exposes a Python entry point under the `causalrag.estimators` group:

```
# in pyproject.toml of the plugin package
[project.entry-points."causalrag.estimators"]
my_estimator = "my_pkg.estimator:MyEstimator"
```

At startup the registry scans entry points and loads them. This is the path for users who want to ship a domain-specific estimator (e.g., a healthcare-specific TMLE variant) without forking CausalRoadmap.

15A. The Flagging System as LLM Foundation — Overview

Section 15A through 15I specify the flagging system as it is actually consumed by the LLM and the estimator registry. Section 15 introduced the flag taxonomy at the conceptual level; this section is the operational specification. The flagging system is the foundation the LLM uses to choose hypotheses, rank them by impact, and select causal methods — without it, the LLM would either generate hypotheses incompatible with the data or recommend methods the data cannot support.

The architectural commitments in this section are five:

- **Flags are typed records, not booleans.** Every flag carries a confidence score, a provenance trail, and a strength level (veto / strong / weak).
- **Flag families enable LLM reasoning.** The 47-flag taxonomy is grouped into ten families so the LLM can reason about combinations rather than enumerate possibilities.
- **Hypothesis generation is flag-conditioned.** The LLM prompt receives flags as structured input; the prompt template adapts to flag combinations rather than treating them as free text.
- **Impact scoring uses flags as features.** Every component of the impact score (power, novelty, identifiability, coverage) is computed against the flag profile.
- **Method dispatch is rule-based, not lookup-based.** A rule engine evaluates required/excluded/preferred flag combinations and ranks candidate methods; this is what makes the dispatch tractable at the registry's eventual size (50+ estimators in v1.0).

15B. The Complete Flag Taxonomy

The complete flag set contains 47 flags organized into ten families. Each flag has a canonical name (SCREAMING_SNAKE_CASE), a logical type (boolean / categorical / numeric), an emission-source authority level, and a downstream consequence class. The Pydantic definition:

```
# causalrag/core/flags.py
from __future__ import annotations
from enum import StrEnum
from typing import Literal
from pydantic import BaseModel, Field

class FlagStrength(StrEnum):
    VETO = "veto" # binding; no downstream override
    STRONG = "strong" # clear evidence; overridable only with --force
    WEAK = "weak" # suggestive; default-tolerant
    INFO = "info" # informational; never blocks

class FlagSource(StrEnum):
    PROFILER = "profiler" # deterministic Stage 1b
    LLM_INVESTIGATOR = "llm_investigator" # Stage 1c
    LLM_EXPERT = "llm_expert" # Stage 1e domain expert
    FEASIBILITY = "feasibility" # Phase 2
    ROADMAP_S4 = "roadmap_s4" # Step 4 (ObservedDataSpec)
    ROADMAP_S5 = "roadmap_s5" # Step 5 (Identifiability)
    ANALYST = "analyst" # explicit user override

class Flag(BaseModel):
    """Typed record of a single flag emission."""
    name: str = "DataFlag" # the enum value
    strength: FlagStrength # how binding
    source: FlagSource # who emitted it
    confidence: float = Field(ge=0.0, le=1.0) # 1.0 for deterministic
    evidence: str # one-line rationale
    parameters: dict = Field(default_factory=dict) # e.g., {'censoring_rate': 0.42}
    emitted_at: datetime
    supersedes: list[str] = [] # IDs of prior flags this replaces
    flag_id: str # uuid for provenance
```

15B.1 The ten flag families

Flags are organized into families because the LLM reasons about families more reliably than about individual flags. The families are:

Family	Concerns	Typical flag count
F1 — Treatment Structure	Nature of the exposure variable: binary, categorical, continuous, mixture, time-varying	7

Family	Concerns	Typical flag count
F2 — Outcome Structure	Outcome type: binary, continuous, count, time-to-event, censoring, competing risks, repeated measures	8
F3 — Temporal & Panel	Time structure: longitudinal, panel, time-varying confounding, single treated unit, staggered adoption	6
F4 — Identification Hooks	Design-implied identification strategies: IV candidate, mediator, negative control, frontdoor admissibility	5
F5 — Sample Adequacy	Sample size, dimensionality, missingness patterns, censoring intensity	5
F6 — Positivity & Support	Positivity violations, support gaps, overlap deficiency, comparator availability	5
F7 — Confounding Profile	Strength and type of confounding, instrument relevance, mediator pathway clarity	3
F8 — Network & Interference	SUTVA violations, spillover risk, clustering, network structure	3
F9 — Special Designs	Synthetic control, DiD, regression discontinuity, encouragement design	3
F10 — Quality Hazards	Suspected leakage, measurement error, proxy treatment, post-treatment confounder	2

Total: 47 flags. The number is large but the families are small. The LLM prompt presents flags grouped by family with a one-sentence summary of which families fire, then itemizes the flags within fired families.

15B.2 The full enumeration

```
class DataFlag(StrEnum):
    # ===== F1: Treatment Structure =====
    BINARY_TREATMENT = "binary_treatment"
    CATEGORICAL_TREATMENT = "categorical_treatment"           # > 2 unordered levels
    ORDINAL_TREATMENT = "ordinal_treatment"                   # ordered levels
    CONTINUOUS_TREATMENT = "continuous_treatment"
    MIXTURE_EXPOSURE = "mixture_exposure"                     # multiple co-administered
    TIME_VARYING_TREATMENT = "time_varying_treatment"
    DOSE_RESPONSE_REQUESTED = "dose_response_requested"       # analyst wants CDRC

    # ===== F2: Outcome Structure =====
    BINARY_OUTCOME = "binary_outcome"
    CATEGORICAL_OUTCOME = "categorical_outcome"
    CONTINUOUS_OUTCOME = "continuous_outcome"
    COUNT_OUTCOME = "count_outcome"
    RIGHT_CENSORED_OUTCOME = "right_censored_outcome"
    INTERVAL_CENSORED_OUTCOME = "interval_censored_outcome"
    COMPETING_RISKS = "competing_risks"
    REPEATED_OUTCOME = "repeated_outcome"
```

```

# ===== F3: Temporal & Panel =====
LONGITUDINAL = "longitudinal"
PANEL_STRUCTURE = "panel_structure"
TIME_VARYING_CONFOUNDING = "time_varying_confounding"
STAGGERED_ADOPTION = "staggered_adoption" # DiD-relevant
SINGLE_TREATED_UNIT = "single_treated_unit"
INTERRUPTED_TIME_SERIES = "interrupted_time_series" # ITSA / CausalImpact

# ===== F4: Identification Hooks =====
INSTRUMENTAL_CANDIDATE_PRESENT = "instrumental_candidate_present"
MEDIATOR_PROPOSED = "mediator_proposed"
NEGATIVE_CONTROL_AVAILABLE = "negative_control_available"
FRONTDOOR_ADMISSIBLE = "frontdoor_admissible" # rare; flag with care
RUNNING_VARIABLE_PRESENT = "running_variable_present" # RDD

# ===== F5: Sample Adequacy =====
SMALL_SAMPLE = "small_sample" # n < 200
HIGH_DIMENSIONAL = "high_dimensional" # p > sqrt(n)
HEAVY_MISSINGNESS = "heavy_missingness" # > 20% in required col
HEAVY_CENSORING = "heavy_censoring" # > 70%
EXTREMELY_HEAVY_CENSORING = "extremely_heavy_censoring" # > 90% (signal-only)

# ===== F6: Positivity & Support =====
POSITIVITY_VIOLATION = "positivity_violation"
SUPPORT_GAP_CONTINUOUS = "support_gap_continuous" # non-overlap regions
OVERLAP_DEFICIT = "overlap_deficit" # binary;  $\hat{p}$  near 0 or 1
COMPARATOR_MISSING = "comparator_missing" # target-trial arm absent
EXTRAPOLATION_REQUIRED = "extrapolation_required"

# ===== F7: Confounding Profile =====
STRONG_CONFOUNDING_SUSPECTED = "strong_confounding_suspected"
SUSPECTED_INFORMATIVE_CENSORING = "suspected_informative_censoring"
UNMEASURED_CONFOUNDING_PLAUSIBLE = "unmeasured_confounding_plausible"

# ===== F8: Network & Interference =====
CLUSTERED = "clustered"
NETWORK_INTERFERENCE = "network_interference"
SUTVA_VIOLATION_SUSPECTED = "sutva_violation_suspected"

# ===== F9: Special Designs =====
SYNTHETIC_CONTROL_FEASIBLE = "synthetic_control_feasible"
DIFFERENCE_IN_DIFFERENCES_FEASIBLE = "did_feasible"
ENCOURAGEMENT_DESIGN = "encouragement_design" # IV via assigned-to-treated

# ===== F10: Quality Hazards =====
POST_TREATMENT_CONFOUNDER_RISK = "post_treatment_confounder_risk"
MEASUREMENT_ERROR_SUSPECTED = "measurement_error_suspected"

```

15C. The Emission Protocol

Emission of any flag follows a strict protocol. The protocol defines who can emit which family, with what authority level, under what evidentiary threshold.

15C.1 The emission matrix

Family	Profiler	LLM Investigator	LLM Expert	Feasibility	Roadmap
F1 Treatment Structure	VETO	—	—	—	REFINE
F2 Outcome Structure	VETO	—	—	—	REFINE
F3 Temporal & Panel	STRONG	WEAK	STRONG	—	REFINE
F4 Identification Hooks	—	STRONG	STRONG	—	VETO (S5)
F5 Sample Adequacy	VETO	—	—	STRONG (refine)	—
F6 Positivity & Support	STRONG	—	—	VETO	VETO (S5)
F7 Confounding Profile	—	WEAK	STRONG	—	—
F8 Network & Interference	WEAK	WEAK	STRONG	—	—
F9 Special Designs	WEAK	WEAK	STRONG	STRONG (confirm)	—
F10 Quality Hazards	STRONG	STRONG	WEAK	—	STRONG (S4 temporal ity)

Reading the matrix: the profiler can emit F1 and F2 flags with VETO authority (deterministic), F6 with STRONG authority (statistical), and so on. The LLM investigator cannot emit F1/F2/F5 flags at all — those are reserved for deterministic emitters. Conflicts resolve by source priority: PROFILER > FEASIBILITY > ROADMAP_S5 > ROADMAP_S4 > LLM_EXPERT > LLM_INVESTIGATOR > ANALYST (analyst can override anything via explicit flag override commands).

15C.2 Confidence and evidentiary thresholds

Every flag emission must include a confidence score and a one-line evidence string. The package enforces evidentiary thresholds per emission strength:

Strength	Minimum confidence	Required evidence type	Effect on dispatch
VETO	1.00	Deterministic computation result	Hard requirement / hard exclusion
STRONG	≥ 0.80	Statistical test or strong LLM	Default-binding; overridable

Strength	Minimum confidence	Required evidence type	Effect on dispatch
		agreement	only with --force
WEAK	≥ 0.50	LLM single-pass with rationale	Influences ranking; never blocks
INFO	any	Any source	Recorded for provenance only

Below-threshold emissions are dropped automatically and logged to the audit trail. This prevents a low-confidence LLM hallucination from upgrading itself to a binding flag through repetition.

15C.3 The emission API

```
from causalrag.core.flags import FlagEmitter, Flag, DataFlag, FlagStrength, FlagSource

class FlagEmitter:
    """Single entry point for any module emitting a flag.

    Enforces the emission matrix, threshold gates, and provenance.
    Modules never construct Flag objects directly; they call emit().
    """
    def __init__(self, audit_log: AuditLog, emission_matrix: dict):
        self.audit = audit_log
        self.matrix = emission_matrix

    def emit(
        self,
        flag: DataFlag,
        *,
        source: FlagSource,
        strength: FlagStrength,
        confidence: float,
        evidence: str,
        parameters: dict | None = None,
    ) -> Flag | None:
        """Emit a flag. Returns None if rejected by matrix or thresholds."""
        family = self._family_of(flag)
        max_strength = self.matrix.get((family, source))
        if max_strength is None or strength.value_rank > max_strength.value_rank:
            self.audit.write({
                "event": "flag_rejected_by_matrix",
                "flag": flag, "source": source, "reason": "source_lacks_authority",
            })
            return None
        min_conf = {FlagStrength.VETO: 1.0, FlagStrength.STRONG: 0.80,
                    FlagStrength.WEAK: 0.50, FlagStrength.INFO: 0.0}[strength]
        if confidence < min_conf:
            self.audit.write({
                "event": "flag_rejected_by_threshold",
                "flag": flag, "confidence": confidence, "required": min_conf,
```

```
    })
    return None
f = Flag(name=flag, source=source, strength=strength,
         confidence=confidence, evidence=evidence,
         parameters=parameters or {}, emitted_at=now_utc(),
         flag_id=uuid4().hex)
self.audit.write({"event": "flag_emitted", **f.model_dump()})
return f
```

15D. How the LLM Consumes Flags — The Foundation

Section 15D specifies the exact prompt structure the LLM sees when it must reason about flags. This is the foundation the user identified — without it, the LLM is guessing; with it, the LLM is performing a structured task with bounded uncertainty.

15D.1 The flag-conditioned hypothesis prompt

When the automated hypothesis generator (Phase 3) invokes the LLM, the prompt includes a structured FlagDigest block — a compact, LLM-readable summary of the entire flag set, organized by family, with confidence and evidence shown inline. The LLM is instructed to generate hypotheses that respect the flag set, not hypotheses the data cannot support.

The prompt template:

```
# causalrag/discovery/prompts/hypothesize.jinja

You are a causal-inference research analyst. You will generate {{ N }}
testable hypotheses against a dataset whose properties are described below.

Your hypotheses MUST satisfy two hard constraints:
(1) Every variable named must exist in the dataset schema.
(2) The hypothesis MUST be answerable given the FLAG DIGEST below.
    Any hypothesis incompatible with the flags will be rejected by
    automated validation and you will be re-prompted.

=== FLAG DIGEST ===
{% for family, flags in flag_digest.items() %}
[{{ family.code }}] [{{ family.name }}] ({{ family.summary }})
{% for f in flags %}
  - [{{ f.name }}] [{{ f.strength }}], conf={{ f.confidence }}
    [{{ f.evidence }}]
    {% if f.parameters %} params: [{{ f.parameters | tojson }}] {% endif %}
{% endfor %}
{% endfor %}

=== APPLICABLE ESTIMAND CLASSES ===
Given the flags above, the following estimand classes are feasible:
{% for est in feasible_estimands %}
  - [{{ est.name }}]: [{{ est.description }}]
    required-flag-fit: [{{ est.flag_match_score }}]
{% endfor %}

=== DOMAIN EXPERT BRIEF ===
[{{ domain_brief }}]

=== HYPOTHESIS HISTORY ===
Already-tested hypotheses (avoid duplicates):
```

```
{% for h in history %} - {{ h.text }}{% endfor %}

=== TASK ===
Generate exactly {{ N }} hypotheses as a JSON array. {{ cf_n }} of them
must be counterfactual queries ('What would Y have been if A had been...').
Schema for each entry:
{
  "id": "H_<int>",
  "text": "natural-language hypothesis",
  "estimand": {"type": "<from feasible_estimands>",
    "treatment": "<column>", "outcome": "<column>",
    "modifiers": [...], "target_population": "..."},
  "rationale": "why this matters; <= 40 words",
  "expected_direction": "positive | negative | unknown",
  "expected_magnitude": "small | medium | large | unknown",
  "flag_compatibility": {
    "requires": [<flag_names you read as enabling this>],
    "would_fail_under": [<flag_names that would invalidate this>]
  }
}
```

15D.2 Why this structure works

Two design choices in this prompt are doing the heavy lifting:

68. **Feasible-estimand pre-filtering.** The prompt does not ask the LLM to choose any estimand class — it lists only the classes already determined to be feasible given the flag set (see §15F for how this list is computed). The LLM cannot propose RMST for a dataset that has no time-to-event outcome.
69. **Self-reporting flag compatibility.** Each hypothesis must declare which flags it "reads as enabling" and which would invalidate it. This forces the LLM to explicitly reason about flag combinations rather than treating them as decoration; the validator then compares the LLM's stated compatibility against the actual flag set, catching contradictions.

15E. The Causal-Question Catalog Indexed by Flag Combination

Section 15E is the menu from which the LLM selects. Rather than asking the LLM to imagine causal questions from scratch, the package maintains a flag-indexed catalog of 30+ canonical causal-question templates. Given a flag set, the catalog returns the templates that fit; the LLM then specializes them with the actual column names and domain context.

Question template	Required flag set (minimal)	Excluded flags	Typical estimand
What is the ATE of A on Y in the population?	F1.BINARY_TREATMENT, F2.* (any)	F6.POSITIVITY_VIOLATION (binding)	ATE
What is the effect of A on Y among the treated?	F1.BINARY_TREATMENT, F2.* (any)	F6.COMPARATOR_MISSING	ATT
How does the effect of A on Y vary across X?	F1.BINARY_TREATMENT, F2.* (any), $n_x \geq 50$ per stratum	F5.SMALL_SAMPLE (if subgroup tiny)	CATE
What is the dose-response curve of A on Y at level a^* ?	F1.CONTINUOUS_TREATMENT	F6.SUPPORT_GAP_CONTINUOUS at a^* (binding)	ADRF / CDRC
What would happen under a feasible shift $d(A,W)$?	F1.CONTINUOUS_TREATMENT or F1.MIXTURE_EXPOSURE	—	MTP / LMTP
What is the RMST contrast at horizon τ ?	F2.RIGHT_CENSORED_OUTCOME, follow-up $\geq \tau$	F5.EXTREMELY_HEAVY_CENSORING (translates)	RMST contrast
What is the survival probability difference at t ?	F2.RIGHT_CENSORED_OUTCOME, n at risk ≥ 30 at t	—	$S(t A=1) - S(t A=0)$
What is the NDE through mediator M?	F4.MEDIATOR_PROPOSED, no F10.POST_TREATMENT_CONFOUNDER_RISK on M	—	NDE
What is the NIE through mediator M?	F4.MEDIATOR_PROPOSED	—	NIE
What is the LATE in compliers?	F4.INSTRUMENTAL_CANDIDATE_PRESENT	no F4.* would mean no IV available	LATE / Wald
What is the time-varying ATE under regime g ?	F3.LONGITUDINAL, F3.TIME_VARYING_CONFOUNDING	—	Longitudinal regime contrast (LTMLE / LMTP)
What would Y have been at site k under intervention a ?	F3.PANEL_STRUCTURE, F3.SINGLE_TREATED_UNIT	—	Synthetic-control counterfactual
What was the immediate impact of intervention at time t_0 ?	F3.INTERRUPTED_TIME_SERIES	—	ITSA / CausalImpact
Did the effect change after	F3.STAGGERED_ADOPTION	—	Callaway-

Question template	Required flag set (minimal)	Excluded flags	Typical estimand
staggered rollout?			Sant'Anna DiD
What is the rank-weighted ATE under prioritization rule?	F1.BINARY_TREATMENT + CATE model fittable	F5.SMALL_SAMPLE	RATE / AUTOC
Is the effect of A monotone in W?	F1.CONTINUOUS_TREATMENT or F1.ORDINAL_TREATMENT	—	Monotone dose-response
What is the policy value of treatment rule $\pi(W)$?	F1.BINARY_TREATMENT, CATE fittable	—	$V(\pi)$ — policy learning
What share of the effect is direct vs. indirect?	F4.MEDIATOR_PROPOSED, F2.* (any)	—	Proportion mediated
What is the effect under counterfactual exposure pattern σ ?	F3.LONGITUDINAL + F1.TIME_VARYING_TREATMENT	—	Stochastic-shift LMTP
Among never-takers, what would $A=1$ do?	F4.ENCOURAGEMENT_DESIGN	—	Principal stratification ATE

The catalog is more than convenient — it is the architectural mechanism that bounds the hypothesis space. Without it the LLM would generate from a near-infinite menu; with it the LLM specializes from a curated menu of 20+ templates, each pre-validated against the flag system.

15F. The Feasible-Estimand Engine

Section 15F describes the rule engine that produces the `feasible_estimands` list shown in the prompt of §15D. The engine takes the current `FlagSet` and returns the estimand classes that can be defended given the flags, ranked by how well-matched the flags are.

15F.1 The matching algorithm

```
from __future__ import annotations
from dataclasses import dataclass
from typing import Sequence
from causalrag.core.flags import DataFlag, FlagSet

@dataclass(frozen=True)
class EstimandSpec:
    """Static descriptor of an estimand class for the registry."""
    name: str # 'ATE', 'CATE', 'RMST', ...
    required: frozenset[DataFlag] # must be present
    excluded: frozenset[DataFlag] # must be absent
    preferred: frozenset[DataFlag] # boosts match score
    disqualifying: frozenset[DataFlag] # absolute veto (positivity, etc.)
    description: str
    typical_method: str # link to registry id

class FeasibleEstimandEngine:
    """Compute the feasible estimand classes given a FlagSet.

    Used by Phase 3 hypothesis generation to constrain the LLM, and
    by Phase 4 Step 3 to validate analyst-supplied estimands.
    """
    def __init__(self, registry: "EstimandRegistry"):
        self.registry = registry

    def feasible(self, flags: FlagSet) -> list["FeasibleEstimand"]:
        """Return all feasible estimands with match scores in [0, 1]."""
        results = []
        for spec in self.registry.all():
            verdict = self._evaluate(spec, flags)
            if verdict.feasible:
                results.append(FeasibleEstimand(
                    name=spec.name, description=spec.description,
                    flag_match_score=verdict.score,
                    typical_method=spec.typical_method,
                    rationale=verdict.rationale,
                ))
        results.sort(key=lambda x: -x.flag_match_score)
        return results
```

```

def _evaluate(self, spec: EstimandSpec, flags: FlagSet) -> "Verdict":
    active = flags.active_names()
    # disqualifying flags veto absolutely
    for f in spec.disqualifying & active:
        return Verdict(feasible=False, score=0.0,
                       rationale=f"disqualified by {f}")
    # required must all be present
    missing = spec.required - active
    if missing:
        return Verdict(feasible=False, score=0.0,
                       rationale=f"missing required {missing}")
    # excluded must all be absent
    forbidden = spec.excluded & active
    if forbidden:
        return Verdict(feasible=False, score=0.0,
                       rationale=f"forbidden by {forbidden}")
    # score: base 0.5 + 0.1 per preferred + 0.1 per required match
    score = 0.5 + 0.1 * len(spec.preferred & active) + 0.1 * len(spec.required)
    return Verdict(feasible=True, score=min(score, 1.0), rationale="matched")

```

15F.2 Why "feasible" is not "identified"

A feasible estimand is one for which the data structure can support an estimator; identification (Phase 4 Step 5) is a strictly stronger property that depends on the chosen DAG. Feasibility is necessary but not sufficient for identification. The two-stage separation is intentional: §15F is fast (rule-engine evaluation in microseconds), called many times during hypothesis generation; identification (Phase 4 Step 5) is slow (DoWhy / Ananke graph traversal), called once per hypothesis. Front-loading feasibility into Phase 3 saves the LLM from proposing 50 hypotheses only to have 30 fail at identification.

15G. The Method Dispatch Rule Engine

Section 15G is the rule engine that maps (estimand × flag set) to ranked candidate methods. It is the operational version of the dispatch matrix shown in §15.2; the table is a static summary, while the engine is the working code.

15G.1 Rule structure

```
from causalrag.core.flags import DataFlag, FlagSet
from causalrag.estimators import registry

class DispatchRule(BaseModel):
    """One rule in the method dispatch engine."""
    rule_id: str
    estimator_id: str          # registry key, e.g. 'rbridge.grf.causal_survival_forest'
    estimand_pattern: str      # 'ATE', 'CATE', or '*' for any
    required: frozenset[DataFlag] = frozenset()
    excluded: frozenset[DataFlag] = frozenset()
    preferred: frozenset[DataFlag] = frozenset()
    sample_size_min: int = 50
    priority: int = 100        # higher = preferred when multiple match
    tier_default: dict[str, bool] = {          # tier-level defaults
        "data-scientist": False, "academic": False, "domain-expert": False
    }
    rationale: str             # one-line explanation
```

15G.2 Example rules (excerpted)

```
DISPATCH_RULES = [
    DispatchRule(
        rule_id="D001",
        estimator_id="python.econml.LinearDML",
        estimand_pattern="ATE",
        required={DataFlag.BINARY_TREATMENT},
        excluded={DataFlag.RIGHT_CENSORED_OUTCOME, DataFlag.LONGITUDINAL,
                  DataFlag.POSITIVITY_VIOLATION},
        preferred={DataFlag.HIGH_DIMENSIONAL},
        priority=100, tier_default={"data-scientist": True,
                                   "academic": True, "domain-expert": True},
        rationale="Default DML; CausalRAG's original estimator.",
    ),
    DispatchRule(
        rule_id="D002",
        estimator_id="rbridge.tlverse.tmle3",
        estimand_pattern="ATE",
        required={DataFlag.BINARY_TREATMENT},
        excluded={DataFlag.LONGITUDINAL, DataFlag.RIGHT_CENSORED_OUTCOME},
        preferred=set(),
        priority=110, tier_default={"academic": True},
    ),
]
```

```

        rationale="Efficient + targeted; preferred in epi.",
    ),
    DispatchRule(
        rule_id="D010",
        estimator_id="rbridge.grf.causal_survival_forest",
        estimand_pattern="RMST",
        required={DataFlag.RIGHT_CENSORED_OUTCOME, DataFlag.BINARY_TREATMENT},
        excluded={DataFlag.LONGITUDINAL, DataFlag.COMPETING_RISKS},
        priority=100, tier_default={"data-scientist": True,
                                   "academic": True, "domain-expert": True},
        rationale="Cui et al. 2023 – canonical for right-censored CATE/RMST.",
    ),
    DispatchRule(
        rule_id="D020",
        estimator_id="rbridge.ltmf.ltmf_tmle",
        estimand_pattern="MTP",
        required={DataFlag.CONTINUOUS_TREATMENT},
        preferred={DataFlag.LONGITUDINAL, DataFlag.TIME_VARYING_CONFOUNDING,
                  DataFlag.COMPETING_RISKS},
        priority=120, tier_default={"academic": True},
        rationale="Díaz et al. 2021 – the modern MTP framework.",
    ),
    # ... 40+ further rules in the v0.5 registry ...
]

```

15G.3 Dispatch algorithm

```

class MethodDispatcher:
    def candidates_for(
        self, *,
        estimand: str,
        flags: FlagSet,
        tier: Literal["data-scientist", "academic", "domain-expert"],
        n: int,
        max_k: int = 5,
    ) -> list["RankedCandidate"]:
        """Return ranked candidates for the given (estimand, flags) pair.

        Algorithm:
        1. Filter to rules whose estimand_pattern matches.
        2. Filter to rules whose required/excluded flag sets are satisfied.
        3. Filter to rules whose sample_size_min <= n.
        4. Score: priority + 10 per preferred flag intersected with active.
        5. Demote any rule not flagged tier_default[tier]=True by 50 points.
        6. Return top max_k.
        """
        active = flags.active_names()
        candidates = []
        for rule in DISPATCH_RULES:
            if not _estimand_match(rule.estimand_pattern, estimand):

```

```
        continue
    if rule.required & active:
        continue
    if rule.excluded & active:
        continue
    if rule.sample_size_min > n:
        continue
    score = rule.priority + 10 * len(rule.preferred & active)
    if not rule.tier_default.get(tier, False):
        score -= 50
    candidates.append(RankedCandidate(
        rule=rule, score=score,
        estimator=registry.get(rule.estimator_id),
    ))
candidates.sort(key=lambda c: -c.score)
return candidates[:max_k]
```

15H. Flag-Driven Impact Scoring for Hypothesis Ranking

Section 15H is the formula that ranks hypotheses by expected impact, used by the automated mode in Phase 3. Every component of the score is computed against the flag set; flags are not just routing inputs, they are predictive features for which hypotheses are most likely to produce a usable answer.

15H.1 The score components

Component	Sym bol	How computed from flags	Weight (default)
Power adequacy	P	Inverse of MDE under (estimand, flags, n); flags determine which MDE formula applies	0.25
Identifiability confidence	I	1.0 if F4 hooks fire matching estimand; declines with F7 unmeasured-confounding flag	0.20
Novelty vs. history	N	1 – max cosine similarity to (treatment, outcome, modifier) triples in history	0.15
Coverage contribution	C	Marginal entropy gain in (treatment × outcome × modifier) cube of accepted hypotheses	0.15
LLM domain importance	L	LLM-rated 0–1; calibrated by F7 confidence trail	0.10
Specificity bonus	S	+0.1 if CATE > ATE under flag set; +0.05 if RMST is feasible (F2.RIGHT_CENSORED_OUTCOME)	0.10
Robustness prospect	R	Number of valid sensitivity methods × inverse of F7/F10 hazard count	0.05

15H.2 The full scoring formula

```
def impact_score(
    hypothesis: "Hypothesis",
    *,
    flag_set: FlagSet,
    history: list["Hypothesis"],
    queue: list["Hypothesis"],
    tier: str,
    weights: dict[str, float] | None = None,
) -> ImpactScore:
    """Compute the impact score for a candidate hypothesis.

    All component scores live in [0, 1]. The aggregate is a weighted
    arithmetic mean; weights default to those in §15H.1 but are overridable.
    Returns the full breakdown so the analyst can see why a hypothesis
    was ranked where it was.
    """
    w = weights or DEFAULT_WEIGHTS[tier]
```

```

P = _power_component(hypothesis, flag_set)
I = _identifiability_component(hypothesis, flag_set)
N = _novelty_component(hypothesis, history)
C = _coverage_component(hypothesis, queue)
L = _llm_importance_component(hypothesis)
S = _specificity_bonus(hypothesis, flag_set)
R = _robustness_prospect(hypothesis, flag_set)
aggregate = (w["P"]*P + w["I"]*I + w["N"]*N + w["C"]*C
              + w["L"]*L + w["S"]*S + w["R"]*R)
return ImpactScore(
    aggregate=aggregate,
    components={"P":P,"I":I,"N":N,"C":C,"L":L,"S":S,"R":R},
    weights=w,
    rationale=_explain(P, I, N, C, L, S, R, flag_set),
)

```

15H.3 Why flags belong in the score

A naive impact score is just the LLM's rated importance. That is exactly the wrong design — it gives the highest scores to the hypotheses the LLM is most confident about, which is also where the LLM is most likely to be hallucinating. By embedding flag-derived components (power, identifiability, robustness prospect) into the score, the system rewards hypotheses that are not just "interesting" but actually answerable from this dataset. The LLM's rating remains a component, but at 10% weight — it informs ranking without dominating it.

15H.4 Tier-specific weight defaults

Tier	P	I	N	C	L	S	R
data-scientist	0.20	0.15	0.15	0.15	0.20	0.10	0.05
academic	0.25	0.25	0.10	0.10	0.05	0.15	0.10
domain-expert	0.30	0.20	0.10	0.10	0.20	0.05	0.05

Reading the table: the academic tier weights identifiability and power most heavily and demotes LLM importance to 0.05; the data-scientist tier balances LLM rating with power and coverage; the domain-expert tier prioritizes power (so the user sees only hypotheses likely to yield a clear answer).

15l. Anti-Pattern Detection — Forbidden Flag Combinations

Section 15l closes the system. The combinatorial space of 47 flags includes combinations that are logically incoherent — for example, a column cannot simultaneously be `BINARY_TREATMENT` and `MIXTURE_EXPOSURE`. The package's anti-pattern detector enumerates these forbidden combinations and refuses to proceed when one is detected; the report surfaces the violation prominently.

15l.1 Catalog of anti-patterns

Anti-pattern	Why forbidden	Resolution
<code>BINARY_TREATMENT</code> $\wedge$ <code>MIXTURE_EXPOSURE</code>	A binary indicator cannot also be a multi-component exposure	Profiler veto wins; LLM downgraded
<code>CONTINUOUS_TREATMENT</code> $\wedge$ <code>BINARY_TREATMENT</code> for same column	Type contradiction	Profiler veto resolves
<code>RIGHT_CENSORED_OUTCOME</code> $\wedge$ no time column	Censoring requires a time axis	Block; route back to S4 (data spec)
<code>LONGITUDINAL</code> $\wedge$ <code>SINGLE_TREATED_UNIT</code> $\wedge$ no <code>PANEL_STRUCTURE</code>	Single unit with no panel structure cannot support longitudinal estimation	Translate to <code>INTERRUPTED_TIME_SERIES</code> or block
<code>MEDIATOR_PROPOSED</code> $\wedge$ <code>POST_TREATMENT_CONFOUNDER_RISK</code> on mediator	Mediator pollution; classic mediation invalid	Block NDE; allow direct-effect with warning
<code>INSTRUMENTAL_CANDIDATE_PRESENT</code> $\wedge$ no temporal precedence over treatment	IV must precede treatment	Downgrade IV claim to candidate-only
<code>EXTREMELY_HEAVY_CENSORING</code> $\wedge$ <code>RMST</code> horizon > follow-up max	Estimand exceeds support	Translate horizon via §18E or block
<code>POSITIVITY_VIOLATION</code> (binding) $\wedge$ requested ATE	Effect is unidentified without trimming or MTP translation	Route to feasible-estimand translator
<code>FRONTDOOR_ADMISSIBLE</code> $\wedge$ <code>MEDIATOR</code> has no parents in DAG	Frontdoor requires the mediator to have observed parents	Block; route to S2 (model)
<code>NETWORK_INTERFERENCE</code> $\wedge$ standard ATE estimator chosen	ATE under SUTVA violation is biased	Force route to spillover-aware estimator
<code>HIGH_DIMENSIONAL</code> $\wedge$ <code>SMALL_SAMPLE</code> $\wedge$ no regularization preference	Standard estimators fail; nuisance overfit certain	Force regularized DML or shrinkage
<code>COMPARATOR_MISSING</code> $\wedge$ ATT requested	No comparison arm; only conditional outcome distribution available	Block or route to single-arm benchmark

15l.2 Anti-pattern detector implementation

```
class AntiPatternDetector:
    """Detects logically incoherent flag combinations and routes resolution.

    Runs three times in the pipeline:
    (1) After Phase 1 – catches profiler/LLM conflicts.
    (2) After Phase 3 hypothesis-generation – catches LLM-introduced
        flag claims that contradict the dataset.
    (3) Before Phase 4 Step 7 – final pre-estimation check.
    """
    PATTERNS: list["AntiPattern"] = [
        AntiPattern(
            id="AP001",
            forbidden={DataFlag.BINARY_TREATMENT,
                      DataFlag.MIXTURE_EXPOSURE},
            scope="same_column",
            resolution="profiler_wins",
            message="Column cannot be both binary and a mixture exposure.",
        ),
        AntiPattern(
            id="AP002",
            forbidden={DataFlag.RIGHT_CENSORED_OUTCOME},
            requires_present={"_has_time_column"}, # virtual flag
            scope="global",
            resolution="block_route_to_s4",
            message="Right-censored outcome requires a paired time column.",
        ),
        AntiPattern(
            id="AP008",
            forbidden={DataFlag.POSITIVITY_VIOLATION},
            estimand_pattern="ATE",
            strength_threshold=FlagStrength.STRONG,
            scope="hypothesis",
            resolution="route_to_feasible_estimand_translator",
            message="ATE under positivity violation requires translation; "
                    "see §18E.",
        ),
        # ... 12+ further patterns ...
    ]

    def scan(self, flags: FlagSet,
             *, hypothesis: "Hypothesis | None" = None) -> list["Detection"]:
        detections = []
        for pat in self.PATTERNS:
            if pat.matches(flags, hypothesis):
                detections.append(Detection(
                    pattern_id=pat.id, message=pat.message,
                    resolution=pat.resolution,
```

```
        severity="block" if pat.is_blocking() else "warn",
    ))
    return detections
```

15l.3 What happens when an anti-pattern fires

Anti-pattern detections are routed by the resolution field:

- `profiler_wins` — the LLM-emitted flag is downgraded; the profiler's veto-grade emission stands.
- `block_route_to_s4` — execution halts and the analyst is directed to Step 4 (ObservedDataSpec) to revise the data spec.
- `route_to_feasible_estimand_translator` — invokes §18E to translate the target to the nearest feasible estimand.
- `force_regularized_estimator` — overrides the dispatch ranking to a method appropriate for the actual flag combination.
- `warn_and_proceed` — INFO-level note in the audit log; the report flags the combination with a yellow banner.

Every detection is logged to `.causalrag/audit.jsonl` with the pattern id, the firing flags, and the resolution taken. The `verify` subcommand (§28B) checks that every detection was resolved — analyses that ignored an anti-pattern detection cannot pass verification.

18A. Dataset Integrity & Limitation Locking — Overview

Section 18A through 18F introduce the architectural commitment that distinguishes CausalRoadmap from every other causal-inference framework: every analysis is locked to the dataset, and the dataset's limitations are the binding constraint on what the package will compute. This is the package's answer to a specific failure mode of modern LLM-assisted causal pipelines — the temptation to estimate quantities the data cannot actually support, then dress the result up in a confidence interval.

The architecture rests on five mechanisms, each documented in its own section: the Dataset Lock, the Data Support Envelope, positivity & overlap diagnostics, feasible estimand translation, and the Genuineness Audit. Together they enforce a single rule: when the analyst's request exceeds what the data supports, the package either refuses the analysis, translates the request to the nearest feasible target, or runs it with prominent warnings — but never silently produces a number.

18B. The Dataset Lock

The Dataset Lock is a tamper-evident, content-addressed record of the dataset under analysis. It is created once during Phase 1 and never modified again. Every downstream artifact carries the Lock's hash; analysis runs against a dataset whose Lock does not match its provenance refuse to execute. This eliminates an entire class of silent reproducibility failures — modifying the underlying file between runs while preserving the protocol YAML.

18B.1 What the Lock contains

- `dataset_hash` — BLAKE3 hash of the raw bytes of every constituent file. Multi-file datasets (partitioned Parquet, joined CSVs) emit a Merkle root.
- `schema_fingerprint` — SHA-256 of the canonicalized schema: ordered column names, dtypes, and detected logical types from the profiler.
- `row_count` — pre-filter and post-filter (after dropping rows with required NA).
- `collection_window` — earliest and latest observed dates if any temporal columns exist.
- `source_provenance` — connector type (CSV, Postgres, FHIR), source URI (sanitized of credentials), and the LLM-extracted data dictionary if present.
- `freeze_timestamp` — ISO 8601 UTC at lock creation.
- `preprocessing_lineage` — every deterministic transformation applied (dtype casts, value-label maps, censoring-column pairing) with a hash of the transformation parameters.

18B.2 Implementation

```
from __future__ import annotations
import hashlib, json, datetime as dt
from pathlib import Path
from typing import Literal
import blake3          # pip install blake3
import pyarrow as pa
from pydantic import BaseModel, Field, computed_field

class DatasetLock(BaseModel):
    """Tamper-evident provenance record for a dataset.

    The lock is created exactly once and frozen. Every downstream
    EstimationResult carries lock.fingerprint(); runs against a
    dataset whose computed fingerprint differs from the stored one
    refuse to execute (see causalrag.audit.LockMismatchError).
    """
    dataset_hash: str          # blake3 of raw bytes (Merkle if multi-file)
    schema_fingerprint: str    # sha256 of canonical schema JSON
    row_count_raw: int
```

```

row_count_analyzable: int          # after required-column NA drop
column_dtypes: dict[str, str]      # name -> logical dtype
collection_window: tuple[str, str] | None = None
source_uri_redacted: str
connector: Literal["csv", "parquet", "sql", "mongo", "fhir", "api", "hf"]
freeze_timestamp: dt.datetime = Field(default_factory=lambda: dt.datetime.now(dt.UTC))
preprocessing_lineage: list[dict] = Field(default_factory=list)
causalrag_version: str

@computed_field
@property
def fingerprint(self) -> str:
    """Single 32-char identifier carried by every downstream artifact."""
    payload = self.model_dump_json(exclude={"freeze_timestamp"}).encode()
    return blake3.blake3(payload).hexdigest()[:32]

@classmethod
def from_path(cls, path: Path, connector: str, version: str) -> "DatasetLock":
    """Build a lock from a filesystem path."""
    h = blake3.blake3()
    for chunk in cls._iter_file_chunks(path):
        h.update(chunk)
    # ... read schema, count rows, compute fingerprints ...
    return cls(dataset_hash=h.hexdigest(), connector=connector,
               causalrag_version=version, ...) # remaining fields omitted

```

18B.3 Modifying a locked dataset

The Lock is intentionally adversarial. The only sanctioned way to change the dataset is causalrag relock, which:

70. Refuses to run if any EstimationResult depends on the current Lock and has not been explicitly invalidated.
71. Recomputes the full Lock against the new bytes.
72. Writes a relock_log.jsonl entry describing the diff (rows added, columns added, value-label remaps).
73. Forces all downstream phases to re-run; partial reuse requires the analyst to mark which artifacts they certify as still valid.

18C. The Data Support Envelope

The Data Support Envelope is the package's machine-readable answer to the question: "what can this dataset actually answer?" It is computed once during Phase 1, attached to the StudyProtocol, and consulted at every phase boundary. When a hypothesis or estimand falls outside the Envelope, the package refuses to run the analysis at face value.

18C.1 Components of the Envelope

Component	Defines	How computed
Treatment support	For each candidate treatment column: the range of observed values and the conditional density $\Pr(A=a \mid W=w)$	Generalized propensity / density-ratio classification; for binary, the propensity bounded away from $\{0,1\}$
Outcome observability	For each candidate outcome: censoring rate, missingness pattern, measurement scale fidelity	Profiler + LLM investigator output
Confounder coverage	Which confounders are measured at what time points; gaps in temporal coverage	Stage 4 (ObservedDataSpec)
Subgroup support	For every effect modifier $\times$ treatment cell: cell count and propensity overlap	Pre-computed during Feasibility Filter
Time-horizon support	For longitudinal data: which forecast horizons remain identifiable as follow-up shrinks	From censoring distribution; survRM2-style restricted horizon
Intervention feasibility	For continuous treatments: which intervention shifts $d(a,w)$ keep the joint (a,w) inside observed support	Non-overlap ratio (Khatibi, Wang, Rahmani 2025; Moodie 2025)
Comparator availability	For target-trial questions: whether comparator arm exists with sufficient n	From treatment-arm cross-tabulation

18C.2 The contract

The contract between the Envelope and every downstream module is one-way: the Envelope can refuse, downgrade, or annotate any request; modules cannot override the Envelope without an explicit, logged `--force-out-of-support` flag. The flag is reserved for genuine research situations where the analyst is studying the failure mode itself; it is never the default.

18D. Positivity & Overlap Diagnostics

Positivity — the requirement that every unit have non-zero probability of every counterfactual treatment level conditional on confounders — is the assumption most often glossed over by automated causal pipelines. CausalRoadmap runs a layered diagnostic suite at Phase 2 and again at Phase 4 Step 5 (identifiability).

18D.1 Binary and categorical treatments

For each candidate (treatment, confounders) pair:

- Fit a propensity model (regularized logistic regression at v0.1; gradient-boosted with calibration at v0.5).
- Compute the distribution of estimated propensities in each arm.
- Flag a violation if $\min(\hat{p}) < 0.05$ or $\max(\hat{p}) > 0.95$ for more than 1% of units (configurable).
- Bootstrap the propensity model to assess instability (Petersen, Porter, Gruber, Wang, van der Laan 2012 framework).
- Run a parametric bootstrap diagnostic — refit the analysis on simulated data drawn from the fitted nuisance model with positivity intact; compare the resulting bias to the original to quantify the threat (Petersen et al. 2012).

18D.2 Continuous treatments — non-overlap ratio

The non-overlap ratio (Khatibi, Wang, Rahmani 2025; Moodie 2025) is the principled diagnostic for continuous treatments. For each candidate intervention level a^* and the confounders W :

```
from __future__ import annotations
import numpy as np
from typing import Callable

def non_overlap_ratio(
    a_star: float,
    *,
    treatment: np.ndarray,
    confounders: np.ndarray,
    density_estimator: Callable[[np.ndarray, np.ndarray], np.ndarray],
    coverage: float = 0.95,
) -> float:
    """Fraction of units for whom a_star lies outside their supported A-range.

    Implements the non-overlap ratio of Khatibi, Wang, Rahmani (arXiv 2502.14566,
    2025) and the regression-based diagnostic of Moodie (Statistics in Medicine,
    2025). A value > 0 indicates a positivity violation at a_star.

    Parameters
    -----
```



```

a_star : the target intervention level (e.g., dose).
treatment : observed continuous treatment, shape (n,).
confounders : observed covariates W, shape (n, p).
density_estimator : f(W, A) -> conditional density f(A | W) at observed pairs.
coverage : fraction of the conditional A-mass that defines "supported".

Returns
-----
float in [0, 1] – fraction of units for whom a_star is unsupported.
"""
n = len(treatment)
unsupported = 0
for i in range(n):
    # build a per-unit support set {a : f(a|W_i) > tau} where tau gives `coverage`
    a_grid = np.linspace(treatment.min(), treatment.max(), 256)
    densities = density_estimator(
        np.tile(confounders[i:i+1], (256, 1)), a_grid
    )
    threshold = _coverage_threshold(densities, coverage)
    if density_estimator(confounders[i:i+1], np.array([a_star])) < threshold:
        unsupported += 1
return unsupported / n

```

18D.3 Longitudinal positivity

In longitudinal settings, positivity must hold at every time point conditional on the entire treatment-confounder history. Sequential positivity violations are subtle and frequent; we adopt the diagnostic suite from Chatton, Schomaker, Luque-Fernandez, Platt, Schnitzer (Epidemiology, November 2025): cumulative product of per-time positivity, propensity-density stability across folds, and explicit reporting of which time points carry the bulk of the risk.

18E. Feasible Estimand Translation

When the analyst's target estimand lies outside the Data Support Envelope, CausalRoadmap does not silently estimate something biased. It translates the target to the nearest feasible estimand and reports both: the original target, the translated target, the translation rule, and the gap. This is the architectural answer to the user's directive that "in the case of modified treatment policies it is not do analysis for data that isn't there."

18E.1 Translation rules by estimand class

Original target	If support is inadequate, translate to	Method
ATE under deterministic $A=1$	ATT (restrict to treated whose covariate support is observed)	DoWhy / EconML ATT
Continuous dose-response at a^*	Feasible CDRC: shift unsupported units to nearest supported dose	Khatibi-Wang-Rahmani feasible CDRC (2025)
MTP with shift $d(a,w) = a + \delta$	Truncated MTP that caps shifts at the observed support boundary	Imtp shift function with truncation
Longitudinal ATE under fixed regime	Cumulative cross-world weighted effect	McClean-Díaz (2025) cumulative cross-world weights
RMST contrast at horizon $\tau >$ follow-up	RMST at largest τ' where follow-up still has $> 30\%$ units at risk	survRM2 with truncated horizon
CATE in subgroup $X=x$ with $n_x < 50$	Stratified ATE with explicit small-n warning	EconML PolicyTree subgroup, plus refusal flag
NIE through mediator M with M post-treatment positivity violation	Direct effect only, with no mediation decomposition	DoWhy direct-effect identification

18E.2 The translation interface

```
from causalrag.core.estimand import CausalEstimand, FeasibleEstimand
from causalrag.core.envelope import DataSupportEnvelope
```

```
class EstimandTranslator:
    """Translate a target estimand into the nearest feasible one.

    The translator is consulted at Phase 4 Step 5 (identifiability) and again
    at Step 6 (statistical estimand definition). Translations are logged into
    the analyst-decision ledger.
    """
    def __init__(self, envelope: DataSupportEnvelope):
        self.envelope = envelope

    def translate(self, target: CausalEstimand) -> FeasibleEstimand:
```

```

    """Return a feasible estimand and the translation rationale."""
    verdict = self.envelope.check(target)
    if verdict.is_supported:
        return FeasibleEstimand(
            original=target, translated=target,
            gap=None, rule="no_translation_needed",
        )
    rule = self._select_rule(target, verdict)
    new_target = rule.apply(target, self.envelope)
    return FeasibleEstimand(
        original=target, translated=new_target,
        gap=verdict.gap_metrics, rule=rule.name,
        rationale=rule.human_explanation(verdict),
    )

def _select_rule(self, target, verdict):
    # dispatch on (target.type, verdict.violation_type) pair
    ...

```

18E.3 Visible to the analyst

When translation occurs, the report shows three things side by side: the original target estimand in the analyst's language, the translated estimand the package actually computed, and a one-paragraph explanation of why translation was necessary and what the gap implies for interpretation. The Causal Model Card flags any translated estimand with a yellow banner. An analyst who insists on the original target despite the warning can pass `--force-out-of-support`, but the report renders that decision in red on the analyst-decision ledger.

18F. The Genuineness Audit

The Genuineness Audit is a suite of six automated guards that run before any `EstimationResult` is allowed into the report. Their joint purpose is to detect analyses that look quantitative but rest on data the analyst does not have. An audit failure either blocks the result or attaches a red flag visible in every downstream artifact.

Guard	Detects	Failure action
G1 — Phantom-variable	Any column referenced in the DAG or estimand that is not in the Dataset Lock's schema	Block; raise <code>PhantomVariableError</code>
G2 — Empty-cell	Any subgroup or treatment arm with $n=0$ or below the analyst-set floor	Block; suggest stratified alternative
G3 — Out-of-support intervention	Continuous intervention level outside observed range, or MTP shift with non-overlap $>$ threshold	Block by default; translate via §18E if user accepts
G4 — Temporal violation	Confounder measured after treatment, mediator placed pre-treatment, follow-up exceeding observed window	Block; route back to Step 4 (<code>ObservedDataSpec</code>)
G5 — Identification gap	<code>DoWhy/Ananke identify_effect</code> returns <code>NoIdentificationException</code>	Block by default; allow <code>--allow-nonidentifiable</code> with prominent red banner
G6 — Imputation creep	Any analysis-time imputation of an outcome value	Block; require explicit missing-data plan with MNAR sensitivity

18F.1 Audit implementation

```
from causalrag.audit import (
    PhantomVariableError, EmptyCellError, OutOfSupportError,
    TemporalViolationError, NonIdentifiableError, ImputationCreepError,
)

class GenuinenessAudit:
    """Six-guard audit run before any EstimationResult enters the report.

    Each guard runs independently and emits an AuditFinding with severity
    'block' or 'warn'. Block-severity findings halt the pipeline.
    """
    def __init__(self, lock: DatasetLock, envelope: DataSupportEnvelope,
                 protocol: StudyProtocol):
        self.lock = lock
        self.envelope = envelope
        self.protocol = protocol

    def run(self, result: EstimationResult) -> AuditReport:
        findings: list[AuditFinding] = []
        findings.append(self._g1_phantom_variable(result))
        findings.append(self._g2_empty_cell(result))
```

```

        findings.append(self._g3_out_of_support(result))
        findings.append(self._g4_temporal_violation(result))
        findings.append(self._g5_identification_gap(result))
        findings.append(self._g6_imputation_creep(result))
        blocked = [f for f in findings if f.severity == "block"
                    and not f.user_override]
        if blocked:
            raise AuditBlockingError(blocked)
        return AuditReport(findings=findings, lock_fingerprint=self.lock.fingerprint)

def _g1_phantom_variable(self, result):
    # set difference of result.referenced_columns vs lock.column_dtypes
    phantoms = set(result.referenced_columns()) - set(self.lock.column_dtypes)
    if phantoms:
        return AuditFinding(
            guard="G1", severity="block",
            message=f"Result references {phantoms} not in dataset.",
        )
    return AuditFinding(guard="G1", severity="pass")

```

18F.2 Where the audit lives in the pipeline

The Genuineness Audit runs three times in every project:

- 74. After Phase 1 Discovery — checking that the proposed DAG, candidate roles, and feasibility report reference only columns in the Lock.
- 75. After Phase 3 Hypothesis generation — checking each hypothesis before it enters Phase 4.
- 76. After every Step 7 EstimationResult — the final, mandatory gate before the report sees the result.

18F.3 The audit log

Every audit run, pass or fail, writes an entry to `.causalrag/audit.jsonl`. The log is append-only (the writer holds an OS lock during write) and is rendered verbatim into the report's provenance section. An empty log is itself a red flag — it means audits were bypassed.

19. Comprehensive Method Catalog

This section enumerates every causal-inference method exposed by CausalRoadmap, organized by category and annotated with backend, supported estimand classes, and required data flags. The catalog is the source of truth for the registry; entries here correspond 1-to-1 with entry points in `causalrag.estimators`.

19.1 Identification & DAG-level

Method	Backend	Estimands	Notes
Back-door / front-door / IV identification	Py (DoWhy)	All	Run automatically in Step 5
ID algorithm (Shpitser-Pearl)	Py (Ananke)	All	ADMGs with latent confounders
gID algorithm (Bhattacharya-Nabi-Shpitser)	Py (Ananke)	All	Generalized identification
pywhy-graphs DAG/CPDAG/PAG	Py	—	Markov-equivalence class manipulation
Bayesian-network do() inference	Py (pyAgrum, pgmpy)	Decision-analytic	Discrete networks

19.2 ATE / ATT point estimation

Method	Backend	Required flags	Notes
IPW / IPTW	Py (DoWhy)	BINARY/ CAT_TREATMENT	Stabilized weights default
AIPW / Doubly Robust	Py (DoWhy, EconML)	BINARY/ CAT_TREATMENT	Default DR baseline
LinearDML	Py (EconML)	Any treatment + many controls	Original CausalRAG estimator
DoubleML (PartialLinearIRM)	Py (DoubleML)	Same as DML	Cluster-robust SEs available
TMLE3 (tmle3, drtmle)	R (tlverse)	BINARY_TREATMENT	Efficient SE; preferred in epi
g-computation / parametric g-formula	Py custom; R (gfoRmula)	LONGITUDINAL	Time-varying confounding
Propensity matching (NN/optimal/full/CEM/genetic/cardinality)	R (MatchIt)	BINARY_TREATMENT	Explainability-critical settings
Synthetic Control / Synth / GSC	Py (pysyncon, sparsesc); R (Synth, gsynth)	SINGLE_TREATED_UNIT + PANEL	Comparative case studies
CausalImpact (BSTS)	Py (tfcausalimpact); R (CausalImpact)	Time series with intervention point	Marketing lift; policy shocks
Marginal effects (delta-method)	R (marginaleffects)	Any model	Reporting after

Method	Backend	Required flags	Notes
			regression
DiD (canonical, staggered)	Py (differences); R (did, fect)	PANEL_STRUCTURE + binary treatment over time	Callaway-Sant'Anna for staggered

19.3 CATE / heterogeneous treatment effects

Method	Backend	Required flags	Notes
Causal Forest / GRF	Py (EconML.grf); R (grf)	BINARY_TREATMENT	Default HTE workhorse
S-/T-/X-learner (Künzel)	Py (EconML, CausalML)	BINARY_TREATMENT	Sklearn meta-learners
R-Learner (Nie-Wager)	Py (EconML)	BINARY_TREATMENT	Robinson decomposition for CATE
DR-Learner (Kennedy)	Py (EconML, DoubleML)	BINARY_TREATMENT	Strong DR with sklearn
BART / BCF (Hahn-Murray-Carvalho)	R (bartCause, bcf); Py (pymc-bart)	BINARY_TREATMENT	Bayesian; posterior CIs
SparseLinearDML / ForestDML / KernelDML	Py (EconML)	HIGH_DIMENSIONAL	Many features
Policy learning (PolicyForest, policytree)	Py (EconML); R (policytree)	BINARY_TREATMENT	Decision support

19.4 Survival-specific (ACIC 2026 emphasis)

Method	Backend	Notes
Causal Survival Forest (Cui et al. 2023, JRSS-B)	R (grf)	Canonical CATE for right-censored outcomes; targets RMST or survival probability; doubly robust w.r.t. censoring
RMST contrast (Royston-Parmar)	R (survRM2)	Marginal RMST difference up to horizon τ
IPCW-adjusted DML	Py wrapper + R nuisance	Time-to-event DML
TMLE for survival (survtmle, MOSS)	R	Doubly robust survival ATE at fixed t
Causal Cox (riskRegression, cmprsk)	R	Hazard ratios with confounding adjustment
Pseudo-observation methods	R (pseudo)	Marginal RMST / CIF regression
Survival meta-learners	Py custom + R	T/S/X with survival base models
Fine-Gray competing risks	R (cmprsk)	When COMPETING_RISKS flag set

19.5 IV methods

Method	Backend	When
2SLS / GMM	Py (linearmodels, statsmodels)	Linear IV
DeepIV / OrthoIV / IV-Forest	Py (EconML)	Nonparametric IV with ML nuisance
Wald estimator	Py	Binary IV, binary treatment
Mendelian randomization	R (MendelianRandomization, TwoSampleMR)	Genetic IV in epidemiology
MR-PRESSO	R	Pleiotropy-robust MR
IV sensitivity (iv.sensemakr)	R	OVb for IV

19.6 Mediation

Method	Backend	Notes
Imai-Keele NDE/NIE	R (mediation); Py (pymediation)	Sequential ignorability
CMAverse	R	Exposure-mediator interaction; multiple mediators
Natural / interventional effects via SCM	Py (DoWhy.gcm)	Mechanism analysis
Causal mediation with TMLE	R (tmle3mediate)	Doubly robust mediation

19.7 Beyond the ATE — stochastic interventions / MTPs

Method	Backend	Notes
Imtp (Díaz-Williams)	R	Binary, categorical, continuous, mixture exposures; point or longitudinal; competing risks
txshift	R	Continuous-exposure additive shifts
tmle3shift	R (tlverse)	Stochastic-shift interventions via TMLE3
Quantile g-computation	Py / R (qgcomp)	Mixture exposures
Counterfactual quantile / distributional effects	Py (DoWhy.gcm); R (DRDID)	Distributional impact

19.8 Sensitivity analysis

Method	Backend	What it bounds
E-value	Py (custom); R (EValue)	Minimum confounder strength to nullify
sensemakr (OLS)	Py (PySensemakr); R (sensemakr)	Partial-R ² OVb; robustness value
dml.sensemakr	R	Partial-R ² OVb for DML score
iv.sensemakr	R	OVb for IV

Method	Backend	What it bounds
Rosenbaum bounds	R (rbounds, sensitivitymw, sensitivitymv)	Bias in matched studies (Γ)
Manski bounds	Py (custom)	Worst-case partial identification
Bayesian bias priors	Py (PyMC custom)	Posterior under skeptical priors
Negative-control outcome (NCO)	R (tipr)	Confounding via known-null association
Specification curve / multiverse	Py custom + DoWhy refute	Distribution over modeling choices

19.9 Causal discovery

Method	Backend	Notes
PC, PC-stable	Py (causal-learn)	Constraint-based
FCI / RFCI	Py (causal-learn)	Latent-aware (PAG output)
GES, GIES, GRaSP	Py (causal-learn)	Score-based
NOTEARS / NOTEARS-MLP / DAGMA	Py (causal-learn, dagma)	Continuous-optimization
LiNGAM family (ICA, Direct, VAR, RCD)	Py (lingam)	Non-Gaussian linear
PCMCi+, LPCMCi	Py (Tigramite)	Time-series causal discovery
LLM-prior-augmented PC / GES (Jiralerspong-DFS, Vashishtha)	Py (this package)	LLM edge priors + statistical discovery
bnlearn structure learners	R (bnlearn)	Bayesian network structure

19.10 Multiple testing and aggregation

Method	Backend	Notes
Benjamini-Hochberg (BH)	Py (statsmodels)	Default FDR control
Benjamini-Yekutieli (BY)	Py (statsmodels)	Under positive regression dependence
Bonferroni / Holm	Py	FWER control
Knockoffs	Py (knockpy); R (knockoff)	FDR with variable-selection guarantees
Permutation / bootstrap	Py	Universal fallback

20. Method Ensembles & Triangulation

CausalRoadmap treats multi-method execution as the default analysis style in the academic and domain-expert tiers. Three forms of ensemble run:

20.1 Within-estimand ensemble

For a single causal estimand and a single DAG, run K (default 3) estimators chosen by the registry. Report:

- Each estimator's point estimate and CI.
- A fixed-effects inverse-variance meta-analytic combined point estimate and CI.
- Cochran Q heterogeneity statistic and I^2 indicating disagreement.
- Sign-consistency flag: do all K CIs share the same sign on zero?

20.2 Cross-DAG multiverse

For a single estimand and a single estimator, run across K candidate DAGs from Phase 1. Render as a specification-curve plot (Simonsohn et al.) with point estimates on the y-axis and DAG identifiers on the x-axis, accompanied by an annotation column showing which edges differ between DAGs.

20.3 Tripartite triangulation

The Lawlor et al. (2016, IJE) tripartite recipe — one DR/DML estimator, one matching-based estimator, one TMLE — runs as a special-purpose ensemble for the high-stakes findings. Triangulation outcomes:

- Consonant (all three CIs overlap with consistent sign): high robustness.
- Two-of-three consonant: moderate robustness; the outlier is investigated.
- Dissonant: low robustness; the report flags this prominently and recommends additional sensitivity work.

20.4 Aggregator output

A `MultiverseResult` object holds the full ensemble output and is what the reporter consumes. It contains every individual `EstimationResult`, the meta-analytic aggregates, the consonance verdict, and a serializable representation suitable for re-loading and re-analysis.

20A. Code-Out Method Catalog — Overview

Section 20A through 20K extend the method catalog of §19 with full code-out signatures, real package names, PhD-level method variants, and explicit guidance on when each method is appropriate vs. inappropriate. Where two competing modern methods disagree (e.g., causal forest vs. modified causal forest), both are exposed and the package runs them in parallel by default in the academic tier.

Every method in this section ships as a CausalEstimator-protocol-compliant adapter under `causalrag.estimators`. The code blocks below are normative — they show the actual signature the v0.5 release will expose. Internal implementation details (cross-fitting, bootstrap, R session management) are factored into shared utilities documented in §17.

Methods are organized into eleven categories aligned with §19, with each category expanded into PhD-level depth: §20B point-treatment ATE/ATT; §20C heterogeneous effects (CATE); §20D continuous treatment / dose-response; §20E modified treatment policies; §20F survival; §20G mediation; §20H instrumental variables; §20I causal discovery; §20J sensitivity analysis; §20K conformal & honest inference.

20B. Point-Treatment ATE / ATT — Full Method Family

For binary treatment and any outcome type, the package exposes the full DR + TMLE + matching family. The default in the data-scientist tier is AIPW. The academic tier runs an ensemble — AIPW, LinearDML, TMLE, MatchIt — and reports a triangulated estimate.

20B.1 AIPW (Doubly Robust)

```
from __future__ import annotations
import numpy as np, pandas as pd
from typing import Sequence
from sklearn.base import BaseEstimator
from causalrag.core.result import EstimationResult
from causalrag.core.flags import DataFlag

def aipw_ate(
    data: pd.DataFrame, *,
    treatment: str, outcome: str, confounders: Sequence[str],
    outcome_model: BaseEstimator,      # e.g., HistGradientBoostingRegressor
    propensity_model: BaseEstimator,    # e.g., LogisticRegressionCV
    cv: int = 5, trim: tuple[float, float] = (0.02, 0.98),
    random_state: int = 17,
) -> EstimationResult:
    """Augmented IPW for ATE under binary treatment (Robins-Rotnitzky-Zhao 1994).

    Doubly robust: consistent if either outcome or propensity model is correct.
    Cross-fitted nuisance estimation per Chernozhukov et al. 2018, Algorithm 3.
    Trims propensity scores at `trim` to defend against positivity violations.
    """
    from sklearn.model_selection import KFold
    n = len(data)
    W = data[list(confounders)].values; A = data[treatment].values; Y = data[outcome].values
    psi = np.empty(n); mu0 = np.empty(n); mu1 = np.empty(n); g = np.empty(n)
    kf = KFold(n_splits=cv, shuffle=True, random_state=random_state)
    for train_idx, test_idx in kf.split(W):
        om = clone(outcome_model).fit(np.column_stack([W[train_idx], A[train_idx]]),
Y[train_idx])
        pm = clone(propensity_model).fit(W[train_idx], A[train_idx])
        mu0[test_idx] = om.predict(np.column_stack([W[test_idx], np.zeros(len(test_idx))]))
        mu1[test_idx] = om.predict(np.column_stack([W[test_idx], np.ones(len(test_idx))]))
        g[test_idx] = np.clip(pm.predict_proba(W[test_idx])[:, 1], 1e-5, 1 - 1e-5) * trim
    psi = (mu1 - mu0) + A * (Y - mu1) / g - (1 - A) * (Y - mu0) / (1 - g)
    return EstimationResult.from_influence_curve(psi, estimand="ATE",
                                                diagnostics={"min_g": g.min(), "max_g":
g.max()})
```

20B.2 TMLE (Targeted Minimum Loss Estimation) — preferred for efficient inference

TMLE is the standard van der Laan estimator. The Python ecosystem now has zEpid for cross-sectional TMLE; the canonical implementation remains tlverse/tmle3 in R, exposed here via rpy2.

```
def tmle_ate(
    data: pd.DataFrame, *,
    treatment: str, outcome: str, confounders: Sequence[str],
    backend: Literal["python", "r"] = "r",
    super_learner_library: list[str] = ("SL.glmnet", "SL.ranger", "SL.xgboost"),
) -> EstimationResult:
    """Targeted Minimum Loss Estimator for the ATE (van der Laan & Rubin 2006).

    Doubly robust + efficient + targeted: corrects bias in the initial outcome
    estimate using a one-step parametric update along the score of the propensity.
    The R backend (tlverse/tmle3 + sl3) is the reference implementation.
    """
    if backend == "r":
        from causalrag.estimators.rbridge._r import require_r_package
        tmle3 = require_r_package("tmle3")
        sl3 = require_r_package("sl3")
        # ... build sl3 learner stacks; call tmle3::tmle3_Spec_ATE() ...
        # ... extract psi, std error, influence curve ...
        return EstimationResult(...)
    else:
        from zepid.causal.doublyrobust import TMLE          # python fallback
        m = TMLE(data, exposure=treatment, outcome=outcome)
        m.exposure_model(" + ".join(confounders))
        m.outcome_model(" + ".join(confounders))
        m.fit()
        return EstimationResult(point=m.average_treatment_effect,
                                ci=(m.average_treatment_effect_ci),
                                se=m.average_treatment_effect_se, estimand="ATE")
```

20B.3 LinearDML (debiased ML) — the CausalRAG original

```
def linear_dml_ate(
    data: pd.DataFrame, *,
    treatment: str, outcome: str, confounders: Sequence[str],
    effect_modifiers: Sequence[str] = (),
    discrete_treatment: bool = True,
    cv: int = 5,
) -> EstimationResult:
    """LinearDML (Chernozhukov et al. 2018) via EconML.

    Two-stage cross-fitted estimator: stage 1 fits  $E[Y|W]$  and  $E[A|W]$  with
    arbitrary ML; stage 2 regresses residuals with a linear final model.
```

```

Inference from the influence function; this is the estimator used in
the CausalRAG poster.
"""
from econml.dml import LinearDML
from sklearn.ensemble import GradientBoostingRegressor, GradientBoostingClassifier
W = data[list(confounders)].values; X = data[list(effect_modifiers)].values if
effect_modifiers else None
A = data[treatment].values; Y = data[outcome].values
est = LinearDML(
    model_y=GradientBoostingRegressor(n_estimators=100, max_depth=4),
    model_t=GradientBoostingClassifier(n_estimators=100, max_depth=4) if
discrete_treatment else GradientBoostingRegressor(),
    discrete_treatment=discrete_treatment, cv=cv, random_state=17,
)
est.fit(Y=Y, T=A, X=X, W=W)
point = float(est.ate(X=X))
lo, hi = est.ate_interval(X=X, alpha=0.05)
return EstimationResult(point=point, ci=(float(lo), float(hi)),
                        estimand="ATE", backend="econml.LinearDML")

```

20B.4 PhD-grade comparison rules

When two of the above estimators disagree by more than 25% of their pooled CI width, the package emits a triangulation warning and reports both. In the academic tier, all four (AIPW, TMLE, LinearDML, MatchIt) run by default and the report contains a tripartite-style table (Lawlor et al. 2016).

20C. Heterogeneous Treatment Effects (CATE) — Full Method Family

For CATE the package exposes four families: meta-learners, causal forests (multiple variants), DR-learner, and BART-based Bayesian Causal Forests. The PhD-grade addition is the Modified Causal Forest (Lechner) and Bayesian Causal Forest for longitudinal data (BCFLong, 2025).

20C.1 Causal Forest (grf) — the canonical baseline

```
def causal_forest_cate(
    data: pd.DataFrame, *,
    treatment: str, outcome: str, confounders: Sequence[str],
    n_trees: int = 4000, honest: bool = True,
    tune_parameters: Literal["none", "all"] = "all",
) -> EstimationResult:
    """Generalized Random Forests CATE (Athey, Tibshirani, Wager 2019).

    Uses the canonical grf R package via rpy2 – Python EconML port is a
    reimplementation, not a wrapper, and diverges in places. The R version
    is the reference; we expose both and warn if they disagree.
    """

    from causalrag.estimators.rbridge._r import require_r_package
    grf = require_r_package("grf")
    # ... pandas -> R data.frame via pandas2ri ...
    # ... cf <- grf::causal_forest(W, Y, A, num.trees=n_trees, honesty=honest) ...
    # ... predict and extract influence curves ...
    return EstimationResult(...)
```

20C.2 Modified Causal Forest (mcf) — Lechner's efficiency improvement

```
def modified_causal_forest_cate(
    data: pd.DataFrame, *,
    treatment: str, outcome: str, confounders: Sequence[str],
    effect_modifiers: Sequence[str] = (),
) -> EstimationResult:
    """Modified Causal Forest (Lechner 2019; Bodory, Busshoff, Lechner 2022).

    Adapts the splitting criterion to account for covariance of estimation
    errors and selection bias. Python implementation: `mcf` package on PyPI.
    Provides inference at all aggregation levels (ATE, GATE, IATE, BGATE).
    Recommended over causal_forest when treatment selection is strong.
    """

    from mcf import ModifiedCausalForest    # pip install mcf
    mcf = ModifiedCausalForest(
        var_y_name=outcome, var_d_name=treatment,
```

```

        var_x_name_ord=list(confounders),
        var_z_name_list=list(effect_modifiers),
    )
    mcf.train(data)
    results, _, _ = mcf.predict(data)
    return EstimationResult.from_mcf_results(results, estimand="CATE")

```

20C.3 R-Learner (Nie-Wager)

```

def r_learner_cate(
    data: pd.DataFrame, *,
    treatment: str, outcome: str, confounders: Sequence[str],
    effect_modifiers: Sequence[str],
    final_stage: Literal["forest", "linear", "xgb"] = "forest",
) -> EstimationResult:
    """R-Learner (Nie & Wager 2021, Biometrika): quasi-oracle CATE via
    Robinson decomposition + arbitrary final-stage learner.
    """
    from econml.dml import NonParamDML
    # ... fit per Robinson; final stage chosen by `final_stage` ...
    return EstimationResult(...)

```

20C.4 Bayesian Causal Forest (BCF) — Hahn, Murray, Carvalho

```

def bayesian_causal_forest_cate(
    data: pd.DataFrame, *,
    treatment: str, outcome: str, confounders: Sequence[str],
    n_samples: int = 4000, n_burn: int = 2000,
) -> EstimationResult:
    """Bayesian Causal Forest (Hahn, Murray, Carvalho 2020, Bayesian Analysis).

    Decomposes outcome into prognostic and treatment-effect surfaces, each
    modeled with BART. Provides full posterior credible intervals – useful
    when uncertainty quantification matters (small n, high stakes).
    R implementation: bcf package. Python: stochtree.
    """
    import stochtree # pip install stochtree
    bcf = stochtree.BCFModel()
    bcf.sample(X=W, Z=A, y=Y, num_gfr=10, num_burnin=n_burn, num_mcmc=n_samples)
    return EstimationResult.from_posterior(bcf.tau_hat_train, estimand="CATE")

```

20C.5 BCFLong — Bayesian Causal Forest for longitudinal data (2025)

BCFLong (Caron, Mealli, Pacciaroni, Papaspiliopoulos 2025) extends BCF with hierarchical random effects per subject — random intercept + time-dependent slope with a horseshoe sparsity prior. The motivating dataset is NO.MS, the largest multiple sclerosis clinical trial database (35,000 patients, 15-year follow-up). For longitudinal observational data with repeated outcome measurements, this is the current PhD-grade Bayesian baseline.


```

def bcf_long_cate(
    data: pd.DataFrame, *,
    treatment: str, outcome: str, confounders: Sequence[str],
    subject_id: str, time: str,
    n_samples: int = 4000,
) -> EstimationResult:
    """BCFLong (Caron et al. 2025, arXiv:2508.08418).

    Hierarchical BCF with random intercept + time-dependent slope per
    subject and horseshoe sparsity prior. Designed for longitudinal data
    with within-individual correlation. Reference implementation in R.
    """

    from causalrag.estimators.rbridge._r import require_r_package
    bcflong = require_r_package("BCFLong")  # or fork until on CRAN
    # ... R-side fit + extract per-subject tau trajectories ...
    return EstimationResult(...)

```

20C.6 RATE — Rank-weighted ATE (Yadlowsky et al. 2025)

```

def rank_weighted_ate(
    cate_estimator: Callable, *,
    data: pd.DataFrame,
    weight_scheme: Literal["AUTOC", "QINI"] = "AUTOC",
) -> EstimationResult:
    """Rank-Weighted Average Treatment Effect (Yadlowsky, Fleming, Shah,
    Brunskill, Wager 2025, JASA).

    Evaluates a CATE-based treatment prioritization rule by computing the
    expected gain of treating the top-q% (by predicted CATE). Doubly robust.
    Reference implementation: grf::rank_average_treatment_effect.
    """

    # ... fit cate_estimator on splits, evaluate prioritization, return AUTOC/QINI ...

```

20D. Continuous Treatment & Dose-Response

Continuous treatment is where naive ATE collapses and where the package's data-locking discipline is most consequential. The Data Support Envelope is enforced rigorously: dose-response is reported only at values where the conditional density $f(A|W)$ has adequate mass; values outside support are translated via the feasible CDRC of §18E or flagged with a hard banner.

20D.1 Generalized propensity score + outcome regression

```
def gps_dose_response(
    data: pd.DataFrame, *,
    treatment: str, outcome: str, confounders: Sequence[str],
    a_grid: np.ndarray,
    propensity_model: Literal["gaussian", "gamma", "density"] = "density",
) -> EstimationResult:
    """Hirano-Imbens generalized propensity score for continuous treatment.

    Parametric baseline; superseded by Colangelo-Lee 2025 DML (§20D.3) for
    PhD-level work. Retained for low-n settings and comparison.
    """
    # ... fit  $f(A|W)$ ; compute GPS at observed  $(A,W)$ ; regress  $Y$  on  $(A, \text{GPS})$  ...
    return EstimationResult(...)
```

20D.2 Causal Dose-Response via HAL plug-in (van der Laan, Shi-Phillips 2024)

Highly Adaptive Lasso has the unique theoretical property of providing valid statistical inference for the causal dose-response curve — a non-pathwise-differentiable parameter where standard $\sqrt{n}$ estimators do not exist. This is the PhD-grade option.

```
def hal_dose_response(
    data: pd.DataFrame, *,
    treatment: str, outcome: str, confounders: Sequence[str],
    a_grid: np.ndarray,
) -> EstimationResult:
    """HAL-based plug-in CDRC estimator (Shi, Phillips, van der Laan 2024).

    Highly Adaptive Lasso is empirically Donsker and achieves  $o(n^{-1/4})$ 
    convergence; van der Laan 2023 establishes pointwise asymptotic normality
    of the HAL plug-in for the marginal dose-response curve.

    Reference R implementation: hal9001 + augmented HAL plug-in (SeraphinaShi
    GitHub: HAL-in-R). No production Python port yet.
    """
    from causalrag.estimators.rbridge._r import require_r_package
    hal9001 = require_r_package("hal9001")
```

```
# ... fit HAL on (Y, A, W); construct undersmoothed plug-in at each a in a_grid ...
return EstimationResult.dose_response(curve=..., ci=..., method="HAL")
```

20D.3 Colangelo-Lee 2025 nonparametric continuous DML

```
def colangelo_lee_dml(
    data: pd.DataFrame, *,
    treatment: str, outcome: str, confounders: Sequence[str],
    a_target: float, bandwidth: float | None = None,
) -> EstimationResult:
    """Double Debiased ML for continuous treatment dose-response.

    Colangelo & Lee (JBES 2025). Kernel-localized DML for the average
    structural function  $E[Y(a^*)]$ ; provides asymptotic normality at
    nonparametric rate. Bandwidth via plug-in optimal  $h$ .
    """
    # ... kernel-localize at a_target; fit nuisance via SuperLearner or GBM ...
    # ... compute doubly robust moment; return point + CI ...
    return EstimationResult(...)
```

20D.4 Feasible CDRC under positivity violations (Khatibi-Wang-Rahmani 2025)

```
def feasible_cdrc(
    data: pd.DataFrame, *,
    treatment: str, outcome: str, confounders: Sequence[str],
    a_grid: np.ndarray, coverage: float = 0.95,
) -> EstimationResult:
    """Feasible dose-response curve (arXiv:2502.14566, 2025).

    For each (unit, a*) pair, if a* is outside the unit's support, map to
    the nearest supported value. Estimand becomes the response to an
    individualized-most-feasible intervention. Returned alongside the
    non-overlap ratio so the analyst sees the gap.
    """
    from causalrag.envelope import non_overlap_ratio
    # ... for each a* in a_grid, compute non_overlap_ratio(a*, ...) ...
    # ... reassign unsupported units to nearest supported a; plug-in g-formula ...
    return EstimationResult.dose_response(curve=..., ci=..., method="feasible_CDRC",
                                          non_overlap=non_overlap_curve)
```

20E. Modified Treatment Policies & Stochastic Interventions

MTPs (Haneuse-Rotnitzky 2013) and LMTPs (Díaz, Williams, Hoffman, Schenck 2021, JASA) are the principled framework for continuous, multivalued, and mixture exposures. They are also the methodological centerpiece of the package's data-locking architecture: MTPs are designed so that the intervention shift $d(a, w)$ keeps the joint (a, w) inside the observed support, satisfying positivity by construction.

20E.1 LMTP — the canonical implementation

```
def lmtpl_effect(
    data: pd.DataFrame, *,
    treatment: str | list[str],          # supports time-varying
    outcome: str,
    confounders: list[str] | list[list[str]], # baseline or per-time
    censoring: str | None = None,
    competing_event: str | None = None,
    shift_function: Callable[[np.ndarray, np.ndarray], np.ndarray] = None,
    estimator: Literal["tmle", "sdr", "sub", "ipw"] = "tmle",
    super_learner_library: list[str] = ("SL.glmnet", "SL.ranger", "SL.xgboost"),
    folds: int = 10,
) -> EstimationResult:
    """Longitudinal Modified Treatment Policy (Díaz et al. 2021 JASA).

    The most general estimator in the package: binary / continuous / mixture
    exposures; point or longitudinal; survival, binary, or continuous outcomes;
    competing risks and informative censoring. R backend: lmtpl.

    The `shift_function` MUST be designed against the Data Support Envelope.
    The package rejects shift functions that move units out of observed support
    (per §18E feasible estimand translation). Default truncation: shifts are
    capped at the 95% conditional support boundary.
    """
    from causalrag.estimators.rbridge._r import require_r_package
    from causalrag.envelope import validate_shift_function
    validate_shift_function(shift_function, data, treatment, confounders) # raises if OOS
    lmtpl = require_r_package("lmtpl")
    SuperLearner = require_r_package("SuperLearner")
    # ... build R-side shift, learners, call lmtpl::lmtpl_tmle (or _sdr/_sub/_ipw) ...
    # ... extract psi, std error, EIF; build EstimationResult ...
    return EstimationResult(...)
```

20E.2 Cumulative cross-world weighted effects (McClean-Díaz 2025)

```
def cumulative_cross_world_effect(
    data: pd.DataFrame, *,
    regime_a: Callable, regime_b: Callable,
```

```

    treatment: list[str], outcome: str,
    confounders: list[list[str]],
) -> EstimationResult:
    """Cumulative cross-world weighted effect (McClellan & Díaz 2025).

    Identifies a contrast between two longitudinal regimes WITHOUT requiring
    full positivity at every time point. Achieves doubly-robust  $\sqrt{n}$  inference.
    Use when comparing two non-implementable regimes (e.g., always-on vs.
    always-off) and positivity fails in one or both.
    """

    # ... estimate cross-world propensity, run DR-style update ...
    return EstimationResult(...)

```

20F. Survival & Time-to-Event

Survival is treated as a first-class flag (RIGHT_CENSORED_OUTCOME). The default estimator is the Causal Survival Forest (Cui, Kosorok, Sverdrup, Wager, Zhu 2023 JRSS-B); secondary options include RMST contrast and longitudinal TMLE for survival.

20F.1 Causal Survival Forest (Cui et al. 2023)

```
def causal_survival_forest(
    data: pd.DataFrame, *,
    treatment: str, time: str, event: str, confounders: Sequence[str],
    horizon: float | None = None,
    target: Literal["RMST", "survival_probability"] = "RMST",
    n_trees: int = 4000,
) -> EstimationResult:
    """Causal Survival Forest (Cui et al. 2023 JRSS-B).

    Doubly robust w.r.t. censoring. Reference: grf::causal_survival_forest.
    Targets RMST( $\tau$ ) or survival probability at  $\tau$ . Horizon truncated to
    largest  $\tau$  where adequate at-risk population remains (Data Support
    Envelope §18C).
    """
    from causalrag.estimators.rbridge._r import require_r_package
    grf = require_r_package("grf")
    if horizon is None:
        horizon = float(np.quantile(data.loc[data[event] == 1, time], 0.95))
    # ... call grf::causal_survival_forest, get average_treatment_effect ...
    return EstimationResult(...)
```

20F.2 RMST contrast

```
def rmst_contrast(
    data: pd.DataFrame, *,
    treatment: str, time: str, event: str,
    horizon: float, adjust: Sequence[str] | None = None,
) -> EstimationResult:
    """Marginal RMST contrast up to horizon  $\tau$  (Royston-Parmar).

    R backend: survRM2 (unadjusted) or adjusted via IPCW-weighted Kaplan-Meier.
    """
    from causalrag.estimators.rbridge._r import require_r_package
    survRM2 = require_r_package("survRM2")
    # ... call survRM2::rmst2() ...
    return EstimationResult(...)
```

20F.3 Longitudinal TMLE for survival (LTMLE)

```
def ltmle_survival(
```

```

    data: pd.DataFrame, *,
    treatment_vars: list[str], outcome_var: str, censoring_vars: list[str],
    confounder_vars_by_time: list[list[str]],
    regime: list[Any],
    super_learner_library: list[str] = ("SL.glmnet", "SL.ranger"),
) -> EstimationResult:
    """Longitudinal Targeted Minimum Loss Estimation for survival.

    R backend: ltmle package + sl3. Handles time-varying confounding,
    informative censoring, and arbitrary regime contrasts. Standard tool
    for adaptive treatment strategies (Zhang et al. 2025, Int J Tech Asses).
    """

    from causalrag.estimators.rbridge._r import require_r_package
    ltmle = require_r_package("ltmle")
    # ... build long-form data.frame; call ltmle::ltmle ...
    return EstimationResult(...)

```

20G. Mediation Analysis

20G.1 Imai-Keele-Tingley NDE / NIE

```
def imai_keelee_mediation(
    data: pd.DataFrame, *,
    treatment: str, mediator: str, outcome: str, confounders: Sequence[str],
    boot: int = 1000,
) -> EstimationResult:
    """NDE and NIE under sequential ignorability (Imai et al. 2010 Psych Methods).

    R backend: mediation package. Sensitivity analysis via medsens.
    """
    from causalrag.estimators.rbridge._r import require_r_package
    mediation = require_r_package("mediation")
    # ... fit outcome model + mediator model; mediation::mediate; medsens ...
    return EstimationResult(...)
```

20G.2 TMLE for natural (in)direct effects — tmle3mediate

```
def tmle3_mediation(
    data: pd.DataFrame, *,
    treatment: str, mediator: str, outcome: str, confounders: Sequence[str],
    estimand: Literal["NDE", "NIE", "PIE", "TE"] = "NDE",
) -> EstimationResult:
    """TMLE-based mediation (Hejazi, Coyle, van der Laan 2022).

    R backend: tlverse/tmle3mediate. Doubly robust; valid inference with
    super learner nuisance estimation.
    """
    from causalrag.estimators.rbridge._r import require_r_package
    tmle3mediate = require_r_package("tmle3mediate")
    # ... build spec; run tmle3 with mediation spec ...
    return EstimationResult(...)
```

20G.3 Longitudinal mediation with HAL-EIC (Ye & van der Laan 2023)

For longitudinal mediation with time-varying confounders, competing risks, and informative censoring, Ye & van der Laan's HAL-EIC one-step TMLE (arXiv 2304.04904) is the PhD-grade method. Doubly robust, locally efficient, and provides asymptotic normality under realistic conditions. Reference implementation in R; exposed via rpy2.

20H. Instrumental Variables

20H.1 2SLS

```
def two_stage_least_squares(  
    data: pd.DataFrame, *,  
    instrument: str, treatment: str, outcome: str, covariates: Sequence[str] = (),  
) -> EstimationResult:  
    """Classic linear IV via 2SLS. linearmodels.IV2SLS reference."""  
    from linearmodels import IV2SLS  
    # ... call IV2SLS.from_formula ...  
    return EstimationResult(...)
```

20H.2 DeepIV / OrthoIV / IV-Forest

```
def deep_iv(  
    data: pd.DataFrame, *,  
    instrument: str, treatment: str, outcome: str, covariates: Sequence[str],  
) -> EstimationResult:  
    """Hartford-Lewis-Leyton-Brown-Taddy 2017 DeepIV via EconML."""  
    from econml.iv.nnet import DeepIVEstimator  
    # ... build treatment NN, response NN; fit ...  
    return EstimationResult(...)
```

20H.3 MR-PRESSO for Mendelian randomization

For genetic IV (Mendelian randomization), the package bridges MR-PRESSO (Verbanck, Chen, Neale, Do 2018) via rpy2. Detects and removes horizontal pleiotropy outliers; produces both raw and corrected estimates. Used in epidemiology and increasingly in clinical research.

20I. Causal Discovery — Constraint, Score, Hybrid

The package exposes the full causal-learn suite plus PhD-grade additions: NOTEARS, DAGMA, DECI (variational), DAGPA (NeurIPS 2025 differentiable constraint-based).

20I.1 PC and FCI

```
def pc_discovery(
    data: pd.DataFrame, *,
    alpha: float = 0.05,
    independence_test: Literal["fisherz", "chisq", "gsq", "kci"] = "fisherz",
) -> CausalGraph:
    """PC algorithm via causal-learn (Spirtes-Glymour 1991)."""
    from causallearn.search.ConstraintBased.PC import pc
    cg = pc(data.values, alpha=alpha, indep_test=independence_test)
    return CausalGraph.from_cpdag(cg.G, names=list(data.columns))
```

20I.2 NOTEARS / DAGMA / DAGPA — continuous optimization

```
def dagma_discovery(data: pd.DataFrame, *, lambdal: float = 0.05) -> CausalGraph:
    """DAGMA (Bello, Aragam, Ravikumar 2022 NeurIPS) via dagma package."""
    from dagma.linear import DagmaLinear          # pip install dagma
    model = DagmaLinear(loss_type="l2")
    W_est = model.fit(data.values, lambdal=lambdal)
    return CausalGraph.from_weight_matrix(W_est, names=list(data.columns))

def dagpa_discovery(data: pd.DataFrame, *, top_k: int = 5) -> list[CausalGraph]:
    """DAGPA (Zhou-Wang NeurIPS 2025) – differentiable constraint-based.

    Reference: PurdueMINDS/DAGPA. Hybrid that combines NOTEARS-style
    optimization with conditional-independence agreement scoring.
    Returns the top-k DAGs ranked by CI agreement.
    """
    # ... currently requires forking the repo; install hook in causalrag doctor ...
```

20I.3 DECI — variational causal discovery

DECI (Geffner et al. 2022) is the variational-inference reference for nonlinear causal discovery with non-Gaussian errors. Available via the causica package from Microsoft Research. Used when the analyst wants posterior uncertainty over the graph rather than a point estimate.

20I.4 LLM-prior fusion

```
def llm_prior_pc(
    data: pd.DataFrame, *,
    domain_brief: str,
    backend: "OllamaClient",
    base_algorithm: Literal["pc", "ges", "dagma"] = "pc",
```

```

) -> CausalGraph:
    """LLM-prior augmented discovery (Jiralerspong et al. 2024 + Ban et al. 2025).

    1. Ask the LLM for an edge-presence prior over column pairs.
    2. Run the statistical discovery algorithm with the prior as a tiebreaker
       on ambiguous independence tests.
    3. Validate every LLM-suggested edge against a CI test on the data.
    4. Surface disagreements rather than overriding.
    """
    prior = backend.query_edge_prior(list(data.columns), brief=domain_brief)
    base = pc_discovery(data) if base_algorithm == "pc" else dagma_discovery(data)
    return CausalGraph.merge_with_prior(base, prior, ci_validator=fisher_z_ci)

```

20J. Sensitivity Analysis — Full Suite

20J.1 E-value

```
def e_value(
    *, point: float, lo: float, hi: float,
    scale: Literal["RR", "OR", "HR", "diff"] = "RR",
) -> dict[str, float]:
    """VanderWeele-Ding 2017 E-value."""
    # ... per the published formula by scale type ...
    return {"e_value": ..., "e_value_ci": ...}
```

20J.2 sensemakr (OLS, DML, IV variants)

```
def sensemakr_ols(model, *, treatment: str, benchmark_covariates: list[str]) -> dict:
    """OLS sensitivity (Cinelli-Hazlett 2020 JRSS-B). PySensemakr Python; R fallback."""
    from sensemakr import Sensemakr # PySensemakr
    s = Sensemakr(model=model, treatment=treatment,
benchmark_covariates=benchmark_covariates)
    return {"robustness_value": s.robustness_value, "contour": s.plot_contour()}

def sensemakr_dml(*, score_function, nuisance_estimates) -> dict:
    """DML sensitivity (Chernozhukov, Cinelli, Newey, Sharma, Syrgkanis 2022).
    R only as of 2026; dml.sensemakr package."""
    from causalrag.estimators.rbridge._r import require_r_package
    dml_sm = require_r_package("dml.sensemakr")
    # ...
```

20J.3 Rosenbaum bounds + Bayesian bias priors

Rosenbaum bounds bridge from R packages `rbounds`, `sensitivitymw`, `sensitivitymv`. Bayesian bias-parameter priors built natively in PyMC: `bias_parameter ~ Normal(0, 0.5)` on the log-RR scale by default; posterior of ATE is marginalized over the prior.

20K. Conformal & Distribution-Free Inference

Conformal causal inference (Lei & Candès 2021, Lei, Sant'Anna, Wasserman 2024) provides distribution-free finite-sample coverage guarantees on counterfactual outcomes and ITEs. The PhD-level addition for high-stakes decisions where standard asymptotic CIs are not enough.

```
def conformal_ite(
    data: pd.DataFrame, *,
    treatment: str, outcome: str, confounders: Sequence[str],
    cate_model: BaseEstimator, alpha: float = 0.10,
) -> pd.DataFrame:
    """Conformal prediction intervals on individual treatment effects.

    Lei-Candès 2021 weighted conformal procedure with treatment-effect
    nonconformity. Returns per-row [lo, hi] with finite-sample coverage
     $\geq 1 - \alpha$  under exchangeability of  $(Y(1)-Y(0))$  residuals.
    Reference: nlamprou/conformalInference.fd; CausalConformal in R.
    """
    # ... split-conformal with weighted residuals; return intervals ...
```

21. CLI Design

The CLI is the primary user surface. It is built on Typer, with Rich for formatted output and Questionary for interactive prompts. Every command updates the StudyProtocol; every command emits an actionable next-step hint.

21.1 Command tree

```
causalrag --help
causalrag init [--name NAME] [--tier data-scientist|academic|domain-expert|auto]
causalrag doctor [--install-r-deps] [--install-ollama-models] [--json]
causalrag config get|set KEY [VALUE]

# Phase 1
causalrag discover DATASET
    [--question "..."] [--data-dictionary FILE]
    [--interactive] [--k-dags 3]
    [--treatment COL] [--outcome COL]
    [--exclude COL,...] [--include COL,...]

# Phase 2
causalrag feasibility
    [--mode default|manual|llm-calibrated]
    [--n-floor 100] [--power-floor 0.70] [--alpha 0.05]
    [--effect-band-source user|llm|empirical]

# Phase 3
causalrag hypothesize
    --mode {manual|automated|hybrid}
    [--n 10] [--counterfactual-ratio 0.30]
    [--estimand-mix ATE:0.3,CATE:0.4,RMST:0.2,NDE:0.1]
    [--seed 17]
causalrag hypothesize add
    --treatment COL --outcome COL --question "..."
    [--estimand CATE --modifier COL]

# Phase 4 – full Roadmap walk
causalrag estimate
    [--hypothesis H03|all]
    [--ensemble]                # cross-estimator triangulation
    [--multiverse-dags]          # cross-DAG multiverse
    [--cv-folds 5] [--bootstrap 1000]

# Phase 5
causalrag sensitivity
    [--methods evaluel, sensemakr, rosenbaum, multiverse, bayes]
    [--benchmark-covariates COL,...]

# Phase 6
```

```
causalrag report
  [--format html|pdf|md|quarto]
  [--include-llm-rationales]
  [--decision-ledger-position top|bottom]

# One-shot
causalrag run DATASET
  [--question "..."]
  [--auto-confirm]

# Interactive
causalrag repl

# Validation
causalrag validate [STUDY_YAML]

# Reproduction
causalrag replay PROTOCOL_FILE
```

21.2 Example workflows

21.2.1 Quick look (data-scientist tier)

```
$ causalrag run claims_2024.parquet --question "what drives 30-day readmission?"
[ok] discovered 47 columns, emitted 5 candidate DAGs
[ok] feasibility passed for 8 (treatment, outcome) pairs
[ok] generated 10 hypotheses (3 counterfactual)
[ok] ran 10 Causal Roadmap walks (avg 47s per walk)
[ok] report written to reports/study_2026-05-13.html
```

Top finding: hospital_followup → readmit_30d, ATE = -0.087 [-0.121, -0.054],
BH-adjusted p = 0.003, E-value = 1.94 (yellow verdict).

21.2.2 Academic tier with explicit control

```
$ causalrag init --tier academic
$ causalrag discover trial.csv --treatment arm --outcome death_30d \
  --question "Does intervention affect 30-day mortality?" --k-dags 5
$ causalrag feasibility --mode manual --n-floor 150 --power-floor 0.80
$ causalrag hypothesize --mode hybrid --n 6 --counterfactual-ratio 0.50
$ causalrag estimate --hypothesis all --ensemble --multiverse-dags
$ causalrag sensitivity --methods evalua,sensemakr,rosenbaum,bayes,multiverse
$ causalrag report --format pdf --include-llm-rationales
```

22. Tiered Defaults

Three user tiers control defaults, prompt verbosity, surfaced knobs, and required confirmations. Every capability is reachable from every tier; the tier only governs what is enabled out of the box.

Setting	data-scientist (default)	academic	domain-expert
Default mode	hybrid	academic ensemble	manual with checklists
Hypotheses N	10	configurable, defaults 6	1-3 with explicit guidance
Estimator strategy	top-1	top-3 ensemble + multiverse	top-1 with explanation
Sensitivity defaults	E-value + sensemakr	all defensible methods	E-value with plain-language verdict
Multiple testing	BH	BH + Bonferroni shown side by side	BH; non-significant suppressed unless asked
LLM acceptance	auto-accept	require confirmation on role assignments	require confirmation on every step
Report verbosity	concise HTML	full HTML + LaTeX-ready tables	narrated; minimal jargon; glossary inline
Decision ledger	bottom of report	top of report	rendered as a step-by-step checklist
DAG visualization	rank-1 only by default	all K candidate DAGs side by side	rank-1 with plain-language edge labels
Glossary links	off	on	on (every term)

22.1 Tier auto-detection

`causalrag init --tier auto` runs an inference:

- If R + tlverse are installed → academic.
- If terminal is non-interactive (CI environment) → data-scientist.
- If the user's git config shows a non-engineering domain or there's a known clinical tool (REDCap, etc.) in PATH → domain-expert.
- Otherwise → data-scientist (the safe default).

Auto-detection always asks one confirmation question before locking the tier in.

23. Interactive REPL

causalrag repl opens a Rich-powered interactive shell. The REPL is the natural workflow for the academic tier and for one-off exploratory analyses.

23.1 REPL features

- Tab completion on column names, hypothesis ids, command verbs.
- Live StudyProtocol panel at the top of the screen showing current dataset, queued hypotheses, and run estimates.
- Special % commands for narrative interactions:
 - %explain H03 — LLM narrates the chosen hypothesis in plain English.
 - %why_failed H05 — LLM explains why Step 5 (identifiability) rejected this hypothesis.
 - %what_if remove confounder X — sandboxed re-run showing the effect on the estimate.
 - %compare H03 H07 — side-by-side comparison of two hypothesis walks.
 - %critique — LLM critiques the current StudyProtocol for missing assumptions or weak choices.
- Auto-save: every command persists to the YAML protocol; the REPL never loses state on exit.

23.2 Example REPL session

```
$ causalrag repl
[ok] loaded my_study/study.causalrag.yaml — 47 columns, 10 hypotheses
```

```
(causalrag) %status
```

```
| Study: claims_readmission_2026 |
| Tier: academic                  |
| Dataset: claims_2024.parquet (n=14,231) |
| Hypotheses queued: 10 (run: 7 pending: 3) |
| Top finding: hospital_followup → readmit_30d |
| ATE = -0.087 [-0.121, -0.054] E-value = 1.94 (🟡) |
```

```
(causalrag) %explain H03
```

```
H03 asks whether scheduling a follow-up appointment within
7 days of discharge causally reduces 30-day readmission
for the cohort of Medicare patients age 65+. The estimand
is an ATE on a binary outcome under back-door adjustment by
age, prior admissions, Charlson index, and admission diagnosis.
```

```
(causalrag) %what_if remove confounder Charlson
```

Re-running H03 with Charlson removed from W ...
ATE shifts from -0.087 to -0.142 (CI now [-0.179, -0.105]).
Sign unchanged; magnitude increases. Indicates Charlson was
blocking a true negative confounder of effect.

```
(causalrag) sensitivity H03 --methods evaluate
```

E-value: 1.94 (covariate-strength scale)
An unmeasured confounder would need a risk-ratio association
of ≥ 1.94 with BOTH treatment and outcome to nullify this.

```
(causalrag) report --format html
```

Report written to reports/study_2026-05-13.html

24. Reporting Outputs

Reports are the analyst-facing artifact. They share a single underlying data model but render to four formats with format-specific affordances.

24.1 HTML report (default)

Single-file HTML built from Jinja2 templates with embedded SVG plots and CSS. Features:

- Collapsible Causal Roadmap sections (Steps 1–8) per hypothesis.
- Sortable findings table with traffic-light verdict column.
- Hover-tooltips on technical terms linking to the inline glossary.
- Search box for navigating large reports.
- Interactive multiverse plots (Plotly) where applicable.
- Decision ledger as a clickable diff.
- Provenance footer with all model digests, package versions, and seed.

24.2 PDF report

Typeset via WeasyPrint from the same HTML source. Suitable for sharing as a supplementary material, archive, or stakeholder briefing. Uses the same color palette and typography as the CausalRAG project documents (Charter / Helvetica). Includes a generated table of contents and page numbers.

24.3 Quarto report

For the academic tier. Emits a .qmd file with code blocks that re-execute against the saved StudyProtocol. Users can extend the report with custom analyses, add chunks, and render to HTML, PDF, or DOCX through Quarto's own toolchain.

24.4 Markdown report

Plain text for archival, LLM consumption, version control, and downstream automation. The Markdown report is the canonical form: HTML, PDF, and Quarto are all derived from it. Includes structured frontmatter with provenance keys.

24.5 Causal Model Card

Inspired by Model Cards (Mitchell et al. 2019) but specialized for causal claims, the Causal Model Card is a one-page summary attached to every report. It contains:

- The CausalQuestion in domain language.
- Point estimate, CI, and traffic-light robustness verdict.
- The three most consequential assumptions in plain language.
- Top three suspected unmeasured confounders and the E-value defense.

- The population the answer is intended to generalize to.
- Known limitations specific to the dataset and estimator.

The Card is what a clinician, policymaker, or product manager should read first — and often is the only thing they read. Designing for it forces the underlying analysis to be honest.

25. Packaging, Dependencies & Extras

CausalRoadmap is a single Python package with a generous set of optional extras. The base install is intentionally minimal so a user can run discovery, feasibility, and a single Python-native estimator without pulling the entire causal-inference ecosystem.

25.1 Python version & build tooling

- Python ≥ 3.11 required (TypeAlias, Self, performance gains; 3.10 is past EOL for new packages).
- Capped at Python 3.13 until rpy2 wheels are reliably available for 3.14+.
- Build backend: hatchling (via hatch) — modern PEP 517 backend, simple metadata.
- Dependency resolution: uv for fast local installs and CI; uv pip compile produces locked dev requirements.
- Lint/format: ruff (lint + format in one tool).
- Type-check: mypy --strict on public APIs; pyright in editor.

25.2 Extras strategy

The pyproject.toml defines extras grouped by concern. Users install only what they need; the base install is fast.

```
[project]
name = "causalrag"
requires-python = ">=3.11"
dependencies = [
    "numpy>=2.0", "pandas>=2.2", "pyarrow>=16",
    "scipy>=1.13", "scikit-learn>=1.5", "statsmodels>=0.14",
    "networkx>=3.3", "pydantic>=2.7",
    "typer[all]>=0.12", "rich>=13.7", "questionary>=2.0",
    "PyYAML>=6.0", "jinja2>=3.1",
    "dowhy>=0.12", "econml>=0.16",
    "psutil>=5.9",
]

[project.optional-dependencies]
llm      = ["ollama>=0.4", "httpx>=0.27"]
gpu      = ["pynvml>=11.5", "torch>=2.4"]
rbridge  = ["rpy2>=3.5"]
survival = ["lifelines>=0.30"]                # + R: grf, survRM2
discovery = ["causal-learn>=0.1.4", "lingam>=1.9",
             "tigramite>=5.2"]
ml       = ["causalml>=0.15", "doubleml>=0.9"]
sensitivity = ["PySensemakr>=0.1", "pymc>=5"]
viz      = ["matplotlib>=3.9", "plotly>=5.22",
```

```

report      = ["graphviz>=0.20"]
dev         = ["weasyprint>=62"]          # quarto-cli external
            = ["pytest>=8", "pytest-cov",
              "pytest-xdist", "hypothesis",
              "ruff", "mypy", "pyright", "syrupy"]
all         = ["causalrag[llm,gpu,rbridge,survival,discovery,
              ml,sensitivity,viz,report]"]

```

25.3 The R dependency

The [rbridge] extra installs rpy2 only; the R interpreter itself and the CRAN packages are out of band. Three supported install paths (also documented at §17.6):

- Native R install + manual `install.packages()` for `grf`, `lmtp`, `sensemakr`, `MatchIt`, `marginalEffects`, `mediation`, `survRM2`, `bnlearn`, `tlverse`.
- Doctor-assisted: `causalrag doctor --install-r-deps` invokes R from Python and runs `install.packages` for every missing dependency.
- Container: ship `Dockerfile.r` and `devcontainer.json` with R 4.4 + every required package preinstalled. This is the recommended path for users who don't want to manage R themselves.

26. Testing Strategy

Testing an LLM-dependent package reproducibly requires layered strategies. CausalRoadmap uses three layers.

26.1 Layer 1 — Cassette-based LLM tests

Every LLM call site has a fixture stored under `tests/fixtures/llm/<test_id>.json` containing the prompt and the canonical response. A `FakeOllamaClient` replays from fixtures during pytest. CI never touches a real LLM; tests are deterministic and fast (sub-second per test).

Conceptually similar to `vcrpy` but custom-built because Ollama's streaming response format needs structured replay.

26.2 Layer 2 — Golden-output snapshots

For the deterministic-after-LLM artifacts (`DiscoveryReport`, `FeasibilityReport`, `EstimationResult`), tests assert against syrupy snapshots. Updates require explicit `pytest --snapshot-update`; this prevents silent drift.

26.3 Layer 3 — Live LLM suite (opt-in)

`pytest -m live_llm` runs against a real Ollama backend with pinned model digests and seeds. Gated behind an `OLLAMA_HOST` environment variable; runs weekly in CI on a dedicated runner, not on every PR. This is the canary for model-version drift.

26.4 Property-based testing

Hypothesis (the Python library, not the causal kind) drives property tests for the `StudyProtocol` round-trip (`serialize` → `load` → `equality`), the `DataFlag` combinatorics (every estimator's required `n` excluded is empty), and the `Roadmap` step gating (every transition's preconditions hold for synthetic inputs).

26.5 Reproducibility tests

A dedicated reproducibility test suite runs the full pipeline twice on a small public dataset (Lalonde NSW or IHDP) with the same seed and asserts byte-equality on every artifact. Failures here block release.

27. Documentation

Documentation strategy aims for parity with the best in the ecosystem (DoWhy, EconML, tlverse), with an additional emphasis on honest discussion of limitations.

27.1 Documentation site

- MkDocs Material at causalroadmap.dev (or similar), with API reference auto-generated via mkdocstrings.
- Notebook-driven tutorials via mkdocs-jupyter, executed in CI to prevent rot.

27.2 Tutorial series

- 77. "Your first causal study in 5 minutes" — data-scientist tier, end-to-end on a public dataset.
- 78. "Reading a discovery report" — domain-expert tier, narrative walkthrough.
- 79. "Reproducing a published RCT secondary analysis" — academic tier.
- 80. "Survival HTE with Causal Survival Forests via rpy2".
- 81. "When the LLM is wrong: overriding the discovery agent".
- 82. "Stochastic interventions with lmtp".
- 83. "From hypothesis to publication-ready report".

27.3 Method-comparison vignettes

In the spirit of the tlverse handbook and DoWhy docs: same dataset analyzed by multiple methods, with discussion of where and why they diverge. Vignettes shipped at v0.5 cover ATE methods, CATE methods, survival methods, and sensitivity methods.

27.4 Honest limitations page

A first-class "Limitations & Failure Modes" page exists from v0.1 and grows with every release. This is the package's intellectual-honesty surface. It enumerates every way the LLM-assisted workflow can mislead, with examples and recommended mitigations. The page is linked from the Causal Model Card in every report.

27.5 Decision flowcharts

Visual decision flowcharts for the two highest-stakes choices analysts face:

- "Which estimator should I use?" — branches by data flags and estimand.
- "Which sensitivity analysis should I run?" — branches by estimator and what assumption is most suspect.

27.6 Glossary

An inline-linked glossary covers every term that appears in LLM-generated rationales. Each entry includes a one-sentence definition, an example, and a link to the most accessible primary reference.

28. CI/CD

Continuous integration uses GitHub Actions. The matrix balances coverage against cost.

28.1 PR-time jobs

- Lint + type-check (ruff + mypy).
- Unit tests across Linux × {Python 3.11, 3.12, 3.13}.
- Integration tests on Linux + Python 3.12 with the cassette-based LLM suite.
- One Windows job (3.12) and one macOS job (3.12) for OS-specific regressions.
- Documentation build (mkdocs build --strict).
- Snapshot diff check (alert on un-updated syrupy snapshots).

28.2 R-equipped CI job

A separate workflow runs on a Docker image preloaded with R 4.4 and the required CRAN packages. Triggered by changes to `causalrag/estimators/rbridge/**` or weekly on schedule. Tests the rbridge adapters against real R sessions.

28.3 Live LLM weekly run

Scheduled weekly: `pytest -m live_llm` on a self-hosted runner with Ollama installed and the recommended models pulled. Catches Ollama version drift, model-version drift, and any prompt-engineering brittleness that the cassettes mask.

28.4 Release workflow

Tagged releases trigger:

- Build wheels (hatch build), upload to PyPI.
- Build conda-forge feedstock PR (from v0.5 onward).
- Publish docs to `causalroadmap.dev`.
- Rebuild + push Docker images (Dockerfile, Dockerfile.r) to `ghcr.io`.
- Generate release notes from `CHANGELOG.md`.

28A. Error Hierarchy & Audit System — Full Specification

Section 28A through 28C specify the package's error classes, the exception flow through phases, and the audit log format. The architectural goal is twofold: every failure mode of an LLM-assisted causal pipeline becomes a named exception, and every analysis run produces an audit trail that is independently re-checkable against the artifacts.

28A.1 Exception hierarchy

```
# causalrag/audit/exceptions.py

class CausalRoadmapError(Exception):
    """Base for every package-defined error."""

# Dataset Lock and integrity
class LockMismatchError(CausalRoadmapError):
    """Dataset hash differs from the Lock recorded in the StudyProtocol."""
class PhantomVariableError(CausalRoadmapError):
    """Reference to a column not present in the Dataset Lock schema."""
class TemporalViolationError(CausalRoadmapError):
    """Confounder measured after treatment, or follow-up beyond observed window."""

# Identification and feasibility
class NonIdentifiableError(CausalRoadmapError):
    """DoWhy/Ananke identify_effect returns no identification expression."""
class OutOfSupportError(CausalRoadmapError):
    """Requested intervention level outside the Data Support Envelope."""
class PositivityViolationError(CausalRoadmapError):
    """Estimated propensity outside [trim_lo, trim_hi] for > threshold of units."""
class EmptyCellError(CausalRoadmapError):
    """Subgroup or treatment arm has n < min_n."""
class FeasibilityFailureError(CausalRoadmapError):
    """Phase 2 filter rejected this (treatment, outcome) pair."""

# LLM-related
class LLMSchemaError(CausalRoadmapError):
    """LLM output failed Pydantic schema validation after max retries."""
class LLMHallucinationError(CausalRoadmapError):
    """LLM proposed a variable, relation, or estimand contradicted by the data."""
class LLMVersionDriftError(CausalRoadmapError):
    """Recorded model digest differs from currently-loaded model."""

# R bridge
class RNotAvailableError(CausalRoadmapError):
    """R or rpy2 not importable."""
```

```

class RPackageMissingError(CausalRoadmapError):
    """Required CRAN package not installed in the R environment."""
class RRuntimeBridgeError(CausalRoadmapError):
    """R-side error raised during bridge call."""

# Audit aggregation
class AuditBlockingError(CausalRoadmapError):
    """One or more block-severity audit findings prevent report generation."""
    def __init__(self, findings: list["AuditFinding"]):
        self.findings = findings
        super().__init__(self._format())
    def _format(self) -> str:
        return "\n".join(f"[{f.guard}] {f.message}" for f in self.findings)

# Imputation and missing data
class ImputationCreepError(CausalRoadmapError):
    """Outcome value would be imputed at analysis time."""
class InformativeCensoringSuspectedError(CausalRoadmapError):
    """Censoring distribution depends on observed covariates."""

```

28A.2 Phase-by-phase exception flow

Phase	Exceptions that can arise	Default handling
Phase 0 (init)	RNotAvailableError, RPackageMissingError	Doctor offers --install-r-deps
Phase 1 (discover)	LockMismatchError, LLMSchemaError, LLMHallucinationError, PhantomVariableError	Block; show diff
Phase 2 (feasibility)	EmptyCellError, FeasibilityFailureError, PositivityViolationError	Warn-by-default; block in academic tier
Phase 3 (hypothesize)	PhantomVariableError, FeasibilityFailureError, LLMHallucinationError	Drop offending hypothesis from queue
Phase 4 Step 1-4	PhantomVariableError, TemporalViolationError, LLMSchemaError	Block hypothesis
Phase 4 Step 5	NonIdentifiableError, OutOfSupportError, PositivityViolationError	Block; route to §18E translator or --allow-nonidentifiable
Phase 4 Step 6-7	FeasibilityFailureError, RRuntimeBridgeError, ImputationCreepError	Block; recommend alternative estimator
Phase 4 Step 8	(none — interpretation can't fail; sensitivity might)	—
Phase 5 (sensitivity)	RPackageMissingError (rare path)	Skip method with warning
Phase 6 (report)	AuditBlockingError	Refuse to write report; print findings

28A.3 The audit log format

Every analysis writes to `.causalrag/audit.jsonl` in append-only mode. Each line is a self-contained JSON record.

```
{
  "timestamp": "2026-05-13T18:47:21Z",
  "event_type": "audit_finding", // or "phase_start", "phase_end", "llm_call", "r_call"
  "phase": "P4",
  "step": "5",
  "hypothesis_id": "H03",
  "guard": "G3",
  "severity": "block",
  "message": "Continuous intervention level 12.5 outside support; non_overlap=0.42",
  "lock_fingerprint": "e8a9b1f3c2d4e5f6a7b8c9d0e1f2a3b4",
  "llm_digest": "qwen3-14b-q4_k_m@sha256:abc...",
  "causalrag_version": "0.1.0",
  "runtime_hash": "... " // python + R + key package versions
}
```

The audit log is rendered into the report's provenance section. An external tool, `causalrag verify <study_directory>`, can replay the log against the artifacts and confirm consistency — a third-party reviewer can re-check the analysis end-to-end without rerunning estimation.

28B. The Verify Subcommand — Independent Audit Replay

causalrag verify is the package's answer to one of the hardest reproducibility problems: how does a third party confirm that the analysis as performed matches what the report describes? The verify subcommand consumes only the study directory and re-checks every audit finding, every fingerprint, every claim — without re-running estimation.

28B.1 What verify checks

- 84.Dataset Lock fingerprint matches the current dataset bytes.
- 85.Every EstimationResult's protocol_hash matches the StudyProtocol it claims to descend from.
- 86.Every audit log entry corresponds to an artifact in the study directory.
- 87.Every LLM cassette matches the prompt hash claimed in the result metadata.
- 88.Every R-bridged result includes the R sessionInfo it claimed to record.
- 89.The decision ledger's count of LLM suggestions matches the cassette count.
- 90.Every column referenced in the report's findings exists in the Dataset Lock schema.
- 91.No phase ran without the required guards passing.

28B.2 Verify implementation sketch

```
from causalrag.audit.verify import verify_study
from causalrag.audit.exceptions import VerificationError

def verify_study(study_dir: Path, *, strict: bool = True) -> VerificationReport:
    """Re-check a completed study end-to-end.

    Reads only persisted artifacts: study.causalrag.yaml, audit.jsonl,
    cassettes/, llm.lock, and EstimationResult files. Does not require
    Python packages used at runtime (e.g., grf) to be installed.
    """
    protocol = StudyProtocol.from_yaml(study_dir / "study.causalrag.yaml")
    lock = DatasetLock.from_yaml(study_dir / ".causalrag" / "dataset.lock.yaml")
    audit_log = list(jsonl_read(study_dir / ".causalrag" / "audit.jsonl"))
    cassettes = list((study_dir / ".causalrag" / "cassettes").glob("*.json"))
    findings: list[VerificationFinding] = []
    findings += _check_lock_match(lock, protocol)
    findings += _check_estimation_provenance(protocol, audit_log)
    findings += _check_cassette_completeness(audit_log, cassettes)
    findings += _check_decision_ledger(protocol, audit_log)
    if strict and any(f.severity == "fail" for f in findings):
        raise VerificationError(findings)
```

```
return VerificationReport(findings=findings)
```

28B.3 The verifier's perspective

A reviewer at a conference, a journal referee, or a regulator can pipe `causalrag verify ./study_dir | tee verification.log` and receive an itemized account of which claims are independently checkable against the artifacts. This is the closest the package gets to the cryptographic ideal of an end-to-end verifiable analysis — short of a full proof checker, which is out of scope for v1.0.

28C. Reproducibility Replay & Continuous Verification

Beyond verify, the package provides causalrag replay <protocol.yaml>, which re-runs the full pipeline from the StudyProtocol and asserts byte-equality on every artifact. Used in CI to catch silent drift from package updates.

```
def replay(protocol_path: Path, *, mode: Literal["strict","tolerant"] = "strict"):
    """Re-execute a study deterministically and assert artifact equality.

    `strict` mode requires byte-equal artifacts and identical audit log.
    `tolerant` mode allows numerical drift below 1e-9 on point estimates
    and warns on any divergence in the audit log.
    """
    protocol = StudyProtocol.from_yaml(protocol_path)
    # set seeds, pin model digests, reload cassettes ...
    rerun_study(protocol)
    # diff every artifact against the original ...
```

In CI: a dedicated workflow runs replay on a small public-dataset study after every PR. If any artifact diffs, the PR cannot be merged without explicit acknowledgment in the changelog.

29. Versioning Roadmap

Three milestones structure the release plan. Each milestone is defined by a feature set, not a calendar — but rough timing estimates are provided to anchor planning.

29.1 v0.1 — Foundational (target: 6–10 weeks from kickoff)

The minimum viable package. A user can run discovery on a real dataset, get a feasibility report, generate hypotheses, run the Causal Roadmap walk on one or more, and read a coherent HTML report.

- Core: StudyProtocol, CausalEstimator protocol, EstimationResult, CausalGraph, DataFlag system.
- Data connectors: CSV, Parquet, SQL (Postgres + SQLite via SQLAlchemy), JSON.
- Discovery agent: Stages 1a-1e on a single rank-1 DAG (no multi-DAG yet).
- LLM: Ollama client; hardware probe; auto-select with 16B floor; cassette infrastructure.
- Feasibility: power gate for binary-treatment × continuous/binary outcome; default + manual modes.
- Hypothesis: manual and automated modes; counterfactual ratio control; impact scoring.
- Roadmap walk: all 8 steps functional; Step 5 uses DoWhy's `identify_effect`.
- Estimators (Python only in v0.1): LinearDML, CausalForestDML, T/S/X/DR-learners, AIPW, IPW, PSM (Python `causalinfer` fallback).
- Discovery methods: PC, FCI, NOTEARS, DirectLiNGAM via `causal-learn`.
- Sensitivity: E-value, PySensemakr.
- CLI: `init`, `doctor`, `discover`, `feasibility`, `hypothesize`, `estimate`, `sensitivity`, `report`, `validate`, `run`.
- Report: HTML output with Roadmap-step collapse, findings table, decision ledger.
- Tests: cassette-based LLM suite; reproducibility check on one public dataset.
- Docs: getting-started + two tutorials; honest-limitations page.

29.2 v0.5 — Methodological breadth (target: +3–4 months after v0.1)

Public beta release. The R bridge unlocks the methodological frontier; multi-DAG analysis becomes the academic-tier default; survival becomes a first-class flag.

- rpy2 bridge: `grf` (causal forest + Causal Survival Forest), `Imtp`, `sensemakr` (R), `MatchIt`, `marginaleffects`.
- Multi-DAG discovery with K candidate DAGs ranked by consistency scorecard.
- Multiverse runner + specification-curve plots.
- Cross-estimator triangulation reporting.

- Survival pipeline: CSF, RMST contrast, IPCW-DML, survival meta-learners.
- Mediation: Imai-Keele via R, CMAverse.
- LLM-prior-augmented causal discovery (Jiralerspong-DFS, Vashishtha).
- Target-trial emulation scaffolding.
- Interactive REPL with the % command set.
- Quarto and PDF report formats.
- Bayesian sensitivity (PyMC bias-parameter priors).
- Rosenbaum bounds via R.
- conda-forge release in addition to PyPI.
- Docker images on ghcr.io including the R-equipped variant.

29.3 v1.0 — Production maturity (target: ~9–12 months after kickoff)

API stability commitment. Every estimator the package promises to support is in place; the plugin API is documented and used by at least one external package.

- tmle3 / sl3 bridge — TMLE for ATE, NIE, longitudinal.
- Bayesian Causal Forest (bartCause, BCF).
- Tigramite / PCMCi+ for time-series causal discovery.
- Full stochastic-intervention reporting matching Imtp's expressiveness.
- Optional cloud-LLM backend (OpenAI, Anthropic) behind explicit --online flag, with feature parity.
- Hugging Face Hub model selection alongside Ollama.
- Plugin API: third-party causalrag-<estimator> packages auto-discovered via entry points; at least one demonstration plugin in the wild.
- Comprehensive vignette set: ATE, CATE, survival, sensitivity, mediation, IV, target trial, stochastic interventions.
- v1.0 API stability commitment — no breaking changes without a major-version bump.

30. Differentiators

Why CausalRoadmap exists when DoWhy, EconML, and CausalML already exist. Each item below is a specific gap in the current ecosystem that this package closes.

- **End-to-end exploratory workflow.** DoWhy gives a user the four-step framework if they already know the DAG, the estimand, and the dataset structure. CausalRoadmap provides the zeroth and first steps: from raw CSV (or SQL or FHIR) to a candidate DAG and a curated hypothesis list. No competitor offers this in an integrated, Pythonic, local-LLM way.
- **Causal Roadmap as architectural backbone.** Every hypothesis walks the eight Petersen-van der Laan steps. No competing package enforces the scientific/statistical separation at the module level. This is the methodological commitment that makes the package defensible at venues like ACIC and SER.
- **Hardware-aware local LLMs.** Most LLM-causal demos assume a frontier API key. CausalRoadmap assumes Ollama, defaults to local, and auto-picks the best 16B+ model for the user's hardware. A hospital data scientist can run the full pipeline on their workstation without sending a single row off the machine.
- **R parity without R fluency.** rpy2-bridged adapters for grf, ltmcp, tmle3, sensemakr extensions, MatchIt, marginaleffects under a single Pythonic API. Solves the "I want Causal Survival Forest but I only know Python" problem that currently sends users to inferior Python alternatives.
- **Survival-first.** Most Python causal libraries treat survival as an afterthought. CausalRoadmap treats RIGHT_CENSORED_OUTCOME as a first-class flag that routes to canonical implementations from day one. Timely given the survival-method emphasis at recent ACIC programs.
- **Triangulation by default.** Multiverse + cross-DAG + cross-estimator robustness are baked into the reporting layer, not opt-in. The Lawlor tripartite recipe runs as the canonical robustness check for high-stakes findings.
- **Causal-aware data prepper.** The discovery agent is genuinely novel. Pandas-AI, Sketch, lida, and YData Profiling all describe data; none of them identifies treatments, confounders, mediators, colliders, IVs, and unmeasured-confounder candidates. The DataFlag system is the architectural payoff.
- **Hallucination-honest design.** Every LLM artifact is structurally validated, statistically checked, and surfaced with provenance. The three-layer gate from the original CausalRAG generalizes to four layers and applies everywhere the LLM touches the pipeline.

31. Failure Modes & Mitigations

The package is designed against specific known failure modes of LLM-assisted causal pipelines. Each failure mode below has a documented mitigation.

Failure mode	Mitigation
Causal Parrot effect (Zečević et al. 2023) — LLM parrots memorized causal facts rather than reasoning	Every LLM-suggested DAG edge must pass a conditional-independence test on the user's data before it is silently accepted in the rank-1 DAG. Disagreements surface to the analyst.
Post-treatment confounder bias — LLMs frequently propose post-treatment variables as confounders	Enforce temporal-order checks in Stage 1c and again in Step 4. Auto-relabel any post-treatment candidate as a mediator or collider candidate; never silently include.
Overconfident automation — analyst treats causalrag run output as final without review	Every report prominently shows LLM role inferences, override counts, and an analyst sign-off checklist. The one-shot run command is deliberately NOT called auto.
Hidden non-identifiability — analyst runs an estimator on a non-identifiable estimand	DoWhy's identify_effect runs for every hypothesis in Step 5. Failures block estimation by default; override requires --allow-nonidentifiable and is logged as red banner.
LLM version drift — same prompt produces different output across model versions	Model digest pinned in llm.lock; recorded in every EstimationResult; mismatch warning on subsequent runs. Cassette-based replay for tests.
R-version skew — different R or package versions silently change results	Every R-bridged EstimationResult records R version and packageVersion() metadata. Report provenance section surfaces these.
Multiple-testing inflation from LLM-generated hypotheses	BH correction applied by default across all hypotheses in a single study. Headline conclusions in the report refer to BH-adjusted values; unadjusted values shown alongside for transparency.
CausalProbe-style brittleness on novel domains — LLM hallucinates confidently on out-of-distribution data	Domain-tag-aware prompts following the G ² -Reasoner pattern; "LLM confidence is low on this domain" banner when column names appear out-of-distribution. Optional retrieval over a curated knowledge corpus.
Analyst abdication — silent acceptance of every default	Analyst-decision ledger records every default accepted and every LLM suggestion not overridden. Ledger is rendered prominently in the report (top of report in academic tier).

32. Naming Decision Process

This document uses the provisional name CausalRoadmap. The name signals tight coupling with the Petersen–van der Laan framework, which is intentional: the package's defensibility at methodological venues rests on that coupling. However the final name is decided before v0.1 PyPI release, with criteria below.

32.1 Decision criteria

- 92. Uniqueness on PyPI and major search surfaces. Required.
- 93. Signals methodological rigor without being cryptic to a first-time visitor.
- 94. Fits naturally into the PyWhy ecosystem if organizational alignment is pursued.
- 95. Does not pre-empt obvious naming for related projects (e.g., not so generic that a domain-specific spinoff would have to invent a worse name).
- 96. Domain-neutral — no oncology, no econometrics, no marketing flavor.

32.2 Final candidates carried forward

- CausalRoadmap (provisional default).
- causality-lab.
- pywhy-explorer (only if PyWhy buy-in is obtained).
- drcausal.
- causalkit.

32.3 Process

Before v0.1 release: PyPI namespace check, GitHub org namespace check, three-day notice to PyWhy maintainers if pywhy-explorer is the leading candidate, and a final commit pinning the chosen name in a NAMING.md decision document.

33. Concrete Next Steps — 4-Week Plan

The plan below is for one developer (Brennan) with the existing CausalRAG TCGA-BRCA codebase as starting reference material. Each week ends with a demoable artifact.

33.1 Week 1 — Skeleton & Protocol

- 97. Create new repo under chosen name; set up pyproject.toml with hatch, ruff, mypy, pytest; GitHub Actions matrix over Linux × {Python 3.11, 3.12} plus one macOS job.
- 98. Implement core/protocol.py — StudyProtocol, VariableSpec, Estimand, DataFlag — with full Pydantic models and YAML serialization. Property-based round-trip tests with Hypothesis.
- 99. Implement core/registry.py and the CausalEstimator Protocol. Port the LinearDML wrapper from the TCGA project as the first registered estimator — end-to-end smoke test.
- 100. Wire up the Typer CLI with init, doctor, validate (no LLM calls yet).
- 101. Week-1 demo: causalrag init && causalrag validate on a stub project.

33.2 Week 2 — LLM Plumbing & Discovery v0

- 102. Build llm/hardware.py with psutil + pynvml + Apple Silicon paths. Ship causalrag doctor that prints the detected profile and the recommended model tier.
- 103. Implement llm/ollama_client.py with deterministic seeding, JSON-schema retry loop, and model-digest capture. Mock with FakeOllamaClient for tests.
- 104. Implement discovery Stages 1a-1c end-to-end against a synthetic dataset and one public dataset (Lalonde NSW or IHDP). Produce a DiscoveryReport.
- 105. Add cassette-based LLM tests using captured fixtures.
- 106. Week-2 demo: causalrag discover lalonde.csv producing a structured DiscoveryReport.

33.3 Week 3 — Identification, Estimation, First Sensitivity

- 107. Implement Stage 1d (flag emission) and Stage 1e (domain expert brief). Integrate DoWhy's identify_effect for Step 5.
- 108. Add Python estimator wrappers for LinearDML, CausalForestDML, T/X/S/DR-learners. Standardize outputs through EstimationResult.
- 109. Generalize the three-layer hallucination gate from CausalRAG to four layers and apply to every LLM call site.
- 110. Add E-value and PySensemakr sensitivity wrappers.

111. Write the "Your first causal study in 5 minutes" tutorial; make it the README example.
112. Week-3 demo: a complete Roadmap walk on a public dataset producing an EstimationResult and sensitivity numbers.

33.4 Week 4 — Hypothesis Generation, Generalized Power, First Report

113. Implement hypothesize/generator.py with the strategic heuristics (coverage, power, novelty, no-post-treatment, counterfactual ratio). Re-use the oncology bounding-box code generalized as in §8.2.
114. Implement feasibility/power.py with MDE calculators for binary-ATE, continuous-ATE, subgroup-CATE. Defer RMST and IV-LATE to v0.5.
115. Build the HTML report (Jinja2 + Rich-rendered tables embedded as HTML). Include LLM provenance and analyst-decision ledger.
116. Cut v0.1.0 tag, push to TestPyPI, open a discussion thread on PyWhy Discord introducing the project.
117. Week-4 demo: causalrag run dataset.csv produces a complete HTML report including the eight-step Roadmap walk for each of N generated hypotheses.

33.5 Stretch goal

If time remains, begin rpy2 scaffolding for v0.5. Implement `_r.py`, ship a causalrag doctor --install-r-deps that pulls grf and sensemakr, plus a single proof-of-concept adapter for grf::causal_forest returning a standard EstimationResult. Even one bridged method proves the architecture.

33.6 Always-on practices from day one

- Every commit's CI runs the cassette LLM tests so contributors can develop without Ollama installed.
- Every PR updates CHANGELOG.md and the relevant tutorial if user-facing behavior changed.
- The "Limitations & Failure Modes" documentation page exists from v0.1 and grows with every release. This is the package's intellectual-honesty surface and a real differentiator.
- Every LLM call in production code has a cassette fixture in tests.
- Every estimator added gets an entry in the method catalog (§19) and a vignette by v0.5.

34. Glossary

Selected terms used throughout this document. The package ships a more extensive inline glossary in the documentation site.

Term	Definition
ATE / ATT / ATC	Average Treatment Effect / on the Treated / on the Controls. Marginal expectations of the potential-outcome contrast.
CATE	Conditional Average Treatment Effect — the ATE within strata defined by an effect modifier X .
RMST	Restricted Mean Survival Time — the expected survival up to a horizon τ ; the standard causal target for right-censored outcomes.
NDE / NIE	Natural Direct / Indirect Effect — the mediation decomposition of a total effect via a mediator M .
LATE	Local Average Treatment Effect — the ATE among compliers in an instrumental-variable setting.
DAG	Directed Acyclic Graph — the canonical encoding of causal assumptions.
NPSEM	Non-Parametric Structural Equation Model — a list of structural equations specifying the data-generating process without parametric form.
Identifiability	The mathematical property that a causal estimand can be computed from observed data under stated assumptions.
Positivity / Overlap	The condition that all units have non-zero probability of every treatment level conditional on covariates.
Exchangeability	Synonym for unconfoundedness; potential outcomes are independent of treatment assignment given covariates.
SUTVA	Stable Unit Treatment Value Assumption — no interference between units; consistency of treatment definition.
Consistency (causal)	Observed outcome under $T=a$ equals the potential outcome $Y(a)$. Distinct from statistical consistency.
DML / Double ML	Double / Debiased Machine Learning — Chernozhukov et al. 2018 framework for orthogonal estimation with ML nuisance.
TMLE	Targeted Maximum Likelihood Estimation — efficient, doubly-robust ATE estimation.
Imtp	Longitudinal Modified Treatment Policies — Díaz-Williams framework for stochastic interventions on any exposure type.
Causal Forest	Athey-Wager nonparametric CATE estimator using generalized random forests.
Causal Survival Forest	Cui et al. 2023 extension of causal forest to right-censored outcomes.
sensemkr	Cinelli-Hazlett partial- R^2 omitted-variable-bias sensitivity framework.
E-value	VanderWeele-Ding minimum confounder strength required to nullify an observed effect.
Target Trial Emulation	Hernán-Robins framework for designing an observational analysis to mimic the protocol of an ideal RCT.

Term	Definition
Multiverse Analysis	Specification curve over reasonable analytic choices to assess robustness.
Triangulation	Lawlor et al. 2016 robustness check via independent methods that depend on different assumptions.
Ollama	Local LLM server that pulls GGUF-quantized open-weights models and serves them via a REST API.
GGUF / Q4_K_M / Q8_0	Quantized model file formats; suffix denotes compression aggressiveness (Q4 $\approx$ 4-bit; Q8 $\approx$ 8-bit).
rpy2	Python library embedding R into the Python process; the foundation of the R bridge.
FeasibilityReport	The Phase 2 artifact: list of admissible (treatment, outcome) pairs with their MDE and power.
StudyProtocol	The central serializable object holding the entire analysis state.
DataFlag	Routing-relevant property of the data or analysis that determines which methods are applicable.
CausalEstimator	The Protocol implemented by every estimator, Python or R-bridged.

35. References

Selected references cited or implicitly invoked in this document. The package documentation site ships a complete annotated bibliography.

35.1 Frameworks

- Petersen, M. L. & van der Laan, M. J. (2014). Causal Models and Learning from Data: Integrating Causal Modeling and Statistical Estimation. *Epidemiology* 25(3), 418–426.
- Hernán, M. A. & Robins, J. M. (2016). Using big data to emulate a target trial when a randomized trial is not available. *American Journal of Epidemiology* 183(8), 758–764.
- Pearl, J. (2009). *Causality: Models, Reasoning, and Inference* (2nd ed.). Cambridge University Press.

35.2 Estimation

- Chernozhukov, V., Chetverikov, D., Demirer, M., Duflo, E., Hansen, C., Newey, W., & Robins, J. (2018). Double/debiased machine learning for treatment and structural parameters. *Econometrics Journal* 21(1), C1–C68.
- Athey, S., Tibshirani, J., & Wager, S. (2019). Generalized random forests. *Annals of Statistics* 47(2), 1148–1178.
- Cui, Y., Kosorok, M. R., Sverdrup, E., Wager, S., & Zhu, R. (2023). Estimating heterogeneous treatment effects with right-censored data via causal survival forests. *Journal of the Royal Statistical Society B* 85(2), 179–211.
- Díaz, I., Williams, N., Hoffman, K. L., & Schenck, E. J. (2021). Non-parametric causal effects based on longitudinal modified treatment policies. *Journal of the American Statistical Association* 118(542), 846–857.
- Nie, X. & Wager, S. (2021). Quasi-oracle estimation of heterogeneous treatment effects. *Biometrika* 108(2), 299–319.
- Künzel, S. R., Sekhon, J. S., Bickel, P. J., & Yu, B. (2019). Metalearners for estimating heterogeneous treatment effects using machine learning. *PNAS* 116(10), 4156–4165.

35.3 Sensitivity

- VanderWeele, T. J. & Ding, P. (2017). Sensitivity analysis in observational research: introducing the E-value. *Annals of Internal Medicine* 167(4), 268–274.
- Cinelli, C. & Hazlett, C. (2020). Making sense of sensitivity: extending omitted variable bias. *Journal of the Royal Statistical Society B* 82(1), 39–67.
- Chernozhukov, V., Cinelli, C., Newey, W., Sharma, A., & Syrgkanis, V. (2022). Long story short: omitted variable bias in causal machine learning. NBER Working Paper 30302.

- Rosenbaum, P. R. (2002). *Observational Studies* (2nd ed.). Springer.
- Lawlor, D. A., Tilling, K., & Davey Smith, G. (2016). Triangulation in aetiological epidemiology. *International Journal of Epidemiology* 45(6), 1866–1886.
- Simonsohn, U., Simmons, J. P., & Nelson, L. D. (2020). Specification curve analysis. *Nature Human Behaviour* 4(11), 1208–1214.

35.4 LLMs and causal inference

- Kıcıman, E., Ness, R., Sharma, A., & Tan, C. (2024). Causal reasoning and large language models: opening a new frontier for causality. arXiv:2305.00050v3.
- Zečević, M., Willig, M., Dhami, D. S., & Kersting, K. (2023). Causal parrots: large language models may talk causality but are not causal. *Transactions on Machine Learning Research*.
- Chi, H. et al. (2024). Unveiling causal reasoning in large language models: reality or mirage? (CausalProbe-2024). *NeurIPS 2024*.
- Wang, H., Han, X., Singh, R., Ma, T., & Chaudhary, V. (2025). CausalRAG: Integrating causal graphs into retrieval-augmented generation. *Findings of ACL 2025*.
- Joshi, N. et al. (2024). LLMs are prone to fallacies in causal inference. *EMNLP 2024*.
- Jiralerspong, M. et al. (2024). Efficient causal graph discovery using large language models.
- Liu, X. et al. (2025). Quantization hurts reasoning? An empirical study on quantized reasoning models. arXiv:2504.04823.

35.5 Software

- Sharma, A. & Kıcıman, E. (2020). DoWhy: an end-to-end library for causal inference. arXiv:2011.04216.
- Battocchi, K. et al. (2019). EconML: a Python package for ML-based heterogeneous treatment effects estimation.
- Zheng, Y. et al. (2024). causal-learn: causal discovery in Python. *Journal of Machine Learning Research* 25(60), 1–8.
- Ikeuchi, T. et al. (2023). Python package for causal discovery based on LiNGAM. *JMLR* 24(14), 1–8.
- Brodersen, K. H. et al. (2015). Inferring causal impact using Bayesian structural time-series models. *Annals of Applied Statistics* 9(1), 247–274.
- Coyle, J. R., Hejazi, N. S., Malenica, I., Phillips, R. V., & Sofrygin, O. (2022). *tlverse: an ecosystem for targeted learning in R*.

33A. Multi-Dataset Federation — Architectural Sketch (v2.0+)

Section 33A through 33D sketch the architecture for multi-dataset causal inference — analyses spanning two or more datasets that contain similar but not identical data. Federation is a v2.0+ feature; v0.1 through v1.0 lock the analysis to a single dataset by design. This section exists to document the architectural hooks already present in v1.0 that make federation viable, and to record the methodological commitments before they have to be implemented.

33A.1 Motivating use cases

- Multi-site electronic health records — the same data fields collected across hospitals, with site as a potential effect modifier or source of confounding.
- Pooled analysis of multiple randomized trials — combining trial datasets to estimate effects in a target population not perfectly represented by any single trial.
- Real-world evidence (RWE) bundling — registry data + claims data + EHR for the same population, with each source contributing partial information.
- Transportability — answering whether an effect estimated in one population is portable to another with different baseline characteristics.
- Cross-jurisdictional policy analysis — same intervention applied in different regions, with regional context as confounding or moderating.

33A.2 What is NOT federation in our terminology

To avoid the term-overloading common in this area, the package draws clean lines:

- Single-dataset multi-cohort stratification (e.g., subgroups within one EHR) is NOT federation — handled in v0.1 via CATE.
- Meta-analysis of published effect estimates is NOT federation — handled in v0.1 via the ensemble aggregator.
- Federated learning in the ML sense (privacy-preserving model training with no data movement) is a specific privacy pattern within federation — addressed in §33C below as one option among several.

33B. Federation Methodological Patterns

The literature distinguishes three patterns; v2.0 will support all three.

33B.1 Pattern A — Centralized pooled analysis with site indicator

The simplest pattern: all datasets are loaded into a single analysis frame, with site as a covariate. Treats site as either a confounder (when site predicts treatment and outcome) or an effect modifier (when site moderates the treatment effect). Tools: existing CausalRoadmap pipeline + a SITE_INDICATOR data flag.

Limitations: requires full data movement, ignores between-site heterogeneity in confounding structure, requires single-version schema.

33B.2 Pattern B — Federated propensity score aggregation

Each site fits its own propensity model; the global propensity is constructed as a mixture using membership weights (probability of site given covariates). The methodological reference is Xiong et al. 2023 and Guo et al. 2025; the recent 2026 extension by McClean and Díaz addresses the longitudinal case. The package's architectural commitment: every site exposes a Site Adapter that materializes its own propensities and density-ratio weights, but never its raw rows.

```
from causalrag.federation import SiteAdapter, FederatedStudy

class SiteAdapter:
    """Local handle for one site's data in a federated study.

    Exposes a uniform interface so the orchestrator never sees raw rows.
    Each site instantiates its own SiteAdapter and registers it with the
    federation hub. The hub computes only summary statistics, propensity
    parameters, and influence-curve aggregations.
    """
    site_id: str
    lock_fingerprint: str          # local Dataset Lock
    schema_alignment: dict[str, str] # local-name -> canonical-name

    def fit_local_propensity(self, treatment, confounders) -> PropensityFit:
        ...
    def fit_local_outcome(self, treatment, outcome, confounders) -> OutcomeFit:
        ...
    def influence_curve_contribution(self, estimand) -> np.ndarray:
        ...
    def site_membership_score(self, query_covariates) -> np.ndarray:
        ...

class FederatedStudy:
    """Orchestrator that aggregates SiteAdapter outputs."""
    def __init__(self, adapters: list[SiteAdapter], target_population: dict):
```

```
...  
def federated_aipw(self) -> EstimationResult:  
    # collect local AIPW psi_i from each site, aggregate ...  
...
```

33B.3 Pattern C — Transportability with target population

The package's most ambitious federation pattern: combining one or more source datasets (often randomized trials with high internal validity) with a target-population dataset (often observational, with the covariate distribution of interest). The methodological literature is anchored by Dahabreh, Robertson, Stuart, Steingrimsson, and Hernán; the 2024 Annual Review of Statistics review by Degtiar & Rose is the standard reference.

Estimands supported in v2.0: target-population ATE, transportability estimand under S-admissibility, target-population CATE, and target-population RMST. The package will refuse to compute these without an explicit target-population covariate distribution and an S-admissibility assumption registered in the StudyProtocol.

33B.4 Federated target trial emulation (FL-TTE)

Wu et al. 2025 (npj Digital Medicine) introduces FL-TTE — federated learning + target trial emulation — validated on Sepsis trials across 192 hospitals using eICU + MIMIC-IV, and Alzheimer trials across five NYC health systems. The package's v2.0 will integrate this pattern as a high-level workflow: each site runs a local target-trial emulation, and the federation hub aggregates via influence-curve summation. Reference implementation: open-source FL-TTE codebase.

33C. Privacy, Governance & Communication

Federation in the regulated-data settings the package targets (clinical, financial, education) is fundamentally a privacy and governance problem, not just a statistical one. The architectural commitments:

- No raw row movement by default. Sites expose only summary statistics, influence-curve contributions, and propensity parameters. Raw data is opt-in per site.
- Differential privacy hooks. The SiteAdapter can be configured with a per-query privacy budget; aggregated outputs are noised to satisfy (ϵ, δ) -DP. Reference: opacus or pydp for the noise layer.
- Schema alignment with cryptographic proof. Each site signs its schema alignment certificate, and the hub refuses sites whose canonical-mapping checksums don't match the federation's registered schema.
- Audit log distribution. Each site's audit.jsonl is mirrored to the hub; the hub's verify subcommand replays the joint audit log.

33D. Why Federation Is v2.0, Not v1.0

Federation multiplies every difficulty already present in single-dataset causal inference. Specifically:

118. Three-way schema alignment (site → canonical → estimand) is hard even with LLM assistance, and the failure modes are subtler — a column that means "hemoglobin A1c percent" at one site can mean "glycated hemoglobin units undefined" at another. LLM-based schema reconciliation is an active research area; we wait for it to mature.
119. Privacy regulations vary by jurisdiction (HIPAA, GDPR, PIPEDA, etc.) and the package would need explicit per-jurisdiction modes.
120. Federated identification — the assumption set required for a federated estimand is strictly stronger than the single-dataset analogue (typically requires S-admissibility plus per-site positivity).
121. Operationally, federation needs an orchestrator service running across organizational boundaries — a deployment pattern fundamentally different from the local-only stance of v0.1 through v1.0.

33D.1 v1.0 hooks that make v2.0 viable

Even though federation is post-v1.0, the v0.1–v1.0 architecture is deliberately designed not to preclude it:

- StudyProtocol is dataset-locked, but the lock is a fingerprint, not the dataset — multi-site StudyProtocols just carry multiple lock fingerprints.
- EstimationResult carries an influence-curve field that aggregates additively across sites.
- The CausalEstimator protocol's fit/estimate split lets a SiteAdapter wrap any v1.0 estimator without modification.
- The audit log format is site-namespaced, so multi-site audit logs are mergeable by design.
- The Data Support Envelope generalizes naturally to a federated envelope — the intersection of per-site envelopes.

These design decisions are explicit in v1.0 even though no v1.0 user will exercise them, because reversing them after release would break backwards compatibility.